Chapter 1. Introduction

DFSMS provides utility programs to assist you in organizing and maintaining data. Utilities are simple programs that perform commonly needed functions. “Guide to utility program functions” on page 1 will help you find the program that performs the function that you need.

Guide to utility program functions

You can use the DFSMS utility programs to perform a variety of tasks, as shown in Table 1 on page 1. The "Task" column shows tasks that you may want to perform. The "Options" column more specifically defines the tasks. The "Primary Utility" column identifies the utility that is especially suited for the task. The "Secondary Utilities" column identifies other utilities that can be used to perform the task.

Table 1. Tasks and utility programs

Task	Options	Primary Utility	Secondary Utilities
Add	a member to a partitioned data set	IEBUPDATE, IEBGENER	IEBDG
	a password	IEHPROGM	
Alter in place	a load module	IEBCOPY	
Catalog	a data set in a catalog	IEHPROGM	
Change	data set organization	IEBUPDTE	IEBGENER, IEBPTPCH
	logical record length	IEBGENER	
Compare	z/OS UNIX System Services (z/OS UNIX) files such as HFS files	IEBCOMPR	
	partitioned data sets	IEBCOMPR	
	sequential data sets	IEBCOMPR	
	PDSEs	IEBCOMPR	
Compress	a partitioned data set	IEBCOPY	
Compress in place	a partitioned data set	IEBCOPY	
Convert to partitioned data set	an unloaded PDSE containing program objects cannot be loaded into a PDS. An unloaded PDSE containing data objects can be loaded into a PDS but all extended attributes will be lost.	IEBCOPY	
	sequential data sets	IEBGENER	IEBUPDTE
	a PDSE	IEBCOPY	
Convert to PDSE	a partitioned data set	IEBCOPY	
	an unloaded copy of a partitioned data set or PDSE	IEBCOPY	
	sequential data sets	IEBGENER	IEBUPDTE
Convert to sequential data set	a partitioned data set or PDSE	IEBGENER	IEBUPDTE

© Copyright IBM Corp. 1979, 2026

Table 1. Tasks and utility programs (continued)

Task	Options	Primary Utility	Secondary Utilities
Copy	a load module or load module library	IEBCOPY	
	a partitioned data set	IEBCOPY	IEHMOVE
	a volume of data sets (on tape or disk)	IEHMOVE	
	job steps	IEBEDIT	
	selected members of a partitioned data set	IEBCOPY	IEHMOVE
	sequential data sets	IEBGENER	IEHMOVE, IEBUPDTE, IEBPTPCH
	a PDSE	IEBCOPY	
	a group of PDSE members	IEBCOPY	
	selected members of a PDSE	IEBCOPY	
Create	a backup copy of a partitioned data set or PDSE	IEBCOPY	
	a character arrangement table module	IEBIMAGE	
	a copy modification module	IEBIMAGE	
	a 3800 or 4248 forms control buffer module	IEBIMAGE	
	a graphic character modification module	IEBIMAGE	
	a library character set module	IEBIMAGE	
	a library of partitioned members	IEBGENER	IEBUPDTE
	a member of a partitioned data set or PDSE	IEBGENER	IEBDG, IEBUPDTE
	a sequential output data set	IEBDG	IEBGENER, IEBPTPCH
	an indexed sequential data set	IEBDG	
	an output job stream	IEBEDIT	
Delete	a data set or member of a partitioned data set	IEHPROGM	
	password	IEHPROGM	
	catalog entries	IEHPROGM	
	records in a partitioned data set or PDSE member	IEBUPDTE	
Edit and convert to partitioned data set or PDSE	a sequential data set	IEBGENER	IEBUPDTE
Edit and copy	a job stream	IEBEDIT	
	a sequential data set	IEBGENER	IEBUPDTE, IEBPTPCH
Edit and list	error statistics by volume (ESV) records	IFHSTATR	
Edit and print	a sequential data set	IEBPTPCH	IEBGENER
Edit and punch	a sequential data set	IEBPTPCH	IEBGENER
Enter	a procedure into a procedure library	IEBUPDTE	
Exclude	a partitioned data set member from a copy operation	IEBCOPY	IEHMOVE
	a PDSE member from a copy operation	IEBCOPY	

Table 1. Tasks and utility programs (continued)

Task	Options	Primary Utility	Secondary Utilities
Expand	a partitioned data set or PDSE	IEBCOPY	
	a sequential data set	IEBGENER	
Generate	test data	IEBDG	
Include	changes to members or sequential data sets	IEBUPDTE	
	a partitioned data set member from a copy operation	IEBCOPY	IEHMOVE
	a PDSE member from a copy operation	IEBCOPY	
Indicate	double-byte character set string by supplying enclosing shift-out/shift-in characters	IEBGENER	IEBPTPCH
Insert records	into a partitioned data set or PDSE	IEBUPDTE	
Label	magnetic tape volumes	IEHINITT	
List	a password entry	IEHPROGM	
	a volume table of contents	IEHLIST	
	number of unused directory blocks and tracks	IEHLIST	IEBCOPY
	partitioned data set or PDSE directories	IEHLIST	IEHPROGM
	CVOL entries	IEHLIST	
Load	an unloaded partitioned data set to a partitioned data set	IEBCOPY	
	an unloaded data set	IEHMOVE	
	an unloaded partitioned data set to a PDSE (for non-load modules only)	IEBCOPY	
	an unloaded PDSE to a partitioned data set (for non-load modules only)	IEBCOPY	
	an unloaded PDSE to a PDSE	IEBCOPY	
Merge	partitioned data sets	IEBCOPY	IEHMOVE
	PDSEs	IEBCOPY	
	partitioned data sets and PDSEs	IEBCOPY	
Modify	a partitioned or sequential data set, or a PDSE	IEBUPDTE	
Move	a volume of data sets	IEHMOVE	
	partitioned data sets	IEHMOVE	
	sequential data sets	IEHMOVE	
Number records	in a new or old member of a partitioned data set or PDSE	IEBUPDTE	
Password protection	add a password	IEHPROGM	
	delete a password	IEHPROGM	
	list passwords	IEHPROGM	
	replace a password	IEHPROGM	

Table 1. Tasks and utility programs (continued)

Task	Options	Primary Utility	Secondary Utilities
Print	sequential data sets	IEBPTPCH	IEBGENER, IEBUPDTE
	partitioned data sets or PDSEs	IEBPTPCH	
	selected records	IEBPTPCH	
	mixed strings of double-byte and single-byte character set data	IEBPTPCH	IEBGENER
	double-byte character set data	IEBPTPCH	IEBGENER
Punch	a partitioned data set member	IEBPTPCH	
	a sequential data set	IEBPTPCH	
	selected records	IEBPTPCH	
	mixed strings of double-byte and single-byte character set data	IEBPTPCH	IEBGENER
	Double-byte character set data	IEBPTPCH	IEBGENER
Reblock	a load module	IEBCOPY	
	a partitioned data set or PDSE	IEBCOPY	
	a sequential data set	IEBGENER	IEBUPDTE
Re-create	a partitioned data set or PDSE	IEBCOPY	
Rename	member of a partitioned data set or PDSE	IEBCOPY	IEHPROGM
	a sequential or partitioned data set, or PDSE	IEHPROGM	
	moved or copied members of a partitioned data set	IEHMOVE	
Renumber	logical records	IEBUPDTE	
Remove	indication of a double-byte character set string by stripping off enclosing shift-out/shift-in characters	IEBGENER	
Replace	a password	IEHPROGM	
	logical records	IEBUPDTE	
	records in a member of a partitioned data set or PDSE	IEBUPDTE	
	selected members of a PDSE	IEBCOPY	IEBUPDTE
	selected members of a partitioned data set	IEBCOPY	IEBUPDTE, IEHMOVE
Scratch	data sets	IEHPROGM	
Uncatalog	data sets	IEHPROGM	
Unload	a partitioned data set	IEBCOPY	IEHMOVE
	a sequential data set	IEHMOVE	
	a PDSE	IEBCOPY	
Update in place	a partitioned data set or PDSE	IEBUPDTE	

System utility programs

System utility programs are used to list or change information that is related to data sets and volumes, such as data set names, catalog entries, and volume labels. Most functions that system utility programs can perform are performed more efficiently with other programs, such as IDCAMS, ISMF, or DFSMSrmm.

Table 2 on page 5 is a list of system utility programs and their purpose.

Table 2. System Utility Programs

System Utility	Alternate Program	Purpose
***IEHINITT**	DFSMSrmm EDGINERS	To write standard labels on tape volumes
IEHLIST	ISMF, PDF 3.4	To list system control data
***IEHMOVE**	DFSMSdss, IEBCOPY	To move or copy collections of data
IEHPROGM	Access method services, PDF 3.2	To build and maintain system control data
***IFHSTATR**	DFSMSrmm, EREP	To select, format, and write information about tape errors from the IFASMFDP tape

*These programs provide functions that are better performed by newer applications, such as ISMF or DFSMSrmm or DFSMSdss. IBM continues to ship these programs for compatibility with the *supported* older system levels.

The system utilities listed next support user data sets allocated with any valid combination of the following dynamic allocation options: XTIOT, UCB NOCAPTURE, and DSAB above the line.

- IEHINITT
- IEHLIST
- IEHPROGM

The system utilities listed next do not support user data sets allocated with any of the following dynamic allocation options: XTIOT, UCB NOCAPTURE, and DSAB above the line.

- IEHMOVE
- IFHSTATR

Data set utility programs

You can use data set utility programs to reorganize, change, or compare data at the data set or record level. These programs are controlled by JCL statements and utility control statements.

These utilities allow you to manipulate partitioned, sequential or indexed sequential data sets, or partitioned data sets extended (PDSEs), which are provided as input to the programs. You can manipulate data ranging from fields within a logical record to entire data sets.

The data set utilities included in this publication cannot be used with VSAM data sets. Information about VSAM data sets can be found in *z/OS DFSMS Using Data Sets*.

Table 3 on page 5 is a list of data set utility programs and their use.

Table 3. Data set utility programs

Data Set Utility	Use
***IEBCOMPR, SuperC, (PDF 3.12)**	Compare records in sequential or partitioned data sets, or PDSEs
IEBCOPY	Copy, compress, or merge partitioned data sets or PDSEs; add RLD count information to load modules; select or exclude specified members in a copy operation; rename or replace selected members of partitioned data sets or PDSEs
IEBDG	Create a test data set consisting of patterned data

Table 3. Data set utility programs (continued)

Data Set Utility	Use
IEBEDIT	Selectively copy job steps and their associated JOB statements
IEBGENER or ICEGENER	Copy records from a sequential data set or convert a data set from sequential organization to partitioned organization
IEBIMAGE	Modify, print, or link modules for use with the IBM 3800 Printing Subsystem, the IBM 3262 Model 5, or the 4248 printer
IEBPTPCH or PDF 3.1 or 3.6	Print or punch records in a sequential or partitioned data set
IEBUPDTE	Incorporate changes to sequential or partitioned data sets, or PDSEs

*These programs provide functions that are better performed by newer applications, such as ISMF or DFSMSrmm or DFSMSdss. IBM continues to ship these programs for compatibility with the *supported* older system levels.

The data set utilities listed next support user data sets allocated with any valid combination of the following dynamic allocation options: XTIOT, UCB NOCAPTURE, and DSAB above the line.

- IEBCOPY
- IEBGENER

The data set utilities listed next do not support user data sets allocated with any of the following dynamic allocation options: XTIOT, UCB NOCAPTURE, and DSAB above the line.

- IEBCOMPR
- IEBDG
- IEBEDIT
- IEBIMAGE
- IEBPTPCH
- IEBUPDTE

Control

System and data set utility programs are controlled by job control and utility control statements. The job control and utility control statements necessary to use utility programs are provided in the major discussion of each utility program.

Job Control Statements

You can start a system or data set utility program in the following ways:

- Place job control statements in a file and give the file to JES to run, for example, by the TSO SUBMIT command.
- Place job control statements, placed in a procedure library and run them with the MVS operator START command or include them in a JOB with the EXEC job control statement.
- Use TSO CALL command.
- Use another program which uses the CALL, LINK, or ATTACH macro.

Most JCL examples shown in this publication specify parameters used in locating uncataloged data sets. With cataloged data sets, the UNIT and VOL=SER parameters are not necessary.

Related reading:

- For information about allocating SMS-managed data sets, see *z/OS DFSMS Using Data Sets*.
- For information about coding JCL statements, see *z/OS MVS JCL Reference*.

Sharing data sets

Except for VSAM data sets or PDSEs, a data set cannot be updated by more than one job or user at a time without the risk of damaging the data set. Some data sets, particularly system data sets (identified by "SYS1"), are always in use. In order to safely update shared data sets, all but one user must stop updating the data set. After the update is finished, all users will have to re-access the data set. Re-accessing the data set is a function of the program using the data set and may involve closing and reopening the data set, or even freeing and reallocating the data set. Not all programs may be capable of doing this, so it is not always possible to safely update a shared data set.Unfortunately, this serialization mechanism does distinguish between data sets with the same name on different volumes.

You can use the DISP parameter on the DD statement, or the TSO ALLOCATE command, or the equivalent text unit for dynamic allocation (SVC 99) to put a lock on a data set so that you can update the data set. Specify DISP=OLD or DISP=MOD whenever you update a data set other than a PDSE.

If you code DISP=SHR in your JCL or the equivalent on the TSO ALLOCATE command or dynamic allocation, realize that the data set you are updating may be simultaneously updated by another user, resulting in an unusable data set, unless it is a PDSE. Another exception is a PDS. If a second program within the GRSplex tries to open the shared PDS for output (not the update in place option), that program gets a 213 ABEND. This detection of a violation of sharing protocol works even with two data sets with the same name on different volumes or two data sets with the same name and volume serial but on different systems. For these two cases the system does not issue ABEND 213.

This problem has been addressed by some components such as program management (binder, linkage editor), MVS allocation (JCL, SVC 99) and ISPF/PDF. Each component provides its own separate interlock and none of them recognize all the other interlocks. Therefore there is no totally safe way to update a data set allocation with DISP=SHR.

Partitioned data sets further complicate sharing because they have a directory and individual members. These sub-parts are generally protected inside the system ISPF/PDF does provide good protection against changes to members and the directory made by other ISPF/PDF user).

PDSE (partitioned data sets extended) are designed to avoid sharing problems. Consider using them in place of partitioned data sets.

When a volume is shared between unlike operating systems (such as between an MVS system and a VM system with shared DASD), DD statements or TSO ALLOC commands may not be able to stop a volume from being simultaneously updated from the two different systems.

Related reading: For more information about sharing data sets see *z/OS DFSMS Using Data Sets*.

Utility control statements

Utility control statements are used to identify a particular function to be performed by a utility program and, when required, to identify specific volumes or data sets to be processed. The utility control statements that a particular utility uses are discussed in the topic for that utility.

Utility control statements are usually included in the input stream. However, they may also be placed in a sequential data set, in a member of a partitioned data set or PDSE, or in a z/OS UNIX System Services (z/OS UNIX) file such as a HFS file. In any case, the data set must have fixed or fixed blocked records with a logical record length of 80. For a z/OS UNIX file, the records can have various lengths; code FILEDATA=TEXT on the DD statement.

Exception: Some utilities allow exceptions to these rules.

The control statements for the utility programs have the following standard format:

label operation operand comments

The *label* symbolically identifies the control statement and, with the exception of system utility program IEHINITT, can be omitted. When included, a name must begin in the first position of the statement and must be followed by one or more blanks. The label can contain from 1 to 8 alphanumeric characters. IEBUPDTE control statements are an exception to this rule. They begin with "./" in positions 1 and 2, with an optional label beginning in position 3.

The *operation* identifies the type of control statement. It must be preceded and followed by one or more blanks.

The *operand* is made up of one or more keyword parameters, separated by commas. The operand field must be preceded and followed by one or more blanks. Commas, parentheses, and blanks can be used only as delimiting characters.

Comments can be written in a utility statement, but they must be separated from the last parameter of the operand field by one or more blanks.

Continuing utility control statements

Utility control statements are contained in columns 1 through 71. A statement that exceeds 71 characters must be continued on one or more additional records. A nonblank character must be placed in column 72 to indicate continuation.

Exception: Some utilities allow exceptions to this rule. In those cases, a utility statement can be interrupted either in column 71 or after any comma.

The continued portion of the utility control statement must begin in column 16 of the following record.

Exception: The IEBPTPCH and the IEBGENER utility programs permit certain exceptions to these requirements.

Related reading: For more information, see:

- For information about the IEBPTPCH utility, see Chapter 11, “IEBPTPCH (Print-Punch) Program,” on page 225,
- For information about the IEBGENER utility, see Chapter 7, “IEBGENER (Sequential Copy/Generate Data Set) Program,” on page 149.

Restrictions

- Utility control statements do not support temporary data set names that begin with an ampersand. You can code the complete name generated for the data set by the system (for example, DSNAME=SYS95296.T000051.RP001.JOBTEMP.TEMPMOD). For utilities where you identify data sets only on DD statements or the dynamic allocation equivalent, you can use a temporary data set name that begins with an ampersand.
- The utility programs described in this publication do not normally support VSAM data sets. For certain exceptions, refer to the various program descriptions.
- You identify ASCII tape data sets with LABEL=(,AL) or OPTCD=Q.

  The utilities that read or write binary (non-text) data on tape do not support ASCII tapes. This is because ASCII tapes require all data to be text. The utilities that do not support ASCII tapes include IEBCOPY, IEHMOVE, and IFHSTATR.

Related reading: For more information, see *z/OS DFSMS Using Data Sets*.

Special referencing aids

To help you locate the correct utility program for your needs and locate the correct example of the program for reference, two special referencing aids are included in this publication.

To locate the correct utility program, refer to Table 1 on page 1 under “Guide to utility program functions” on page 1.

To locate the correct example, use the figure (called an "example directory") that precedes each program's examples. Table 4 on page 9 shows a portion of the example directory for IEBCOPY. The figure shows that IEBCOPY Example 1 is an example of copying a partitioned data set from one disk volume to another and that IEBCOPY Example 2 is an example of copying from three input partitioned data sets to an existing output partitioned data set.

Table 4. Example directory

Operation	Device	Comments	Example
COPY	Disk	Full Copy. The input and output data sets are partitioned.	1
COPY	Disk	Multiple input partitioned data sets. Fixed-blocked and fixed-record formats.	2

Chapter 4. IEBCOPY (Library Copy) Program

IEBCOPY is a data set utility that is used to copy or merge members between one or more partitioned data sets, or partitioned data sets extended (PDSEs), in full or in part. You can also use IEBCOPY to create a backup of a partitioned data set into a sequential data set (called an unload data set or PDSU), and to copy members from the backup into a partitioned data set.

Starting in z/OS V1R13, IEBCOPY is no longer APF-authorized. It does not need to be run from an authorized library, and calling programs do not need to be APF-authorized.

You can use IEBCOPY to perform the following tasks:

- Make a copy of a partitioned data set or PDSE.
- Merge partitioned data sets (except when unloading).
- Create a sequential form of a partitioned data set or PDSE for a backup or transport.
- Reload one or more members from a PDSU into a partitioned data set or PDSE. (Note that an unloaded load module cannot be converted to a program object in this process.)
- Select specific members of a partitioned data set or PDSE to be copied, loaded, or unloaded.
- Replace members of a partitioned data set or PDSE.
- Rename selected members of a partitioned data set or PDSE when copied.
- Exclude members from a data set to be copied, unloaded, or loaded. (Except on COPYGRP).
- Compress a partitioned data set in place.
- Upgrade a load module for faster loading by MVS program fetch.
- Copy and reblock load modules.
- Convert load modules in a partitioned data set to program objects in a PDSE when copying a partitioned data set to a PDSE.
- Convert a partitioned data set to a PDSE or a PDSE to a partitioned data set.
- Copy to or from a PDS or PDSE data set, a member and its aliases together as a group (COPYGROUP).
- Copy the member generations of a PDSE V2 data set to another (COPYGRP, COPYGROUP).

In addition, IEBCOPY automatically lists the number of unused directory blocks and the number of unused tracks that are available for member records if the output data set is a partitioned data set.

Related reading: For important information on shared partitioned data sets see “Job Control Statements” on page 6.

Converting Load Modules to Program Objects or the Reverse

Program objects are created automatically when load modules are copied into a PDSE. Likewise, program objects are automatically converted back to load modules when they are copied into a partitioned data set. Note that some program objects cannot be converted into load modules because they use features of program objects that do not exist in load modules.

IEBCOPY is not able to *directly* convert between program objects and load modules when loading or unloading a PDSE or partitioned data set. A load operation can only reload load modules into partitioned data sets or reload program objects into a PDSE. Unloading a partitioned data set can only place load modules into the unload data set. Similarly, unloading a PDSE can only place program objects into the unload data set.

Therefore, to convert an unloaded load module into a program object, reload the load module into a partitioned data set and then copy the partitioned data set to a PDSE. To convert a program object into an unloaded load module, copy the PDSE to a partitioned data set and then unload the partitioned data set.

© Copyright IBM Corp. 1979, 2026

If your partitioned data set contains both load modules and data members, then you will have to convert the partitioned data set into two separate PDSEs - one for program objects and the second for data members.

Related reading: For more information about differences between program objects and load modules, see *z/OS MVS Program Management: User's Guide and Reference*.

Converting Partitioned Data Sets to PDSEs

You can use IEBCOPY to convert partitioned data sets to PDSEs.

To convert a partitioned data set to a PDSE, create a PDSE and copy the partitioned data set into the new PDSE. This can be accomplished with one use of IEBCOPY.

You cannot convert a partitioned data set that has any of the following features:

- Both load modules and nonload modules in the same partitioned data set. Individual members of a partitioned data set can be converted by copying them into two separate PDSEs, the first for data and the second for program objects.
- Note lists. Load modules that contain note lists can be placed into PDSEs because they are converted automatically into program objects.
- Nonzero key lengths in the members.

Copying Data Sets

IEBCOPY can be used to totally or partially copy a partitioned data set from one direct access volume to another. In addition, a data set can be copied to its own volume, provided its data set name is changed. (If the data set name is not changed, IEBCOPY interprets the request as a compress-in-place, provided the SELECT / EXCLUDE statement is not coded.)

When you use IEBCOPY to copy a PDSE to a PDSE, either volume to volume or to its own volume, all DDM attributes are also copied.

Members copied into a partitioned data set are not physically reordered; members are copied in the physical order in which they occur in the original data set.

Merging Data Sets

Merging data sets is done by copying or loading the additional members to an existing partitioned data set. The merge operation (ordering of the directory of the output data set) is automatically performed by IEBCOPY.

Increasing Directory Space for a Partitioned Data Set

IEBCOPY cannot increase the number of directory blocks in a partitioned data set. (A PDSE directory automatically expands as needed.) If you are not sure there will be enough directory blocks in the output partitioned data set you are merging to, then you should expand the output data set directory space before beginning the merge operation.

Use IEHLIST to determine how much directory space remains in a partitioned data set. If more blocks are needed, this procedure will extend most data sets:

1. Rename the data set.
2. Allocate a new data set with enough space and directory blocks.
3. Copy the renamed data set to the newly allocated data set.
4. Delete the renamed data set (or save it as a backup).

Unloading (Backing up) Data Sets

IEBCOPY can be used to create a backup copy of a partitioned data set by copying (unloading) it to a sequential data set on DASD, tape, or other device supported by QSAM.

IEBCOPY creates an unload data set when you specify physical sequential organization (DSORG=PS) for the output data set. To create a partitioned data set, specify DSORG=PO and DSNTYPE=PDS or DSNTYPE=LIBRARY.

Requirement: If you do not explicitly state the DSORG, it might be set opposite to your intention by ACS routines or other JCL parameters. For example, if you specify LIKE= or DCB= or a model DSCB and omit the DSORG, then whatever DSORG the referenced object has will be implicitly added to your DD statement. Always specify a DSORG if you are not sure what will be taken from a referenced object.

Attention: Do not change the DCB parameters of an unload data set after IEBCOPY finishes creating it, or IEBCOPY might not be able to reload it.

To unload more than one partitioned data set to the same tape volume in one execution of IEBCOPY, multiple copy operations must be used and multiple sequential data sets must be allocated to successive files on the tape.

IEBCOPY can copy a PDSU to a PDSU directly without the need to reload it to a PDS and then unload the PDS to create the new PDSU. If a selective copy is not required then it will be faster to use IEBGENER to copy the PDSU to a new PDSU.

Only a COPY operation can create an unload data set; COPYMOD cannot.

Copying Directory Information between a Partitioned Data Set and a PDSE

The PDSE directory can contain attributes in addition to those traditionally kept in a partitioned data set directory entry.

Some PDSE extended attributes are recorded on an unload data set and will be reloaded when the target is a PDSE.

If you reload an unloaded PDSE that contains *program objects* to a partitioned data set, an error message is issued and the operation fails.

Information that is kept as user data in a partitioned data set directory, such as PDF statistics, will move to a PDSE directory and back from a PDSE directory to a partitioned data set directory without change.

Loading or Copying Unload Data Sets

Using IEBCOPY, you can re-create a partitioned data set from an unloaded copy of a partitioned data set by copying the sequential (unloaded) data set to a partitioned data set. A partitioned data set may be loaded from an unloaded PDSE if it does not contain program objects, and a PDSE may be loaded from an unloaded partitioned data set if it does not contain load modules.

Note: <A load operation cannot convert unloaded load modules to program objects or program objects to unloaded load modules.

You can create a single partitioned data set from multiple input sequential (unloaded) data sets.

For unload and load operations, requests are handled in the same way as for a copy operation. You can choose to process specific members and to rename them.

A partitioned data set in the unload format will have a variable spanned record format. When the unload data set is subsequently loaded, the output data set will have the same characteristics it had before the unload operation, unless you provide overriding characteristics when you reload the data set.

IEBCOPY Unload Data Set DCB Parameters

An unload data set is always a variable spanned record format with sequential organization, (RECFM=VS and DSORG=PS).

The logical record length of the unload data set is intended to hold a block from the input data set plus a header, with these considerations:

1. The LRECL is calculated as being the larger of:
   a. 280 bytes, or
   b. 16 bytes + the block size + the key length of the input data set.
2. If the LRECL exceeds 32760, it is reduced to 32760.

   Note: Applications reading an unload data set should be aware that for RECFM=VS data sets, the actual length of an assembled logical record could exceed 32760 bytes, even if the LRECL was reduced to 32760 by this step.

   Note: When the input data set is a PDSE and has a block size greater than 32744, the LRECL of the unload data set is set to X, i.e. LRECL=32768. LRECL=X is used to process logical records that exceed 32760 bytes.
3. If the user supplies an LRECL larger than the one calculated here, it will be placed in the data set label; however, the size of the logical record that IEBCOPY creates will not be increased.

The block size (BLKSIZE) for an unload data set is determined by the following steps:

1. The initial block size is set to the block size supplied by the user, or if the user did not supply a block size, it is calculated as the LRECL plus 4.
2. If the block size is less than 284, it is increased to 284.
3. If the block size exceeds 32760, it is reduced to 32760.
4. The block size value is then compared with the largest block size acceptable to the output device. If the output device capacity is smaller then the block size, it is set to the maximum allowed for the output device.

Because the unload data set is unblocked, increasing the block size beyond LRECL plus 4 will not result in longer physical records or better utilization.

The block size is stored in the first control record (COPYR1) and used at load time. If the block size of the unload data set is changed after it is created, IEBCOPY might not be able to reload it.

Recommendation: Do not set the PDSU block size equal to the PDS block size or your PDSU will have very poor space utilization and performance. Let IEBCOPY pick the block size or choose a PDSU block size 20 bytes greater than the PDS block size.

Selecting Members to be Copied, Unloaded, or Loaded

Select specific members to be processed from one or more data sets by coding a SELECT statement to name the members. Alternatively, all members but a specific few can be designated by coding an EXCLUDE statement to name members not to be processed.

You cannot use both a SELECT and an EXCLUDE statement in the same copy operation (same set of input ddnames).

A maximum of eight characters can be given for the member or alias name on a copy operation.

Selected members are searched for in a low-to-high (a-to-z) collating sequence, regardless of the order in which they are specified on the SELECT statement; however, they are copied in the same physical sequence in which they appear on the input partitioned data set or PDSE.

Once a member designated in a SELECT statement is found in an input data set, no search is made for it on any subsequent input data set. When all of the selected members are found, the operation ends, even if all data sets have not yet been processed.

Example:

If members A and B are specified and A is found on the first of three input data sets, it is not searched for again when the second and third data sets are searched.

If B is found on the second input data set, the operation is successfully ended after the second input data set has been processed, and the third input data set is never examined.

When you copy, you can rename members unless the input and output data sets are the same.

When using COPYGROUP, if you specify SELECT MEMBER name filter pattern masking, you can also specify EXCLUDE MEMBER name pattern masking. Otherwise, EXCLUDE is not supported with COPYGROUP.

When using COPYGROUP, you can use SELECT MEMBER name filter pattern masking instead of specifying a full member name. The filter characters asterisk (*) and percent sign (%) provide the member name filtering capability. With SELECT MEMBER name filter pattern masking, all of the members of the input data set that match the filter mask pattern criteria will be selected.

You cannot use both a SELECT and an EXCLUDE statement in the same COPY operation (same set of input ddnames); they are mutually exclusive.

Excluding Members from a Copy Operation

Members from one or more input data sets can be excluded from a copy, unload, or load operation. The excluded member is searched for on every input data set in the copy, unload, or load operation and is always omitted. Members are excluded from the input data sets named on an INDD statement that precedes the EXCLUDE statement.

A maximum of eight characters can be given for the member or alias name on a copy operation.

Restriction: EXCLUDE is not allowed for COPYGRP or for COPYGROUP. It is allowed for COPYGROUP when a SELECT MEMBER statement uses member name filter pattern masking, and the EXCLUDE statement does also.

The replace option can be specified on the data set level in an exclusive copy or load, in which case, nonexcluded members on the input data set replace identically named members on the output data set.

Related reading: For more information about the replace option, see "Replacing Members in a Data Set" on page 68.

Copying Members That Have Alias Names (COPY Statement)

This topic discusses using the COPY statement for copying a PDS, PDSU, or PDSE that has members with alias names. The COPYGRP or COPYGROUP statement is recommended for copying *program objects*.

Tip: If the COPY statement is used to copy program objects, errors can occur.

- If you are copying an entire data set to a new data set (one that has no members before the copy operation), all members and their aliases will be copied, and they will have the same relationship to one another as they had on the original data set.
- If you are merging a data set with another data set, no members or aliases on the output data set will be changed unless you specify that input members are to replace output members.

  Example:

  In all instances, if you have a member A with alias B on your input data set, and a member C with alias B on your output data set, if you do *not* indicate replacement, member A will be copied over, but the alias name B will continue to refer to C. If you *do* indicate replacement, B will be copied as an alias of member A in the newly merged data set.
- When selectively copying from a partitioned data set, you must specify every name that you want copied, including their aliases.

  Example:

  If you are selecting member A, and member A has the aliases B and C, to copy all three names you must specify SELECT MEMBER=(A,B,C). This will result in *one* copy of the data for that member, and all three names placed in the directory and associated with that member data.

If you specify SELECT MEMBER=(A,C). This will result in *one* copy of the data for that member, and two names, A and C, placed in the directory and associated with that member data.

- When copying to or from a PDSE, you must specify the member's name. An alias will not become the member name in the output data set.
- If you are exclusively copying a data set (using the EXCLUDE statement), you must specify not only a member's name, but also all of its alias names to exclude the member data from the copy operation.

  Example:

  If you want to exclude member A from the copy operation, and A has the alias names B and C, you must specify EXCLUDE MEMBER=(A,B,C). If you specify only MEMBER=A, then the member is copied to the output data set with the alias names B and C.
- The rules for replacing or renaming members apply to both aliases and members; no distinction is made between them.

Related reading: For information about copying program objects, see "Copying Program Objects (COPYGRP and COPYGROUP Statements)" on page 69.

Replacing Members in a Data Set

You can use IEBCOPY's COPY or COPYGRP or COPYGROUP statement to replace members on an output partitioned data set or PDSE. The explanations in this topic are for the COPY statement.

With the COPY statement you can perform the following tasks:

- You can specify replacement on the data set level. In this case, every member of an input data set will be copied to the output data set. Each member on the output data set that has a name identical to the name of a member on the input data set will be replaced.
- You can indicate replacement on the member level. In this case, you can indicate that a particular member of the input data set is to replace an identically named member of the output data set, and indicate that another member is to be copied only if it is not already present on the output data set.

  When you specify replacement on the member level, you can also rename an input member. The output data set directory is searched for the new name to see if the member should be copied. For instance, you could rename member A to B, and have it replace member B on the output data set.

Related reading: For information about replacing members using COPYGRP, See "Copying Program Objects (COPYGRP and COPYGROUP Statements)" on page 69.

Specifying Replacement on the Data Set Level

When you merge partitioned data sets, or load an unload data set into a partitioned data set that already has members, the input and output data sets might have members with the same names. Under normal processing, these input members will not replace the output members that have their names. To specify that all input members are to be copied to the output data set, and thus replace any output members of the same name, use the replace (**R**) option on an INDD or COPY statement.

When replace (R) is specified on the data set level, the input data is processed as follows:

- In a full copy or load process, all members in an input data set are copied to an output partitioned data set; members whose names already exist in the output partitioned data set are replaced by members copied or loaded from the input data set.
- In a selective copy or load process, all selected input members will be copied to the output data set, replacing any identically named output data set members. Specifying replace (R) on the data set level when performing a selective copy relieves you of the need to specify replace (R) for each member you want copied.
- In an exclusive copy process, all nonexcluded members on the input data sets are copied or loaded to an output partitioned data set, replacing those members with duplicate names on the output partitioned data set.

Specifying Replacement on the Member Level

When you specify the name of a member to be copied in the MEMBER operand of the SELECT statement, specify the replace (R) option for input members which have identically named members in the output data set. In this way, you can copy many members from an input data set, but allow only a few of them to replace members in the output data set.

Processing Considerations for Replacing Members

There are differences between full, selective, and exclusive copy or load processing. These differences should be remembered when specifying the replace (R) option on either the data set or member level, and when the output data set contains member names common to some or all the input data sets being copied or loaded. These differences are:

- When a full copy or load is performed, the output partitioned data set contains the replacing members that were on the *last* input data set copied.
- When a selective copy or load is performed, the output partitioned data set contains the selected replacing members that were found on the *earliest* input data set searched. After a selected member is found, it is not searched for again. Therefore, after it is found, a selected member is copied or loaded. If the same member exists on another input data set, it is not searched for, and hence, not copied or loaded.
- When an exclusive copy or load is performed, the output partitioned data set contains all members, except those specified for exclusion, that were on the *last* input data set copied or loaded.

Renaming Selected Members

Using the SELECT statement to rename members

To use the SELECT statement to rename members, you should place the old name, the new name, and optionally a replace (R) indicator together inside parentheses as an operand of the MEMBER parameter on a SELECT statement. MEMBER parameter operands may consist of as many old name/new name/replace sets as you need.

The replace (R) option must be used if the new name matches a name of a member in the output data set, or the member will not be copied. It does not matter if replace is indicated globally for all members by using the INDD parameter, or if it is indicated for individual members in the MEMBER parameter.

The selected members are not renamed in the input data set directory. They are just added to the output data set with the new name.

Copying Program Objects (COPYGRP and COPYGROUP Statements)

It is recommended that you use the COPYGRP or COPYGROUP statement to copy PDS load modules or PDSE program objects and their aliases from or to a PDSE data set. Program objects can have aliases that are longer than eight characters. Using the COPYGRP or COPYGROUP statement will ensure that these longer aliases are copied along with their member.

Use the COPYGRP or COPYGROUP statement to begin a *group* copy, unload, or load. A *group* consists of a member and all of its aliases. COPYGRP or COPYGROUP treats the group as a single entity.

COPYGRP can be used to copy a data set when either the input data set or the output data set, or both, are PDSE:

PDSE to PDSE
PDSE to PDS
PDS to PDSE

COPYGROUP can be used to copy a data set when either the input data set or the output data set, or both, are PDS or PDSE:

PDSE to PDSE
PDSE to PDS
PDS to PDSE
PDS to PDS

For unloading groups:

Using COPYGRP: PDSE to PS
Using COPYGROUP: PDSE to PS, or PDS to PS

For loading groups:

Using COPYGRP: PS to PDSE
Using COPYGROUP: PS to PDSE, or PS to PDS

If neither data set is a PDSE, the request is treated as a COPY operation subject to the syntax requirements of COPYGRP.

When using the COPYGRP or COPYGROUP statement:

- All aliases in a group will be copied with the member or neither the aliases or the member in a group will be copied.
- There can be only one INDD per copy operation.
- You can use the SELECT statement to selectively copy members. Either the member name or an alias can be specified to copy the member and all of its aliases.
- Do not indicate replace (R) on the SELECT statement.
- The EXCLUDE statement is not supported.

COPYGROUP

The COPYGROUP statement has the same effect as the COPYGRP statement when either the input or the output data set or both are partitioned format, that is either PDS or PDSEs. The function of a COPYGROUP statement differs from COPYGRP only if both of the data sets are PDSs. COPYGROUP performs a full group copy operation when both data sets are PDSs. By contrast, a COPYGRP statement with two PDSs is the same as a COPY statement with those data sets.

To abbreviate a COPYGROUP statement, use CP. The abbreviation for COPYGRP remains CG.

When using the COPYGROUP statement:

- Member name pattern masking can be used, instead of a full member name specification, to select multiple member from the input data set.
- If member name pattern masking is used, the member rename option will not be honored. Member name pattern masking and member rename are mutually exclusive.

Replacing Program Objects

If the replace (R) option is indicated on the INDD parameter,

- The output data set members and their aliases will be replaced if they have the same member and alias names as the input data set's members and aliases.

  Example:

  The input data set has member A with alias B; the output data set has member A with alias B. The input data set's member and alias will replace the output data set member and alias.
- The copy will fail if a member's alias in the output data is the same as a differently named member's alias in the input data set.

  Example:

  The input data set has member A with alias B; the output data set has member C with alias B. The copy will fail because the alias B points to a member with a different name on the output data set.

- If the output data set's members and aliases do not match the input data set's members and aliases, then all of the input data set's members and aliases are copied.

  Example:

  The input data set has member A with alias B; the output data set has member C with alias D. After the copy, the output data set will contain A with alias B and C with alias D.

  The EXCLUDE statement is not supported.

Compressing a Partitioned Data Set

A partitioned data set will contain unused areas (sometimes called gas) where a deleted member or the old version of an updated member once resided. This unused space is only reclaimed when a partitioned data set is copied to a new data set, or after a compress-in-place operation successfully completes. It has no meaning for a PDSE and is ignored if requested.

The simplest way to request a compress-in-place operation, is to specify the same ddname for both the OUTDD and INDD parameters of a COPY statement, without specifying the SELECT / EXCLUDE statement.

However, a compress is actually performed when both the input and output is the same data set on the same volume. For example, this job step will compress data set Pacanowska:

```
//COMPRESS EXEC    PGM=IEBCOPY
//A        DD  DSNAME='Pacanowska',DISP=OLD
//B        DD  DSNAME='Pacanowska',DISP=OLD
//SYSPRINT DD SYSOUT=*
//SYSIN    DD *
          COPY OUTDD=B,INDD=A
```

If multiple entries are made on the INDD statement, a compress-in-place occurs when any of the input ddnames matches the OUTDD name. The compress operation is performed in the same relative order as the ddnames in the INDD list.

For example, consider the COPY statement:

```
       COPY OUTDD=B,INDD=(A,B,C,B)
```

- The data set for ddname A is copied to ddname B
- The data set B is compressed
- ddname C is copied to ddname B
- The data set B is compressed again.

It is a good idea to make a copy of the data set that you intend to compress-in-place before you actually do so. You can use the same execution of IEBCOPY to do both, and a following job step could delete the backup copy if the IEBCOPY job step ends with a return code of 0.

Attention: A partitioned data set can be destroyed if IEBCOPY is interrupted during processing, for example, by a power failure, abend, TSO attention, or I/O error. Keep a backup of the partitioned data set until the successful completion of a compress-in-place.

Attention: Do not compress a partitioned data set currently being used by more than one user. If you do, the other users will see the data set as damaged, even though it is not. If any of these other users update the partitioned data set after you compress it, then their update will actually damage the partitioned data set.

Processing Considerations for Compress

- If you try to perform a compress-in-place on a PDSE, IEBCOPY will ignore your request and continue processing with the next control statement.
- A compress-in-place does not release extents assigned to the data set.

- During a compress operation, you cannot include, exclude, or rename selected members.
- You may not change any data set DCB parameters, such as block size, when compressing a data set.

Altering Load Modules

ALTERMOD is designed as a one-time update operation against load modules from old systems. You can use IEBCOPY to update partitioned data set load modules that were written by a linkage editor prior to MVS/370, so that they will load faster. Load modules processed with the linkage editor in MVS/370 and subsequent versions of MVS and DFSMS do not require alterations, nor do program objects in a PDSE.

ALTERMOD will place correct relocation dictionary (RLD) counts and segment block counts into control records inside the module. ALTERMOD performs this update without making a new copy of the load module. It can be used to alter modules that might have erroneous RLD counts. Examples of modules that might have erroneous RLD counts are modules that were created by a program other than the linkage editor or copied by a program other than IEBCOPY.

The ALTERMOD statement will not function under these conditions:

- The load modules are in scatter-load format or link edited with the noneditable (NE) attribute.
- The data set is a PDSE. (It is ignored.)

When you use a SELECT statement to identify members to be processed by ALTERMOD, you cannot rename them.

Copying and Reblocking Load Modules

The COPYMOD statement lets you COPY and reblock the load modules to a block size appropriate for the device to which you are copying the data set.

The text records, relocation dictionary (RLD)/control records, and note list records of overlay load modules will be rebuilt when you use COPYMOD. Other records such as SYM and CESD records will be copied without any changes. The load modules processed by COPYMOD can be link edited again.

- Load modules in page-aligned format are copied without reblocking, as if the operation was COPY not COPYMOD, and the functions of ALTERMOD are performed against the copy that was made.
- Load modules in scatter-load format and modules that were link-edited with the noneditable (NE) attribute will be copied, but not reblocked or altered.
- Members that are not recognized as load modules will be copied, but not reblocked or altered.
- Load modules that have the downward compatible (DC) linkage editor attribute are reblocked to a maximum block size of 1024 (1K) regardless of the value specified on the MINBLK or MAXBLK parameter.

The block size in the output data set label is increased by COPYMOD as needed to match the MAXBLK value.

COPYMOD does not write records longer than the output data set block size. However, if COPYMOD cannot process a member, and COPY is used instead, COPY will copy all records, including those records longer than the output data set block size.

The reblocking function of COPYMOD lets you specify the following block sizes:

- A maximum block size for compatibility with other systems or programs
- A minimum block size to specify the smallest block that should be written on the end of a track

IEBCOPY will determine the amount of space remaining on a track before assigning a size to the next block to be written. If this amount is smaller than the output block size, IEBCOPY will try to determine if a smaller block can be written to use the remaining space on the track. The maximum block size produced by the COPYMOD function is 32760 bytes.

Changed COPYMOD operation

Before MVS/DFP Version 3, Release 2, the default MAXBLK size for COPYMOD was 32760. The intention was to make text blocks as long as possible to reduce the number of text blocks read to fetch the module. This often resulted in physical records created in the data set which were longer than the BLKSIZE in the data set label. (These blocks are called *fat blocks*.) This condition leads to I/O errors or a violation of data integrity.

Starting with Version 3, Release 2, the default MAXBLK value is the data set block size, not 32760. Further, if a MAXBLK value is specified that is larger than the block size in the data set label, then the value in the data set label is increased to the specified MAXBLK value.

How IEBCOPY Uses Virtual Storage for Tables and Buffers

Starting with IEBCOPY ESCON® Performance Updates the minimum work area size is about 208K. If your job has a small REGION size, you might have to increase it.

The recommended minimum REGION sizes for IEBCOPY jobs are 1M when only partitioned data sets are copied, and 2M when a PDSE is copied.

The WORK=nnn parameter in the JCL EXEC PARM field controls how much space IEBCOPY requests for a work area. The REGION size must exceed the work size plus the program size before an increase in the value of the work parameter will have an effect.

From this work area comes tables, buffers, and storage for partitioned data set directories. When there is not enough work area, the partitioned data set directories spill to SYSUT3 and SYSUT4. If more storage is still needed, IEBCOPY will stop with a message. Larger WORK and REGION values allow larger directories to be processed without opening the spill data sets assigned to SYSUT3 and SYSUT4.

Large WORK and REGION sizes should be accompanied by a SIZE parameter when small data sets are copied. This is to prevent the buffers from becoming too large and causing a degradation in performance or a shortage in real storage.

How IEBCOPY Allocates Tables and Buffers

1. IEBCOPY obtains as large a work area as possible, up to the value of the WORK parameter.
2. An initial minimum sized buffer and channel program construction area is allocated from the work area.

   The size of this area is approximately 4 times the size of the largest input or output device track, plus about 3%.

   If IEBCOPY cannot allocate a minimum sized work area, it will stop with a message.
3. IEBCOPY processes control statements, reads the partitioned data set directories, and builds a table of members to be copied. When the work area cannot hold the table and all directories, the directory entries are spilled to SYSUT3 and SYSUT4 to allow the table to grow. When no room remains and all directories are spilled, then IEBCOPY ends with a message.
4. If enough storage remains unused in the work area, it is used as a second buffer.

   If a SIZE= parameter is specified, the size of the second buffer is limited so that the total size of both buffers does not exceed the specified value. If the SIZE= value will not allow a minimum sized second buffer, it is not allocated.

Tip: When SIZE=999999 (or any number that is a few thousand less than the WORK= value) is coded, IEBCOPY might be able to allocate buffers in the work area but not have enough room remaining for tables. If this happens, increase the REGION= and WORK= values, or remove the SIZE= parameter.

When data sets with huge directories are copied, make the largest amount of virtual storage available to retain directory information. Specify WORK=8M (or another large value) and a correspondingly large REGION.

When data sets with small directories are copied with large work area sizes, the second I/O buffer can become very large (megabytes) and cause real storage shortages. This could result in increased system paging and system sluggishness, because most of the buffer is backed by real frames which are fixed for duration of the I/O. In this case, specify SIZE=1M or a smaller value to limit the amount of storage used for buffers, but allow lots of storage to be used for directory information.

Avoiding the Need to Supply Control Statements

When the SYSIN DD statement is a DD DUMMY, points to an empty file, or is omitted, IEBCOPY will generate a control statement that allows you to run IEBCOPY without supplying a control statement data set for SYSIN.

The generated statement can take several forms, depending on what parameters are specified in the JCL EXEC PARM field.

1. When COMPRESS and REPLACE are not specified, the generated statement is: *xxxxx* OUTDD=SYSUT2,INDD=SYSUT1. The *xxxxx* will be ALTERMOD, COPYGROUP, COPYGRP or COPYMOD if you code one of those PARM options or a valid abbreviation. If you do not code one of them, the generated control statement will be COPY. The COPY, COPYGROUP, COPYGRP or COPYMOD will copy without replacing from the data set designated by the SYSUT1 DD statement to the data set designated by the SYSUT2 DD statement.
2. When COMPRESS is not specified but REPLACE is specified, the generated statement is: COPY OUTDD=SYSUT2,INDD=(SYSUT1,R) which will copy with replace from the data set designated by the SYSUT1 DD statement to the data set designated by the SYSUT2 DD statement.
3. When COMPRESS is specified, then the generated statement is: COPY OUTDD=SYSUT2,INDD=SYSUT2 which will compress in place the data set designated by the SYSUT2 DD statement.

When you specify any of the preceding parameters in the JCL EXEC PARM field, then IEBCOPY opens SYSIN and automatically generates a control statement.

Restrictions

To use IEBCOPY, see the following characteristics, rules, and restrictions:

- IEBCOPY *does* support VIO (virtual I/O) data sets.
- IEBCOPY uses the EXCP access method. Therefore:
  - Some common DCB parameters such as BUFNO are ignored.
- Starting in z/OS V1R13, IEBCOPY is no longer APF-authorized.
- IEBCOPY must not be loaded in supervisor state or in protection key zero. It is an application program that does not use the special system interfaces that are assumed for the system kernel running in supervisor state or in protection key zero.
- Variable spanned format record (VS or VBS) are not supported for a partitioned data set.
- Shared or in use data sets should not be compressed in place or updated unless the subject data set is made *nonsharable*. See “Sharing data sets” on page 7.
- When a PDSE is involved and only a small amount of virtual storage is available to the PDSE processing routines, then messages about the shortage might only appear on the console and not in the SYSPRINT data set.
- Load modules having the downward compatible (DC) linkage editor attribute is reblocked to a maximum block size of 1024 (1K) when encountered during COPYMOD processing, regardless of the number specified on the MINBLK and MAXBLK parameters.
- Reblocking cannot be performed if either the input or the output data set has:
  - Undefined format records
  - Keyed records
  - Track overflow records

  - Note lists or user TTRNs

  Or if compress-in-place is specified. (Load modules, with undefined record formats and note lists, might be reblocked using the COPYMOD statement.)
- The compress-in-place function cannot be performed for the following:
  - Unload data sets.
  - Data sets with track overflow records.
  - Data sets with keyed records.
  - Unmovable data sets.
  - PDSEs (request is ignored).
- PDSEs cannot contain members with note lists, keys, or track overflow. You cannot mix load modules and nonload modules in the same PDSE.
- Using OPTCD=W for any DASD might dramatically slow down a COPY operation. OPTCD is in the data set label, and therefore can be active when OPTCD is *not* coded on the DD statement.
- OPTCD=W is honored when coded in the JCL. OPTCD=W, if present in the data set label, is deleted from the label.

Note: If IEBCOPY copies a record which is physically longer than the block size of the output partitioned data set, message IEB175I (return code 4) is issued to warn you that the data set contains *fat blocks*, which are physical records that are created in the data set that are longer than the BLKSIZE in the data set label.

- COPYMOD size limits
  - The load modules used as input cannot have more than 60 CSECTS in a single text block.
  - Overlay load modules cannot have more than 255 segments.
- IEBCOPY user TTR limits
  - There are up to three user TTRN fields in the directory.
  - Only one of these fields might have n>0.
  - The maximum length of the note list record identified by the user TTRN with n>0 is 1291 bytes including any block and record descriptor word.
  - No TTRN fields in a note list record might have n>0.
- A load module from an unload data set cannot be reloaded into a PDSE as a program object. The load modules should be reloaded into a partitioned data set and then the partitioned data set should be copied to a PDSE to convert the unloaded load module into a program object.
- IEBCOPY size limits
  - The maximum number of renames allowed on all SELECT or EXCLUDE cards for one copy operation is (2 * max_trk) / 16, where "max_trk" is the cylinder length of the larger device used in the copy operation. For a 3380, this limit is about 5925. For a 3390 the limit is about 7050, and for a 9345, the limit is about 5800.
  - Do not use a PDSU block size smaller than the PDS block size +20.
  - SYSUT4 space must be a single contiguous extent.
  - A PDSE program object must have a block size of at least 4000 bytes.
- IEBCOPY specifies the REREAD or LEAVE parameters for the CLOSE macro. If you specify FREE=CLOSE in the JCL, the data set is not unallocated until the end of the job step, and informational message IEC988I is issued to indicate that the data sets are not unallocated during close.
- IEBCOPY can require significant amounts of virtual storage when the input data set (INDD) is a PDSE. The amount of private area extended virtual storage required when using a PDSE as the input data set is approximately one megabyte per two hundred PDSE members that are included. Since the PDSE copy or unload operation requires a sizable amount of private area extended virtual storage, large PDSE data set storage requirements might exceed the total amount of available private area storage.

The actual amount of private area virtual storage available varies based on local system configuration and loading requirements. Some PDSE copy operations might fail because of excessive virtual storage requirements.

Following are some estimates of storage requirements, for use as examples:

– If an INDD PDSE includes 500,000 members:

```
500,000 members/200 per megabyte = 2,500; 2,500 X 1 megabyte = 2.6+ gigabytes
```

– If an INDD PDSE includes 300,000 members:

```
300,000 members/200 per megabyte = 1,500; 1,500 X 1 megabyte = 1.5+ gigabytes
```

- Some PDS data members, for example in IMS ACBLIBs, have attributes that resemble load module attributes. Using COPYGROUP to copy such members from a PDS to a PDS might result in message IGW0155T. Instead, use COPYGRP and specify the member and alias names to be copied.

Input and Output

IEBCOPY uses the following input:

- A partitioned data set, or a PDSE, or unload data set that contains members to be copied, merged, altered, reblocked, loaded, or unloaded.
- An optional control data set that contains utility control statements. The control data set is required when:

  There is more than one input or one output data set to be processed,
  Designated members are to be selected or excluded, or
  A load module library is to be altered or reblocked (ALTERMOD or COPYMOD is needed).

IEBCOPY produces the following output:

- Output data sets, which contain the copied, merged, altered, reblocked, or unloaded members. The output data set is either a new data set (from a copy, reblock, load, or unload) or an old data set (from a merge, compress-in-place, copy, alter, or load).
- A message data set, which lists control statements, IEBCOPY activities, and error messages, as applicable.

IEBCOPY might require:

- Optional spill data sets, which are temporary data sets used to provide space when not enough virtual storage is available for the input or output partitioned data set directories. These data sets are opened only when needed.

If IEBCOPY is invoked from an application program, you can dynamically allocate the data sets by issuing SVC 99 before calling IEBCOPY.

For IEBCOPY return codes, see Appendix A, “Invoking Utility Programs from an Application Program,” on page 337.

Control

IEBCOPY is controlled by JCL and utility control statements.

Job Control Statements

Table 15 on page 77 shows the job control statements for IEBCOPY.

Table 15. Job control statements for IEBCOPY

Statement	Use
JOB	Starts the job.
EXEC	Starts IEBCOPY.
SYSMDUMP DD	Defines the dump data set when an abnormal end is issued.
SYSPRINT DD	Defines a sequential data set used for listing control statements and messages.
SYSUT1 or anyname1 DD	Defines a partitioned data set or unload data set for input. A partitioned data set must reside on DASD or be a VIO data set. The unload data set is a sequential data set created as the result of an unload operation and may reside on DASD or tape or any other device supported by the QSAM access method. The unload data set can be basic format, large format, extended format, compressed format or tape.
SYSUT2 or anyname2 DD	Defines a partitioned data set or unload data set for output. A partitioned data set must reside on DASD or be a VIO data set. The unload data set is a sequential data set that was created as the result of an unload operation and may reside on DASD or tape or any other device supported by the QSAM access method.
SYSUT3 DD	Defines a spill data set on a DASD or VIO device. SYSUT3 is used when there is no space in virtual storage for some or all of the current input data set directory entries.
SYSUT4 DD	Defines a spill data set on a DASD or VIO device. SYSUT4 is used when there is no space in virtual storage for the output data set directory.
SYSIN DD	Defines the optional control data set.

EXEC Statement

The syntax of the EXEC statement is:

Label	Statement	Parameters
//[stepname]	EXEC	[,PGM=IEBCOPY. [,REGION={*n*|*n*K|*n*M}] [,PARM=<*parms*>]

where:

PGM=IEBCOPY
specifies that you want to run the IEBCOPY program.

REGION={*n*|*n*K|*n*M}
specifies the amount of storage to be made available to IEBCOPY by the operating system. Specify 1M if you are only using partitioned data sets. If you are using any PDSE, then specify 2M.

Notes:

1. The number *n* can be any number of digits, and is specified in decimal. The K causes the number to be multiplied by 1024 bytes (1 kilobyte) and M causes the number to be multiplied by 1024K or 1048576.
2. Specifying a larger REGION size might not have an effect unless the WORK= value is also increased.
3. Specifying a REGION size on the JOB statement will override any REGION specified on the EXEC statement.

PARM=

You may specify any of the parameters in any order to IEBCOPY. Separate multiple parameters with a comma between each one.

ALTERMOD *or* AM

If you direct IEBCOPY to generate a control statement, specify this parameter to make the control statement perform an ALTERMOD operation instead of a COPY operation.

BYPASS_AUTH

Specify this parameter if you wish to direct IEBCOPY to bypass RACF authorization checking for a DASD sequential or partitioned (PDS or PDSE) data set. This option does not apply to tape, spooled data sets, work data sets, or data set password checking. If the BYPASS_AUTH option is specified, IEBCOPY must be invoked by an APF-authorized caller. If the parameter is passed from a non APF-authorized caller, abend 913-80 will be issued.

CMWA=nK

Specify this parameter to increase the COPYMOD work area size of 120K if larger load modules are being processed. This will be evident because message IEB1133E will be issued in this instance.

COMPRESS

If you direct IEBCOPY to generate a control statement, specify this parameter to make the control statement perform a compress-in-place operation instead of a COPY operation.

COPY *or* C

If you wish to direct IEBCOPY to generate a control statement, specify this parameter to make the control statement perform a COPY operation.

COPYGROUP *or* CP

If you wish to direct IEBCOPY to generate a control statement, specify this parameter to make the control statement perform a COPYGROUP operation.

COPYGRP *or* CG

If you wish to direct IEBCOPY to generate a control statement, specify this parameter to make the control statement perform a COPYGRP operation.

COPYMOD

If you wish to direct IEBCOPY to generate a control statement, specify this parameter to make the control statement perform a COPYMOD operation.

LC=*n*
LPP=*n*
LINECOUNT=*n*

n is the number of lines, including headings, to print on each page of the SYSPRINT output listing. Default is 60.

LIST=NO
LIST=YES

sets the default value for the LIST= operand when it is omitted from the COPY, COPYMOD, or ALTERMOD statement. Default is **LIST=YES**.

RC4NOREP

Use of this parameter will cause IEBCOPY to set a return code of X'04' when a module is not copied from the source data set to the target data set because REPLACE was not specified. When 'RC4NOREP' is specified, message IEB1067W will be issued for each module NOT copied due to the REPLACE option not being specified. Note: Message IEB1067W will be issued regardless of the LIST option requested.

REPLACE

If you wish to direct IEBCOPY to generate a control statement, specify this parameter to make the control statement perform a copy with replace operation.

SIZE={*n*|*n*K|*n*M}

specifies the maximum number of bytes of virtual storage that IEBCOPY may use as a buffer.

It is best to let IEBCOPY choose buffer sizes by not using this parameter.

The minimum buffer size is approximately 4 times the largest track size of the devices being used, plus about 3%. There is no maximum for this value, but IEBCOPY cannot use more than the quantity available in the work area.

The number *n* can be any number of digits, and is specified in decimal. The K causes the number to be multiplied by 1024 bytes (1 kilobyte) and M causes the number to be multiplied by 1024K or 1048576.

SPCLCMOD

specifies that when a member is found to be ineligible for COPYMOD, processing (for example, the module is page aligned) and the module is scheduled for processing as a normal COPY, the detection of a record larger than the blocksize of the target data set will cause the entire operation to be terminated (no fat blocks will be created).

If the COPY operation resulting from COPYMOD ineligibility is successful, the return code of the operation will be set to zero instead of four (normally return code is set to 4 to indicate that a COPY instead of a COPYMOD was done for the affected modules, if LIST=YES is in effect).

WORK={*n*|*n*K|*n*M|1M}
specifies the number of bytes of virtual storage to request for a work area to hold directory entries, internal tables, and I/O buffers. The default request will be for 1M. The actual amount obtained will not exceed the space available in the REGION.

The number *n* can be any number of digits, and is specified in decimal. The K causes the number to be multiplied by 1024 bytes (1 kilobyte) and M causes the number to be multiplied by 1024K or 1048576.

Related reading: For more information, see:

- "Avoiding the Need to Supply Control Statements" on page 74,
- "How IEBCOPY Allocates Tables and Buffers" on page 73,
- "How IEBCOPY Uses Virtual Storage for Tables and Buffers" on page 73.

SYSMDUMP DD Statement

The SYSMDUMP DD is an optional statement and can be specified if IEBCOPY abends and a dump is required for diagnosis.

The SYSMDUMP DD data set can be basic, large, extended, or compressed format or a member of a PDS or PDSE.

These are the valid DCB parameters you may specify:

RECFM=[FBS]
The record format is fixed blocked standard.

LRECL=
The minimum logical record length that you may specify is 4160 for fixed length.

SYSPRINT DD Statement

IEBCOPY writes a log of the control statements and its actions to the SYSPRINT DD statement. IEBCOPY proceeds if SYSPRINT is unusable and issues summary messages to the console.

You may assign SYSPRINT to SYSOUT or to any QSAM data set. These are the valid DCB parameters you may specify:

DSORG=PS
The output always has sequential organization.

RECFM=[F|FB|V|VB|FA|FBA|VA|VBA]
The record format may be fixed, fixed blocked, and variable or variable blocked. Any of these record formats can be specified to include ISO/ANSI control characters (for example, VBA instead of VB).

LRECL=
The minimum logical record length that you may specify is 60 for fixed length, or 64 for variable length. The maximum is 250.

BLKSIZE=
If you are using fixed blocked records, the block size may be any multiple of the logical record length. If you are using variable or variable blocked records, the block size must be a minimum of the logical record length plus four. The block size cannot exceed 32760 bytes.

If you do not specify anything for DCB parameters, and the data set label has no DCB parameters, IEBCOPY will choose RECFM=FBA, LRECL=121, and a block size. When the output device is DASD or a standard labeled tape, the system will determine the block size to be used. When the output device is not DASD or a standard labeled tape, IEBCOPY will use the largest value that will work for the specific device, adjusted as needed for the RECFM.

If you specify any parameters other than those that IEBCOPY would choose, IEBCOPY will adapt to them. If you omit RECFM and specify an LRECL other than 121, then IEBCOPY will set RECFM=VBA and it will set an optimal BLKSIZE.

Tip: Giving LRECL a value that is 1-byte less than the width of your TSO terminal will allow you to view entire records from a listing without scrolling left or right.

SYSUT1 (anyname1) and SYSUT2 (anyname2) DD Statements

DD statements are required for input and output data sets. There must be one DD statement for each unique data set used in the job step. You must specify a unique output DD statement (for the unload data set) for every input data set that you unload, for example you cannot append two unload data sets to the same output data set. You cannot unload multiple data sets to the same unload data set.

Data sets that are used as input data sets in one copy operation can be used as output data sets in another copy operation.

Input data sets cannot be concatenated together on the same DD statement.

Fixed or variable records can be reblocked, but you cannot convert fixed format records into variable format records, or variable format records into fixed format records. Reblocking or deblocking is performed automatically when the block size of the input data set is not equal to the block size of the output data set.

For variable format data sets, the output LRECL must be greater than or equal to the input LRECL. For fixed format data sets the input and output LRECL must match. For undefined format data sets, the output BLKSIZE must be greater than or equal to the input BLKSIZE, and any LRECL is ignored.

A PDSE might require somewhat more space than a partitioned data set requires to hold the same data. When copying from a partitioned data set to a PDSE, IEBCOPY will override a secondary space allocation of zero for the output PDSE, rather than stop the job. The copy will obtain any additional space it requires in one unit increments of the primary allocation (tracks, cylinders, or blocks). The secondary space quantity in the data set label will not be changed.

When IEBCOPY must supply DCB parameters for the output data set, it will use the corresponding values from the input data set. In particular, system determined block size will not be used for the output data set block size, nor will the system determined block size or reblockable flags be set in the output data set label.

You should allow IEBCOPY to choose the DCB parameters for an unload data set. If you must specify a block size, specify a value that is at least 20 bytes greater than the input data set block size. (This avoids creating spanned records in the unload data set.)

OPTCD=W may be used to request write verification of the data written and causes DASD cached controllers (for instance, 3880-21, 3990-3) to perform a "write through" operation.

Starting with ESCON support, OPTCD=W also causes a different set of channel programs to be used for writing to ECKD™ capable DASD and for reading from all DASD. These programs could run much slower than the default ones; therefore, there is a significant performance cost for using OPTCD=W.

The write verification requested by OPTCD=W is usually unnecessary, because extensive error recovery occurs in both hardware and error recovery procedures (ERP) used by the MVS I/O supervisor. OPTCD=W does not cause the data read from the device to be compared with the data in virtual storage and cannot detect data garbled in transmission from virtual storage to the device control unit. OPTCD=W causes the control unit to read data from the device (after it has been written) to validate the error checking codes stored with the data.

Related reading: For more information, see "IEBCOPY Unload Data Set DCB Parameters" on page 65 .

SYSUT3 and SYSUT4 DD statements

In most uses of IEBCOPY, you do not need to provide space on spill data sets (SYSUT3 and SYSUT4). If IEBCOPY is running in a region size of 2M or more, neither of the spill data sets are needed if the output data set will have fewer than 1600 directory blocks.

In order to conserve space on DASD, you can use VIO for these data sets. However, it is more efficient to increase the region size (and WORK=parameter value) and not use the SYSUT3 and SYSUT4 data sets at all. You cannot use multivolume data sets for these data sets.

The space required for SYSUT3 depends on the number of members to be copied or loaded. The space to be allocated for SYSUT4 must be equal to or greater than the number of blocks allocated to the largest output partitioned data set directory in the IEBCOPY jobstep. Use a block size of 80 to calculate space requirements.

The space required depends on the number of directory blocks to be written to the output data set. SYSUT4 is only used if more than one data set is specified on a given INDD list. The data set contains one directory block per data block. Use a block size of 256 and a key length of 8 to calculate space requirements.

Requirement: SYSUT4 must be in a single contiguous extent, because SYSUT4 will contain a copy of the output partitioned data set directory. The design of data management requires that a partitioned data set directory be within a single extent.

IEBCOPY ignores all DCB information that is specified for SYSUT3 or SYSUT4.

SYSIN DD Statement

The SYSIN DD statement is optional. If it is omitted, or a DUMMY data set, or an empty data set, IEBCOPY will generate a control statement using options in the PARM field.

SYSIN will not be opened if COPY, COPYMOD, REPLACE, or COMPRESS appears in the PARM field.

If you are copying, loading, or unloading a *single* data set, or wish to compress a data set in place, you can avoid using utility control statements by using SYSUT1 for the input data set and SYSUT2 for the output data set.

Either fixed, variable, or undefined record format is acceptable, as is any BLKSIZE and LRECL consistent with the record format and each other. If the record format indicates carriage controls, the carriage control character in each record is ignored.

Sequence numbers are optional. IEBCOPY will look for sequence numbers (8 digits long) at the front of variable format records and at the end of fixed format records. If IEBCOPY finds them in the first control statement, it will ignore those columns in all control statements.

Related reading: For more information, see "Avoiding the Need to Supply Control Statements" on page 74 .

Utility Control Statements

IEBCOPY is controlled by the utility control statements in Table 16 on page 82.

Table 16. IEBCOPY utility control statements. The minor statements (SELECT or EXCLUDE) can follow each major statement to restrict the scope of the major statements.

Statement	Use
	Major Statements
ALTERMOD	Indicates the beginning of an alter-in-place operation for load modules.
COPY	Indicates the beginning of a COPY operation.
COPYGRP	Indicates the beginning of a COPYGRP operation.
COPYGROUP	Indicates the beginning of a COPYGROUP operation.
COPYMOD	Indicates the beginning of a copy and load module reblock operation.
INDD=	Indicates the beginning of another copy step.

Table 17. IEBCOPY utility control statements (continued). The minor statements (SELECT or EXCLUDE) can follow each major statement to restrict the scope of the major statements.

Statement	Use
	Minor Statements
EXCLUDE	Specifies members in the input data set to be excluded from the copy step.
SELECT	Specifies which members in the input data set are to be copied.

COPY will accept the first letter as an abbreviation for all its key words, except MINBLK. COPYMOD may be abbreviated CM. COPYGRP may be abbreviated CG. COPYGROUP may be abbreviated CP.

To continue the copy control statement, stop at a comma. Put any nonblank character in column 72 and start in column 16 on the next record.

To make a comment statement, place an asterisk (*) in the left column where the label field goes. The record will be printed, then ignored. You may also place a comment on any control statement which also has an operand. Leave 1 or more spaces after the operand, and then start your comment.

- To request a *COPY* operation, specify partitioned data sets as input and output.
- To request an *UNLOAD* operation, specify a partitioned input data set and a sequential output data set.
- To request a *LOAD* operation, specify a sequential input data set and a partitioned output data set.

IEBCOPY uses a copy operation/copy step concept. A copy operation starts with a COPY, COPYGRP, COPYGROUP, COPYMOD, or ALTERMOD statement, and continues until another COPY, COPYGRP, COPYGROUP, COPYMOD, or ALTERMOD statement is found, or the end of the control data set is found. Within each copy operation, one or more copy steps are present. Any INDD statement directly following a SELECT or EXCLUDE statement marks the beginning of the next copy step and the end of the preceding copy step within the copy operation. If such an INDD statement cannot be found in the copy operation, the copy operation will consist of only one copy step.

Table 18 on page 82 shows the copy operation/copy step concept. Two copy operations are shown in the figure. The first begins with the statement named COPOPER1, and the second begins with the statement named COPOPER2.

Table 18. Multiple copy operations within a job step

Step	Label	Command	Parameters
First Copy Operation			

Table 18. Multiple copy operations within a job step (continued)

Step	Label	Command	Parameters
STEP 1	COPOPER1	COPY	OUTDD=AA INDD=ZZ INDD=(BB,CC) INDD=DD INDD=EE
		SELECT	MEMBER=(MEMA,MEMB)
		SELECT	MEMBER=(MEMC)
STEP 2			INDD=GG INDD=HH
		EXCLUDE	MEMBER=(MEMD,MEMH)
Second Copy Operation			
STEP 1	COPOPER2	COPY	OUTDD=YY INDD=(MM,PP) LIST=NO
		SELECT	MEMBER=MEMB
STEP 2			INDD=KK INDD=(LL,NN)

The first copy operation shown in Table 18 on page 82 is copying two groups of members. The first begins with the COPY statement and continues through the two SELECT statements. The second begins with the first INDD statement following the two SELECT statements and continues through the EXCLUDE statement preceding the second COPY statement.

The second copy operation has two steps. The first step begins with the COPY statement and continues through the SELECT statement. The second begins with the INDD statement immediately following the SELECT statement.

ALTERMOD Statement

The ALTERMOD statement is required to alter load modules in place. ALTERMOD will only work with a partitioned data set, not a PDSE.

The syntax of the ALTERMOD statement is:

Label	Statement	Parameters
[*label*]	ALTERMOD	OUTDD=*DDname* [,LIST={YES|NO}]

where:

OUTDD=*DDname*
specifies the ddname of the partitioned data set that is to be altered.

LIST={YES|NO}
specifies whether the names of the altered members are to be listed in the SYSPRINT data set. When this parameter is omitted, the default from the EXEC PARM field applies.

COPY Statement

Use the COPY statement to begin one or more copy, unload, or load operations. Any number of operations can follow a single COPY statement; any number of COPY statements can appear within a single job step.

The syntax of the COPY statement is:

Label	Statement	Parameters
[*label*]	COPY	OUTDD=*DDname* ,INDD=[(]{*DDname*|(*DDname*,R) }[,...][)] [,LIST={YES|NO}]

where:

OUTDD=*DDname*
specifies the name of a DD statement that locates the output data set.

INDD=[(]{*DDname*| (*DDname*,R)}[,...][)]
specifies the names of DD statements that locate the input data sets.

When an INDD= appears in a record by itself (that is, not with a COPY keyword), it functions as a control statement and begins a new step in the current COPY operation.

These values can be coded:

DDname
the ddname of the DD statement for the input data set. For an unload operation, only one ddname should be specified per COPY statement. If more than one ddname is specified for a copy or load operation, the input data sets are processed in the same sequence as the ddnames are specified.

R
specifies that all members to be copied or loaded from this input data set are to replace any identically named members on the output partitioned data set. (In addition, members whose names are not on the output data set are copied or loaded as usual.) When this option is specified, the ddname and the R parameter must be enclosed in a set of parentheses; if it is specified with the first ddname in INDD, the entire field, exclusive of the INDD parameter, must be enclosed in a second set of parentheses.

LIST={YES|NO}
specifies whether the names of copied members are to be listed in the SYSPRINT data set at the end of each input data set. When this parameter is omitted, the default from the EXEC PARM field applies.

Usage Notes for COPY

The control statement operation and keyword parameters can be abbreviated to their first letters; for example, COPY can be abbreviated to C and OUTDD can be abbreviated to O.

If there are no keywords other than OUTDD on the COPY record, comments may not be placed on the record.

You can use COPY to copy *and* reblock data sets with fixed-blocked or variable-blocked records. You cannot use COPY to reblock undefined records or load module libraries. Instead, use COPYMOD to copy *and* reblock a load library.

Only one INDD and one OUTDD keyword may be placed on a single record. OUTDD must appear on the COPY statement. When INDD appears on a separate record, no other operands may be specified on that record, and INDD is not preceded by a comma.

A COPY statement must precede SELECT or EXCLUDE statements when members are selected for or excluded from a copy, unload, or load step. In addition, if an input ddname is specified on a separate INDD statement, it must follow the COPY statement and precede the SELECT or EXCLUDE statement which apply to it.

COPYGRP statement

Use the COPYGRP statement to begin a group copy, unload, or load. A group consists of a member and all of its aliases. COPYGRP treats the group as a single entity.

The syntax of the COPYGRP statement is:

Label	Statement	Parameters
[*label*]	COPYGRP	OUTDD=*DDname* ,INDD={*DDname*|((*DDname*,R))} [Y,LIST={YES | NO}] [Y,GENS={ALL| NONE}]

Where:

OUTDD=*DDname*
Specifies the name of a DD statement that locates the output data set.

INDD={*DDname*| ((*DDname*,R))}
Specifies the name of a DD statement that locates the input data set.

Restriction: Multiple INDD statements are not allowed for COPYGRP.

These values can be coded:

DDname
Specifies the ddname, which is specified on a DD statement, of an input data set.

R
Specifies that a group to be copied or loaded from this input data set is to replace a group in the output data set. When this option is specified, the ddname and the R parameter must be enclosed in a set of parentheses; and the entire field, except the INDD parameter, must be enclosed in a second set of parentheses.

LIST={YES|NO}
Specifies whether the names of copied members are to be listed in the SYSPRINT data set at the end of each input data set. When this parameter is omitted, the default from the EXEC PARM field applies.

GENS={ALL|NONE}
Indicates whether the member generations of a Version 2 PDSE are copied. This keyword is valid only when copying from a PDSE V2 with member generations to another.

GENS=ALL
Specifies that all members and generations to be copied from this input PDSE data set are to replace any identically named members on the output PDSE data set.

GENS=NONE
Specifies not to copy the member generations of the input PDSE data set.

Usage Notes for COPYGRP

The control statement can be abbreviated to CG. Only one input ddname can be used per COPYGRP control statement.

The INDD and OUTDD keyword must appear on the COPYGRP statement.

A COPYGRP statement must precede the SELECT statement when members are selected for a copy, unload, or load step. A SELECT statement following a COPYGRP cannot contain the R (replace) parameter.

An EXCLUDE statement cannot follow a COPYGRP statement.

For COPYGRP, either or both of the input and output data sets must be PDSEs; if both are PDSes, no group copy function occurs.

COPYGROUP statement

Use the COPYGROUP statement to begin a group copy, unload, or load. A group consists of a member and all of its aliases. COPYGROUP treats the group as a single entity.

The syntax of the COPYGROUP statement is:

Label	Statement	Parameters
[*label*]	COPYGROUP	OUTDD=*DDname* ,INDD={*DDname*|((*DDname*,R))} [Y,LIST={YES | NO}] [Y,GENS={ALL| NONE}]

Where:

OUTDD=*DDname*
Specifies the name of a DD statement that locates the output data set.

INDD={*DDname*| ((*DDname*,R))}
Specifies the name of a DD statement that locates the input data set.

Restriction: Multiple INDD statements are not allowed for COPYGROUP.

These values can be coded:

DDname
Specifies the ddname, which is specified on a DD statement, of an input data set.

R
Specifies that a group to be copied or loaded from this input data set is to replace a group in the output data set. When this option is specified, the ddname and the R parameter must be enclosed in a set of parentheses; and the entire field, except the INDD parameter, must be enclosed in a second set of parentheses.

LIST={YES|NO}
Specifies whether the names of copied members are to be listed in the SYSPRINT data set at the end of each input data set. When this parameter is omitted, the default from the EXEC PARM field applies.

GENS={ALL|NONE}
Indicates whether the member generations of a Version 2 PDSE are copied. This keyword is valid only when copying from a PDSE V2 with member generations to another.

GENS=ALL
Specifies that all members and generations to be copied from this input PDSE data set are to replace any identically named members on the output PDSE data set.

GENS=NONE
Specifies not to copy the member generations of the input PDSE data set.

Usage Notes for COPYGROUP

The control statement can be abbreviated to CP.

Only one input ddname can be used per COPYGROUP control statement. The INDD and OUTDD keyword must appear on the COPYGROUP statement.

A COPYGROUP statement must precede the SELECT statement when members are selected for a copy, unload, or load step. A SELECT statement following a COPYGROUP cannot contain the R (replace) parameter.

An EXCLUDE statement cannot follow a COPYGROUP statement.

COPYGROUP cannot be used to load to a PDS or unload from a PDS.

When the INDD and OUTDD data sets are both PDSs, COPYGROUP can only be used if the INDD and OUTDD data sets have the same blocksize.

COPYMOD Statement

Use the COPYMOD statement to copy load module libraries that you want to reblock. The output data set must be partitioned. The input data set may be a partitioned or a sequential data set created by an unload operation. If the input to COPYMOD is not a valid load module, COPYMOD functions as a COPY.

The syntax of the COPYMOD statement is:

Label	Statement	Parameters
[*label*]	COPYMOD	OUTDD=*DDname* ,INDD=[(]{*DDname*|(*DDname*,R) }[,...][)] [,MAXBLK={*nnnnn*|*nn*K}] [,MINBLK={*nnnnn*|*nn*K}] [,LIST={YES|NO}]

where:

OUTDD=*DDname*
specifies the name of a DD statement that locates the output data set.

INDD=[(]{*DDname*| (*DDname*,R)}[,...][)]
specifies the names of DD statements that locate the input data sets.

When an INDD=appears in a record by itself (that is, not with a COPY keyword), it functions as a control statement and begins a new step in the current copy operation.

These values can be coded:

DDname
specifies the ddname, which is specified on a DD statement, of an input data set which is a load module library.

R
specifies that all members to be copied from this input data set are to replace any identically named members on the output load module library. (In addition, members whose names are not on the output data set are copied as usual.) When this option is specified, the ddname and the R parameter must be enclosed in a set of parentheses; if it is specified with the first ddname in INDD, the entire field, exclusive of the INDD parameter, must be enclosed in a second set of parentheses.

MAXBLK={*nnnnn*|*nn*K}
specifies the maximum block size for records in the output partitioned data set. MAXBLK is normally used to specify a smaller block size than the default, in order to make the record length of the data set compatible with different devices or programs.

The *nnnnn* value is specified as a decimal number; K indicates that the *nn* value is to be multiplied by 1024 bytes.

MAXBLK may be specified with or without MINBLK. Specifying a MAXBLK that is larger than the blocksize of the target data set will change the data set blocksize to the MAXBLK value.

Default: The COPYMOD MAXBLK parameter defaults to the output data set block size. If the output data set block size is zero, it defaults to the input data set block size.

MINBLK={*nnnnn*|*nn*K}
specifies the minimum block size for records in the output partitioned data set. MINBLK specifies the smallest block that should be written on the end of a track.

The MINBLK keyword is provided for compatibility with earlier MVS releases in which a larger, less-than-track-size MINBLK value could enhance program fetch performance for the module. Under normal circumstances, MINBLK should not be specified.

The *nnnnn* value is specified as a decimal number; K indicates that the *nn* value is to be multiplied by 1024 bytes.

MINBLK may be specified with or without MAXBLK.

Default: 1K (1024). If a value greater than MAXBLK is specified, MINBLK is set to the MAXBLK value actually used (whether specified or defaulted). If a value less than 1K is specified, MINBLK is set to 1K.

LIST={YES|NO}

specifies whether the names of copied members are to be listed in the SYSPRINT data set at the end of each input data set. When this parameter is omitted, the default from the EXEC PARM field applies.

INDD=Statement

In addition, when INDD, a COPY statement parameter, appears on a record other than the COPY statement, it is referred to as an INDD statement. It functions as a control statement in this context.

If one or more INDD statements are immediately followed by end of file or another COPY or COPYMOD or ALTERMOD statement, a full copy, unload, or load operation is completed using the most recent previously specified output data set.

EXCLUDE Statement

The EXCLUDE statement specifies members to be excluded from the copy, unload, or load step. All members in the input data set except those specified on each EXCLUDE statement are included in the operation. More than one EXCLUDE statement may be used in succession, in which case, the second and subsequent statements are treated as a continuation of the first.

The EXCLUDE statement must follow either a COPY statement, an ALTERMOD, a COPYMOD statement, or one or more INDD= statements. An EXCLUDE statement cannot appear with a SELECT statement in the same copy, unload, or load step. The EXCLUDE statement cannot be used with a compress-in-place or COPYGRP.

When using COPYGROUP and a SELECT MEMBER statement that uses pattern masking has been provided, an EXCLUDE statement with member name pattern masking may also be specified. When EXCLUDE MEMBER pattern masking is specified, the members excluded from the COPYGROUP operation will be a subset of those members that were selected for inclusion by the SELECT MEMBER pattern mask or masks.

If neither SELECT nor EXCLUDE is specified, the entire data set is copied (a "full copy").

The syntax of the EXCLUDE statement is:

Label	Statement	Parameters
[*label*]	EXCLUDE	MEMBER=[()]*name1*[,*name2*][,...][)]

where:

MEMBER=[()*name1*[, *name2*][,...][)]
specifies members on the input data set that are not to be copied, unloaded, or loaded to the output data set. When using COPYGROUP and a SELECT MEMBER statement that uses pattern masking has been provided, you can also provide a member pattern mask of members to be excluded from the input data set. The members are not deleted from the input data set.

The values that can be coded are:

name or filter pattern mask
identifies a specific member to be processed. All names and new names specified in one copy step must be unique. You cannot duplicate either old names, or new names, or both, under any circumstances. You cannot specify a name that is more than eight characters in length.

When using COPYGROUP and a SELECT statement that uses member name pattern masking, you can specify an EXCLUDE MEMBER= member pattern mask of members to be excluded from the input data set. The following rules and examples apply to member name filter pattern masking on the EXCLUDE statement:

- Pattern masking characters are asterisk (*) and percent sign (%).
- The asterisk character is used in the filter pattern mask to indicate that any characters within the member name that follow the pattern mask are acceptable.
- The percent character is used in the filter pattern mask to indicate that any character within the member name in a particular position is acceptable.
- The pattern must be part of a COPYGROUP operation where a SELECT statement that has specified MEMBER= with filter pattern masking is also specified.
- The pattern must be part of a EXCLUDE statement.
- Filtering will only be performed with primary member names.

See Table 19 on page 91. for examples of member name filter patterns.

The control statement operation and keyword parameters can be abbreviated to their first letters; EXCLUDE can be abbreviated to E and MEMBER can be abbreviated to M.

Each member name on an EXCLUDE statement must be unique.

SELECT Statement

The SELECT statement specifies members to be selected from input data sets to be altered, copied, loaded, or unloaded to an output data set. This statement is also used to rename or replace selected members on the output data set. More than one SELECT statement may be used in succession, in which case the second and subsequent statements are treated as a continuation of the first.

The SELECT statement must follow either a COPY statement, a COPYGRP statement, a COPYMOD statement, or one or more INDD statements. A SELECT statement cannot appear with an EXCLUDE statement in the same copy, unload, or load step, and it cannot be used with a compress-in-place function. When both the INDD and OUTDD statements are pointing to the same data set, the members in the SELECT statement are replaced or copied to a new name within the same data set, based on the replace and newname options.

When a selected member is found on an input data set, it is not searched for again in this copy step.

Each member name on a SELECT statement must be unique.

A selected member will not replace an identically named member in the output data set unless the replace option is specified on either the data set or member level. In addition, unless the replace option

is specified, a renamed member will not replace a member in the output data set that has the same new name as the renamed member.

The syntax of the SELECT statement is:

Label	Statement	Parameters
[*label*]	SELECT	MEMBER=({*name1*| (*name1*,*newname1*[,R])| (*name1*,,R)} [,{*name2*| (*name2*,*newname2*[,R])| (*name2*,,R)}][,...])

where:

MEMBER=({*name*|(*name*, *newname*[,R])|(*name*,,R)} [,...])
specifies the members to be selected from the input data set.

To rename a member, specify the old name of the member, followed by the new name and, optionally, the R (replace) parameter. This group must then be enclosed in parentheses.If member name pattern masking is being used to select the members from the input data set, then the rename option is not supported.

To replace a member, specify the name of the member and the R parameter. Two commas must separate the R from the member name, and the group must be enclosed in parentheses.

When any option within parentheses is specified anywhere in the MEMBER field, the entire field, exclusive of the MEMBER keyword, must be enclosed in a second set of parentheses.

The values that can be coded are:

name or filter pattern mask
identifies a specific member to be processed. All names and new names specified in one copy step must be unique. You cannot duplicate either old names, or new names, or both, under any circumstances. You cannot rename A to B and B to C, because B will appear twice. You cannot specify a name that is more than eight characters in length.

When the COPYGROUP statement is used, the member name can either identify a specific member to be processed, or a member name pattern mask where multiple names may meet the name pattern criteria. Member name pattern masking cannot be used with *newname*. See “SELECT Member name filter pattern masking” on page 91 for rules and examples of member name pattern masking.

newname
specifies a new name for a selected member. Member names can consist of A - Z, 0 - 9, or $ # @ _ } \, or {, and cannot be more than eight characters in length. The member is copied, unloaded, or loaded to the output data set using its new name. If the name already appears on the output partitioned data set or PDSE, the member is not copied unless replacement is also specified. *Newname* cannot be specified with ALTERMOD.

R
specifies that the input member is to replace any identically named member that exists on the output data set. If the input member's name is identical to any output member's alias name, the name will refer to the new member and not the old member.

R may not be coded with ALTERMOD or COPYGRP.

The control statement operation and keyword parameters can be abbreviated to their first letters; for example, SELECT can be abbreviated to S and MEMBER can be abbreviated to M.

Related reading: For a description of replacing members, see “Replacing Members in a Data Set” on page 68.

SELECT Member name filter pattern masking

The following rules and examples apply to member name filter pattern masking on the SELECT statement.

- The pattern masking characters are asterisk (*) and percent sign (%).
- The asterisk character is used in the filter pattern mask to indicate that any characters within the member name that follow the pattern mask are acceptable.
- The percent character is used in the filter pattern mask to indicate that any character within the member name in a particular position is acceptable.
- The pattern must be part of a COPYGROUP operation.
- The pattern must be part of a SELECT statement.
- Filtering will only be performed with primary member names.
- Alias names in a SELECT statement will be ignored and not copied.

Examples of member name filter pattern usage appear in Table 19 on page 91.

Table 19. Member name filter pattern examples

Member name pattern	Description	Examples of member names that would match	Examples of member names that would NOT match
*****	Any member name	ABC, A12#$F	None
CD*	Member names starting with CD	CDEFGH	ABCD
***CD**	Member names ending with CD	ABCD	CDEFGH
CD%	Three-character member names starting with CD	CD$	CD, CDEFG
%CD	Member names with second character C and third character D	BCDE, $CD	CD, CDEFG
%CD*	Member names with second character C and third character D	BCD, BCDE, $CD	BBCDE, CD
***CD%**	Member names with second to last character D and third to last character C	CDE, BCDE, BBCDE	CD
AB%C	Member names that have the substring AB, followed by any one character, followed by the substring C	ABKC, KKABKCKK	ABC
%	Any one-character member name	A, B, C	ABC
%%	Any two-character member name	AB, BC, CD	ABC
******	Not a valid member name pattern	Not Applicable	Not Applicable

IEBCOPY Examples

The following examples illustrate some of the uses of IEBCOPY. Table 20 on page 91 can be used as a quick-reference guide to IEBCOPY examples. The numbers in the "Example" column refer to examples that follow.

Table 20. IEBCOPY example directory

Operation	Device	Comments	Example
Alter in Place	Disk	Selected members are altered in place.	“Example 10: Alter Load Modules in Place” on page 110
Convert to PDSE	Disk	Converts a partitioned data set to a PDSE.	“Example 13: Convert a Partitioned Data Set to a PDSE” on page 112

Table 20. IEBCOPY example directory (continued)

Operation	Device	Comments	Example
COPY	Disk	Copies a full data set from one disk volume to another.	"Example 1: Copy an entire data set" on page 93
COPY	Disk	Copies three input data sets to an existing output data set.	"Example 2: Merge four data sets" on page 94
COPY	Disk	Copy a PDSE to a PDSE.	"Example 14: Copy Groups from a PDSE to a PDSE" on page 113
COPY	Disk	Selects members from two input data sets and copies them to an existing output data set. One member replaces an identically named member that already exists on the output data set.	"Example 3: Copy and Replace Selected Members of a Data Set" on page 95
COPY	Disks	Selects, excludes, and copies members from input data sets to one output data set. Illustrates multiple copy operations.	"Example 6: Multiple Copy Operations with One Output Data Set" on page 100
COPY	Disks	Selects, excludes, and copies members from input data sets to different output data sets. Illustrates multiple copy operations.	"Example 7: Multiple Copy Operations with Different Output Data Sets" on page 103
COPY	Disks	Copy an entire PDSE to a PDSE with the replace (R) option.	"Example 15: Copy Groups from a PDSE to a PDSE with Replace" on page 113
COPY	Disks	Copy a selected group by specifying an alias.	"Example 16: Copy a Selected Group from a PDSE to a PDSE" on page 114
COPYGROUP	Disks	Copy a selected group of PDS members and their aliases to another PDS.	"Example 17: Copy Selected Members and their Aliases from a PDS to a PDS" on page 114
COPYGROUP	Disks	Copy selected members from a PDS to another PDS.	"Example 18: Copy Selected Members from a PDS to a PDS" on page 115
COPY and Compress-in-place	Disk	Copies two input data sets to an existing output data set, which is compressed in place. Copies and replaces all members of one data set. Members on the output data set have the same name as those replaced.	"Example 5: Merge Data Sets and Compress the Merged Data Set" on page 98
Copy and reblock	Disk (3380)	Copies a load library to devices with different optimal block sizes. Reblocking must take place before the member can be added to the load library.	"Example 11: Replace a Load Module Using COPYMOD" on page 111
Copy and reblock	Disk (3380)	Copies load library to devices having different block sizes. Reblocks the library to size compatible with each device to which the library will be copied, then copies to those devices.	"Example 12: Reblock Load Library and Distribute It to Different Device Types" on page 111
Load	Tape and Disk	Loads a sequential data set to disk.	"Example 8: Loading a Data Set" on page 108
Unload and Compress-in-place	Disk and Tape	Unloads a partitioned data set to a tape volume to create a compressed backup copy.	"Example 4: Unload and Compress a Data Set" on page 97
Unload, Load, and COPY	Disk and Tape	Excludes, unloads, loads, and copies selected members.	"Example 9: Unload Selected Members, Load, Copy and Merge" on page 109

Examples that use **disk** or **tape** in place of actual device numbers must be changed before use. The actual device numbers depend on how your installation has defined the devices to your system.

Example 1: Copy an entire data set

In this example, a partitioned data set (DATASET5) is copied from one disk volume to another. Figure 4 on page 93 shows the input and output data sets before and after processing.

```
//COPY     JOB   ...
//JOBSTEP  EXEC  PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//SYSUT1   DD  DSNAME=DATASET5,UNIT=disk,VOL=SER=111113,
//             DISP=SHR
//SYSUT2   DD  DSNAME=DATASET4,UNIT=disk,VOL=SER=111112,
//             DISP=(NEW,KEEP),SPACE=(TRK,(5,1,2))
```

The control statements are as follows:

- SYSUT1 DD defines a partitioned data set, DATASET5, that contains two members (A and C).
- SYSUT2 DD defines a new partitioned data set, DATASET4, that is to be kept after the copy operation. Five tracks are allocated for the data set; two blocks are allocated for directory entries.
- Because the partitioned data set has only two members, SYSUT3 and SYSUT4 DD are not needed.
- Because the input and output data sets are identified as SYSUT1 and SYSUT2, the SYSIN data set is not needed. The SYSUT1 data set will be copied in full to the SYSUT2 data set. After the copy operation is finished, DATASET4 will contain the same members that are in DATASET5. However, there will be no embedded, unused space in DATASET4. If you are copying a PDSE, the processing is the same, except that there is no embedded, unused space in a PDSE.

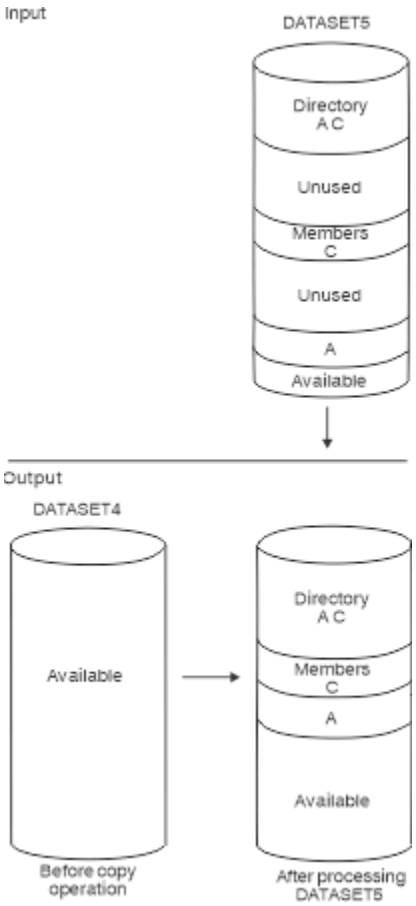


Figure 4. Copying a partitioned data set—full copy

Example 2: Merge four data sets

In this example, members are copied from three input partitioned data sets (DATASET1, DATASET5, and DATASET6) to an existing output partitioned data set (DATASET2). The sequence in which the control statements occur controls the manner and sequence in which partitioned data sets are processed. Figure 5 on page 94 shows the input and output data sets before and after processing.

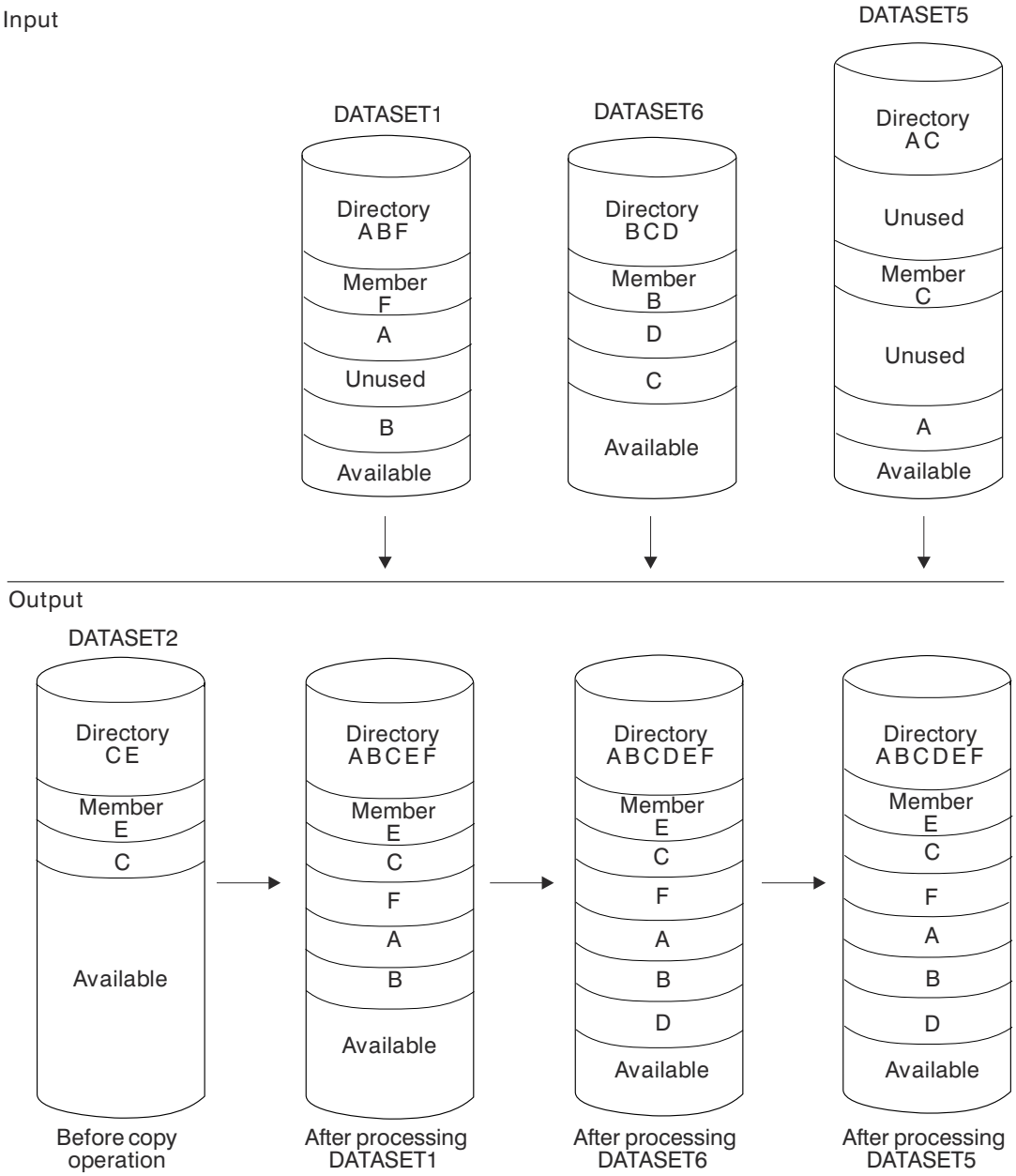


Figure 5. Copying from three input partitioned data sets

The example follows:

```
//COPY     JOB   ...
//JOBSTEP  EXEC  PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//IN1      DD  DSNAME=DATASET1,UNIT=disk,VOL=SER=111112,
//             DISP=SHR
//IN5      DD  DSNAME=DATASET5,UNIT=disk,VOL=SER=111114,
//             DISP=OLD
//OUT2     DD  DSNAME=DATASET2,UNIT=disk,VOL=SER=111115,
```

```
//             DISP=(OLD,KEEP)
//IN6      DD  DSNAME=DATASET6,UNIT=disk,VOL=SER=111117,
//             DISP=(OLD,DELETE)
//SYSUT3   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSIN    DD  *
COPYOPER   COPY  OUTDD=OUT2
                 INDD=IN1
                 INDD=IN6
                 INDD=IN5
/*
```

The control statements are as follows:

- IN1 DD defines a partitioned data set (DATASET1). This data set contains three members (A, B, and F) in fixed format with a logical record length of 80 bytes and a block size of 80 bytes.
- IN5 DD defines a partitioned data set (DATASET5). This data set contains two members (A and C) in fixed blocked format with a logical record length of 80 bytes and a block size of 160 bytes.
- OUT2 DD defines a partitioned data set (DATASET2). This data set contains two members (C and E) in fixed-block format. The members have a logical record length of 80 bytes and a block size of 240 bytes.
- IN6 DD defines a partitioned data set (DATASET6). This data set contains three members (B, C, and D) in fixed-block format with a logical record length of 80 bytes and a block size of 400 bytes. This data set is to be deleted when processing is completed.
- SYSUT3 defines a temporary spill data set.
- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPY statement and three INDD statements.
- COPY indicates the start of the copy operation. The OUTDD parameter specifies DATASET2 as the output data set.
- The first INDD statement specifies DATASET1 as the first input data set to be processed. All members (A, B, and F) are copied to DATASET2.
- The second INDD statement specifies DATASET6 as the second input data set to be processed. Processing occurs as follows:
  1. Since replacement is not specified, members B and C, which already exist in DATASET2, are not copied to DATASET2.
  2. Member D is copied to DATASET2.
  3. All members in DATASET6 are lost when the data set is deleted.
- The third INDD statement specifies DATASET5 as the third input data set to be processed. No members are copied to DATASET2 because all exist in DATASET2.

Example 3: Copy and Replace Selected Members of a Data Set

In this example, two members (A and B) are selected from two input partitioned data sets (DATASET5 and DATASET6) and copied to an existing output partitioned data set (DATASET1). Member B replaces an identically named member that already exists on the output data set. Figure 6 on page 96 shows the input and output data sets before and after processing.

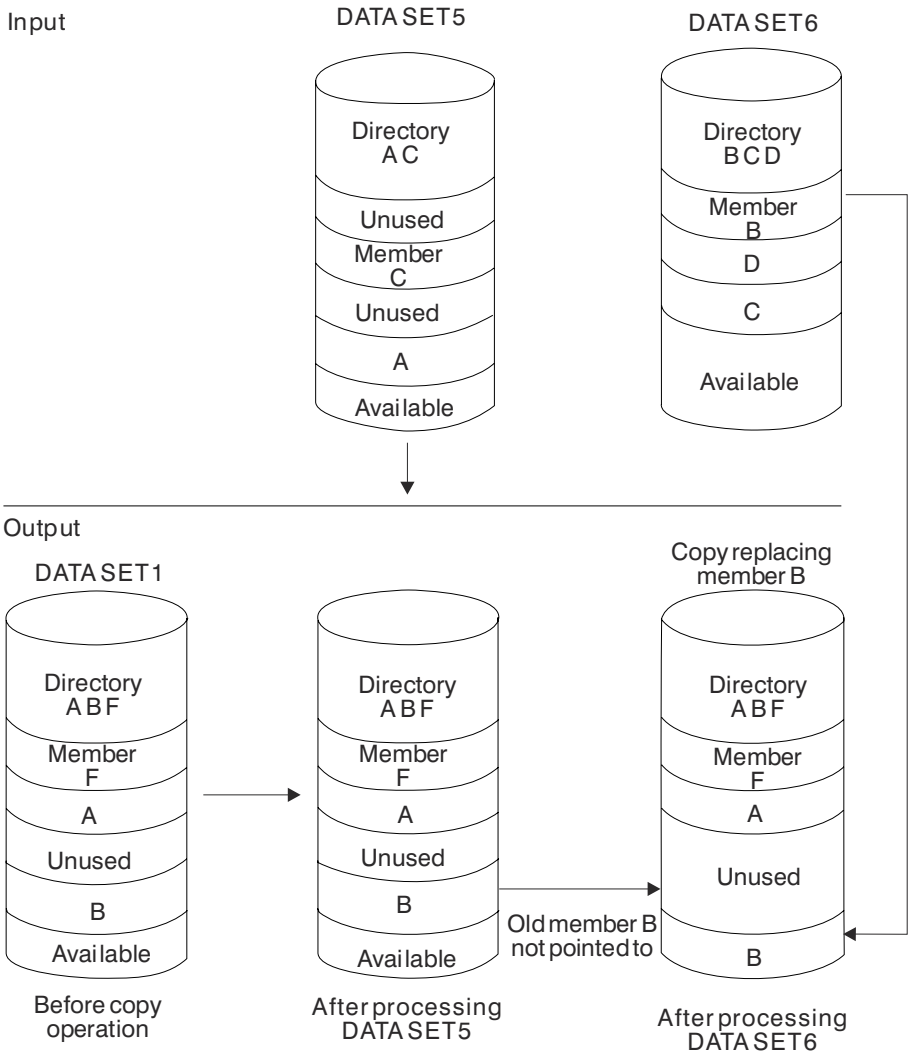


Figure 6. Selective copy with Replace specified on the member level

```
//COPY     JOB     ...
//JOBSTEP  EXEC  PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//OUT1     DD  DSNAME=DATASET1,UNIT=disk,VOL=SER=111112,
//             DISP=(OLD,KEEP)
//IN6      DD  DSNAME=DATASET6,UNIT=disk,VOL=SER=111115,
//             DISP=OLD
//IN5      DD  DSNAME=DATASET5,UNIT=disk,VOL=SER=111116,
//             DISP=(OLD,KEEP)
//SYSUT3   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSUT4   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSIN    DD  *
COPYOPER  COPY    OUTDD=OUT1,
                  INDD=IN5,IN6
          SELECT  MEMBER=((B,,R),A)
/*
```

The control statements are as follows:

- OUT1 DD defines a partitioned data set (DATASET1), which contains three members (A, B and F).
- IN6 DD defines a partitioned data set (DATASET6), which contains three members (B, C and D).
- IN5 DD defines a partitioned data set (DATASET5), which contains two members (A and C).
- SYSUT3 and SYSUT4 DD define temporary spill data sets. One track is allocated for each on a disk volume.

- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPY statement, an INDD statement, and a SELECT statement.
- COPY indicates the start of the copy operation. The use of a SELECT statement causes a selective copy. The OUTDD parameter specifies DATASET1 as the output data set.
- INDD specifies DATASET5 as the first input data set to be processed and DATASET6 as the second input data set to be processed. Processing occurs as follows:
  1. Selected members are searched for on DATASET5.
  2. Member A is found, but is not copied to DATASET1 because DATASET1 already has a member named "A", and the replace option is not specified for member A.
  3. Selected members not found on DATASET5 are searched for on DATASET6.
  4. Member B is found and copied to DATASET1, even though there is already a DATASET1 member "B" in DATASET1, because the replace option is specified for member B on the member level. The pointer in DATASET1's directory is changed to point to the new (copied) member B; thus, the space occupied by the old member B is unused.
- SELECT specifies the members to be selected from the input data sets (DATASET5 and DATASET6) to be copied to the output data set (DATASET1).

Example 4: Unload and Compress a Data Set

In this example, a partitioned data set is unloaded to a tape volume to create a backup copy of the data set. If this step is successful, the partitioned data set is to be compressed in place.

```
//SAVE     JOB  ...
//STEP1    EXEC PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//SYSUT1   DD  DSNAME=PARTPDS,UNIT=disk,VOL=SER=PCP001,
//             DISP=OLD
//SYSUT2   DD  DSNAME=SAVDATA,UNIT=tape,VOL=SER=TAPE03,
//             DISP=(NEW,KEEP),LABEL=(,SL)
//SYSUT3   DD  DSNAME=TEMP1,UNIT=disk,VOL=SER=111111,
//             DISP=(NEW,DELETE),SPACE=(80,(60,45))
//SYSIN    DD  DUMMY
//STEP2    EXEC PGM=IEBCOPY,COND=(0,NE),PARM='SIZE=500K'
//SYSPRINT DD  SYSOUT=A
//COMPDS   DD  DSNAME=PARTPDS,UNIT=disk,DISP=OLD,
//             VOL=SER=PCP001
//SYSUT3   DD  DSNAME=TEMPA,UNIT=disk,VOL=SER=111111,
//             DISP=(NEW,DELETE),SPACE=(80,(60,45))
//SYSIN    DD  *
           COPY OUTDD=COMPDS,INDD=COMPDS
/*
```

The control statements are as follows:

- SYSUT1 DD defines a partitioned data set (PARTPDS) that resides on a disk volume and is assumed to have 700 members. The number of members is used to calculate the space allocation on SYSUT3.
- SYSUT2 DD defines a sequential data set to hold PARTPDS in unloaded form. Block size information can optionally be added; this data set must be NEW.
- SYSUT3 DD defines the temporary spill data set. The SYSUT4 data set is never used for an unload operation.
- SYSIN DD defines the control data set. Because SYSIN is dummied and SYSUT2 defines a sequential data set, all members of the SYSUT1 data set will be unloaded to the SYSUT2 data set.
- The second EXEC statement marks the beginning of the compress-in-place operation. The SIZE parameter indicates that the buffers are to be as large as possible. The COND parameter indicates that the compress-in-place is to be performed only if the unload operation was successful.
- COMPDS DD defines a partitioned data set (PARTPDS) that contains 700 members and resides on a disk volume.
- SYSUT3 DD defines the temporary spill data set to be used if there is not enough space in main storage for the input data set's directory entries. TEMPA contains one 80-character record for each member.

- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPY statement.
- COPY marks the beginning of the copy operation. Because the same DD statement is specified for both the INDD and OUTDD operands, the data set is compressed in place. If a PDSE is being used, this step will not be processed.

If you want to unload more than one data set in a single use of IEBCOPY, you must use a separate COPY statement for each unload operation. Only one input data set may be specified in an unload operation.

Example 5: Merge Data Sets and Compress the Merged Data Set

In this example, two input partitioned data sets (DATASET5 and DATASET6) are copied to an existing output partitioned data set (DATASET1). In addition, all members on DATASET6 are copied; members on the output data set that have the same names as the copied members are replaced. After DATASET6 is processed, the output data set (DATASET1) is compressed in place. Figure 7 on page 99 shows the input and output data sets before and after processing.

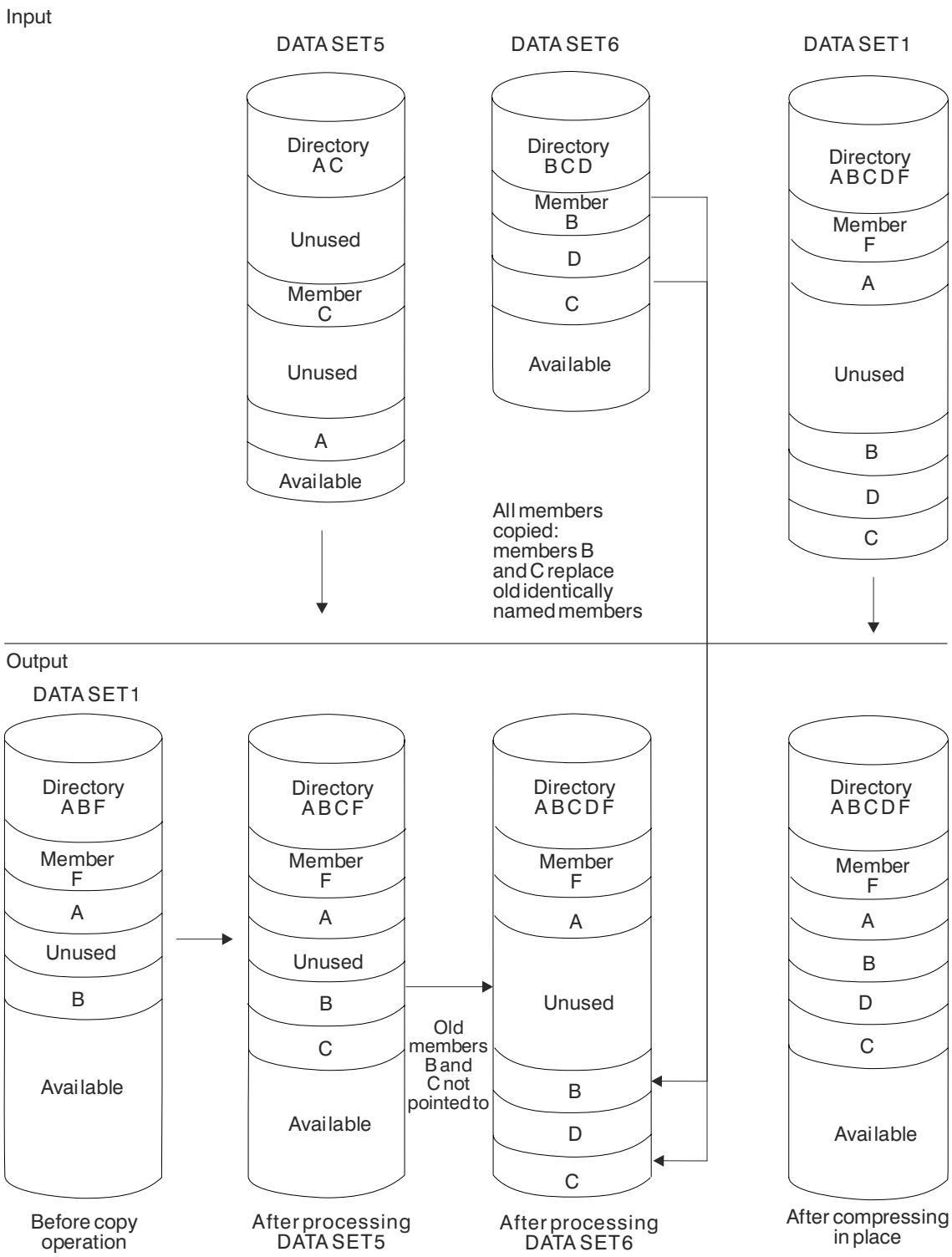


Figure 7. Compress-in-Place following full copy with "replace"specified

```
//COPY     JOB  ...
//JOBSTEP  EXEC PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//INOUT1   DD  DSNAME=DATASET1,UNIT=disk,VOL=SER=111112,
//             DISP=(OLD,KEEP)
//IN5      DD  DSNAME=DATASET5,UNIT=disk,VOL=SER=111114,
//             DISP=OLD
//IN6      DD  DSNAME=DATASET6,UNIT=disk,VOL=SER=111115,
//             DISP=(OLD,KEEP)
```

```
//SYSUT3   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSUT4   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSIN    DD  *
COPYOPER COPY  OUTDD=INOUT1,INDD=(IN5,(IN6,R),INOUT1)
/*
```

The control statements are as follows:

- INOUT1 DD defines a partitioned data set (DATASET1), which contains three members (A, B and F).
- IN5 DD defines a partitioned data set (DATASET5), which contains two members (A and C).
- IN6 DD defines a partitioned data set (DATASET6), which contains three members (B, C and D).
- SYSUT3 and SYSUT4 DD define temporary spill data sets. One track is allocated for each on a disk volume.
- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPY statement.
- COPY indicates the start of the copy operation. The OUTDD operand specifies DATASET1 as the output data set.

  The INDD operand specifies DATASET5 as the first input data set to be processed. It then specifies DATASET6 as the second input data set to be processed. In addition, the replace option is specified for all members copied from DATASET6. Finally, it specifies DATASET1 as the last input data set to be processed. Since DATASET1 is also the output data set, DATASET1 is compressed in place. However, if DATASET1 is a PDSE, the compress-in-place operation will not be processed.

  Processing occurs as follows:

  1. Member A is not copied from DATASET5 into DATASET1 because it already exists on DATASET1 and the replace option was not specified for DATASET5.
  2. Member C is copied from DATASET5 to DATASET1, occupying the first available space.
  3. All members are copied from DATASET6 to DATASET1, immediately following the last member. Members B and C are copied even though the output data set already contains members with the same names because the replace option is specified on the data set level.

  The pointers in DATASET1's directory are changed to point to the new members B and C. Thus, the space occupied by the old members B and C is unused. The members currently on DATASET1 are compressed in place, thereby eliminating embedded unused space.

Example 6: Multiple Copy Operations with One Output Data Set

In this example, members are selected, excluded, and copied from input partitioned data sets onto an output partitioned data set. This example is designed to illustrate multiple copy operations.

The example follows. Figure 8 on page 102 and Figure 9 on page 103 show the input and output data sets before and after processing.

```
//COPY     JOB  ...
//JOBSTEP  EXEC PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//INOUTA   DD  DSNAME=DATASETA,UNIT=disk,VOL=SER=111113,
//             DISP=OLD
//INB      DD  DSNAME=DATASETB,UNIT=disk,VOL=SER=111115,
//             DISP=(OLD,KEEP)
//INC      DD  DSNAME=DATASETC,UNIT=disk,VOL=SER=111114,
//             DISP=(OLD,KEEP)
//IND      DD  DSNAME=DATASETD,UNIT=disk,VOL=SER=111116,
//             DISP=OLD
//INE      DD  DSNAME=DATASETE,UNIT=disk,VOL=SER=111117,
//             DISP=OLD
//OUTX     DD  DSNAME=DATASETX,UNIT=disk,VOL=SER=111112,
//             DISP=(NEW,KEEP),SPACE=(TRK,(3,1,2))
//SYSUT3   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSIN    DD  *
COPERST1 COPY  O=OUTX,I=INOUTA
         COPY  OUTDD=INOUTA,INDD=INOUTA
               INDD=INB
         COPY  OUTDD=INOUTA
```

```
                INDD=IND
        EXCLUDE  MEMBER=MM
                INDD=INC
         SELECT  MEMBER=((ML,MD,R))
                INDD=INE
  /*
```

The control statements are as follows:

- INOUTA DD defines a partitioned data, DATASETA, which contains seven members (MA, MB, MC, MD, ME, MF and MG).
- INB DD defines a partitioned data set, DATASETB, which contains two members (MA and MJ).
- INC DD defines a partitioned data set, DATASETC, which contains four members (MF, ML, MM and MN).
- IND DD defines a partitioned data set, DATASETD, which contains two members (MM and MP).
- INE DD defines a partitioned data set, DATASETE, which contains four members (MD, ME, MF and MT).
- OUTX DD defines a partitioned data set (DATASETX). This data set is new and is to be kept after the copy operation. Three tracks are allocated for the data set on a disk volume. Two blocks are allocated for directory entries.
- SYSUT3 defines a temporary spill data set.
- SYSIN DD defines the control data set, which follows in the input stream. The data set contains two COPY statements, several INDD statements, a SELECT statement, and an EXCLUDE statement.
- The first COPY statement indicates the start of the first copy operation. This copy operation is done to create a backup copy of DATASETA.
- The second COPY statement indicates the start of another copy operation. Since DATASETA is specified in both the INDD and OUTDD parameters, DATASETA is compressed in place.

  The output data set is compressed in place first to save space because it is known that it contains embedded, unused space.

  The following INDD statement specifies DATASETB as the next input data set to be copied. Only member MJ is copied, because DATASETA already contains a member named "MA".
- The third COPY statement indicates the start of the third copy operation. The OUTDD parameter specifies DATASETA as the output data set. This copy operation contains more than one copy step.

  The first INDD statement specifies DATASETD as the first input data set to be processed. Only member MP is copied to DATASETA because the EXCLUDE statement specifies that member MM is to be excluded from the first copy step within this copy operation.

  The second INDD statement marks the beginning of the second copy step for this copy operation and specifies DATASETC as the second input data set to be processed. The SELECT statement specifies that member ML of DATASETC is to be renamed "MD", and that the new member will replace any member in DATASETA that happens to be named "MD". Member ML is searched for, found, copied to DATASETA and renamed.

  The third INDD statement marks the beginning of the third copy step for this copy operation and specifies DATASETE as the last data set to be copied. Only member MT is copied, because DATASETA already contains the other members. Because the INDD statement is not followed by an EXCLUDE or SELECT statement, a full copy is performed.

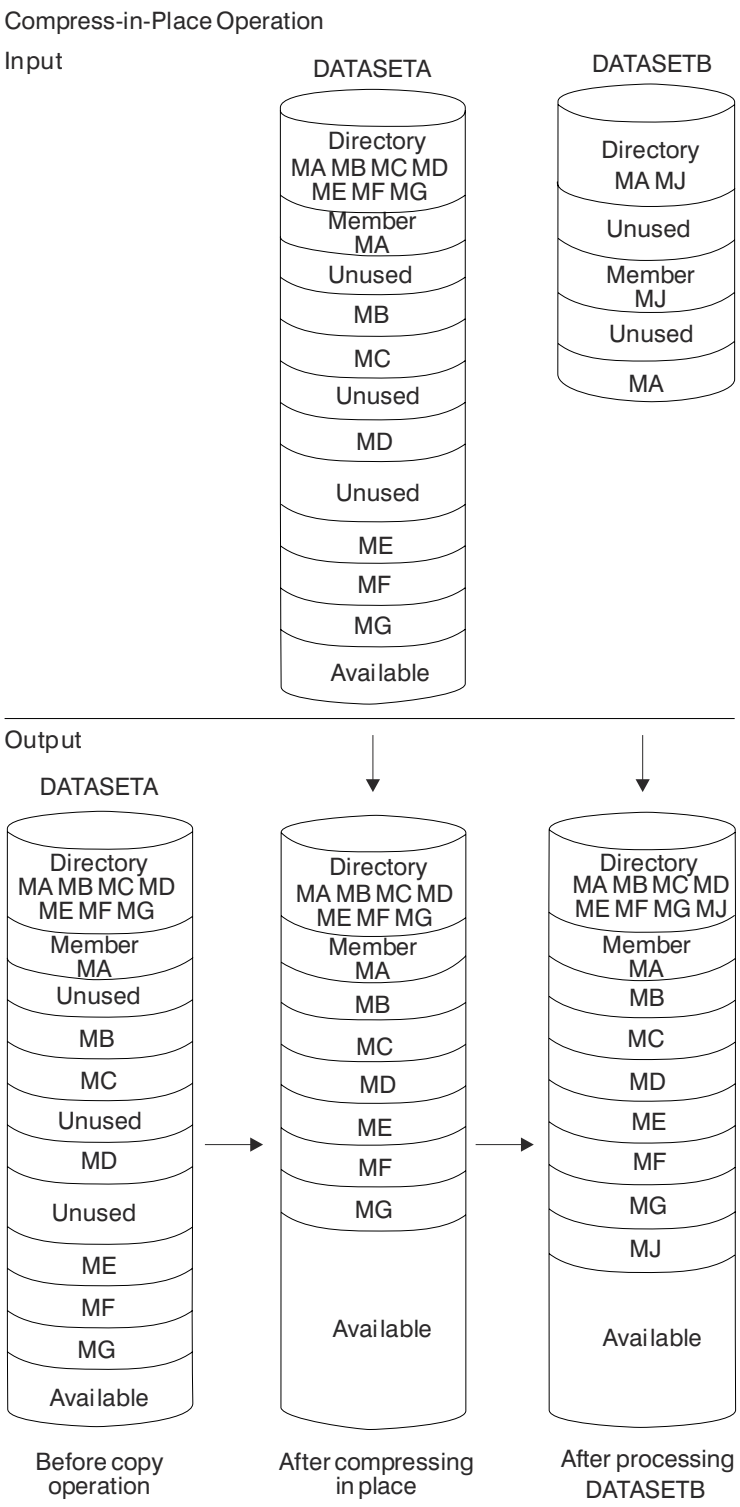


Figure 8. Multiple copy operations/copy steps (Part 1 of 2)

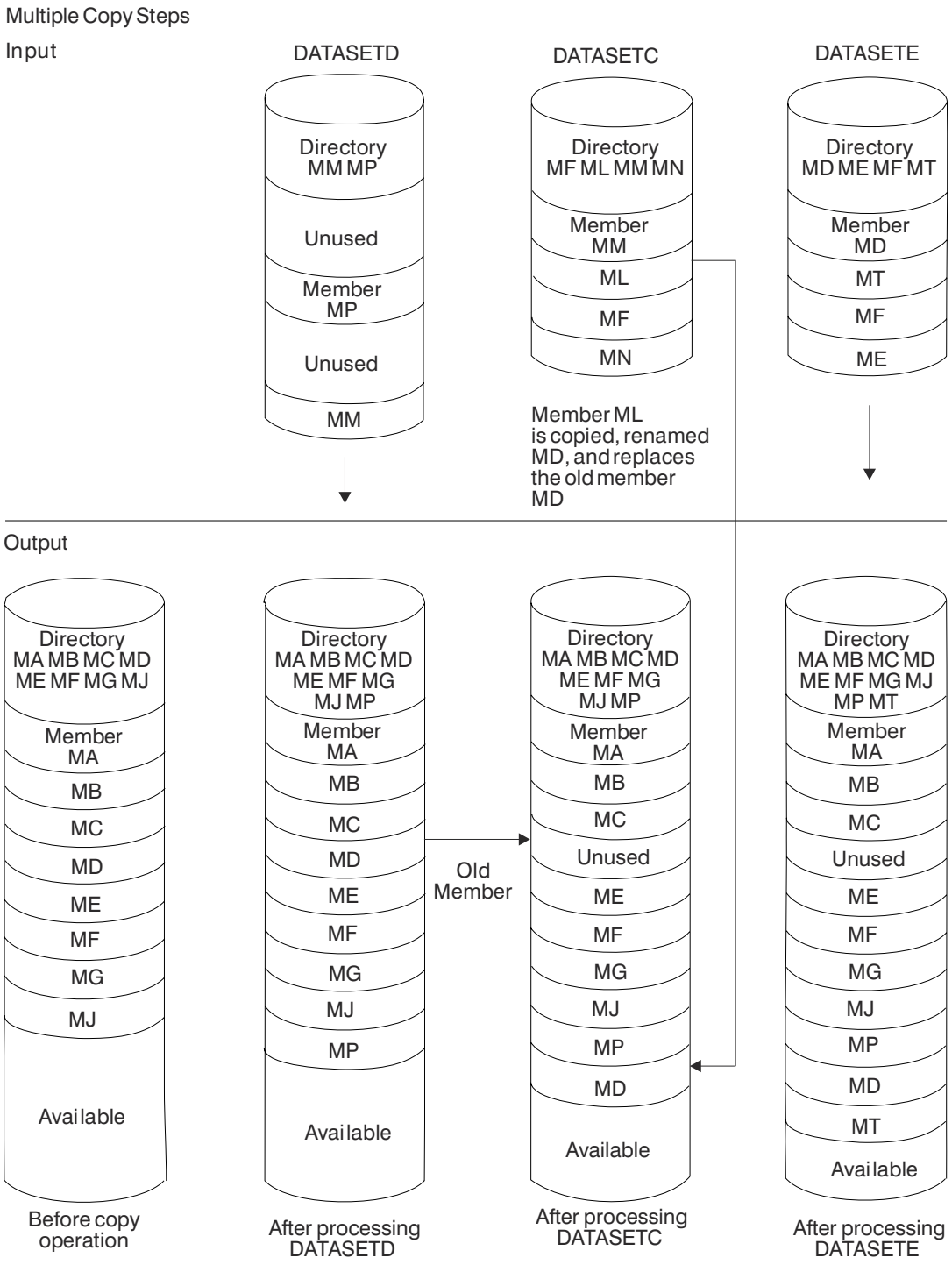


Figure 9. Multiple copy operations/copy steps (Part 2 of 2)

Example 7: Multiple Copy Operations with Different Output Data Sets

In this example, members are selected, excluded, and copied from input partitioned data sets to an output partitioned data set. This example is designed to illustrate multiple copy operations. Figure 10 on page 106, Figure 11 on page 107, and Figure 12 on page 108 show the input and output data sets before and after processing.

The example follows:

```
//COPY     JOB  ...
//JOBSTEP  EXEC PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//OUTA     DD  DSNAME=DATASETA,UNIT=disk,VOL=SER=111113,
//             DISP=OLD
//INOUTB   DD  DSNAME=DATASETB,VOL=SER=111115,UNIT=disk,
//             DISP=(OLD,KEEP)
//INOUTC   DD  DSNAME=DATASETC,VOL=SER=111114,UNIT=disk,
//             DISP=(OLD,KEEP)
//INOUTD   DD  DSNAME=DATASETD,VOL=SER=111116,DISP=OLD,
//             UNIT=disk
//INE      DD  DSNAME=DATASETE,VOL=SER=111117,DISP=OLD,
//             UNIT=disk
//SYSUT3   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSUT4   DD  UNIT=SYSDA,SPACE=(TRK,(1))
//SYSIN    DD  *
          COPY  OUTDD=OUTA
                INDD=INE
        SELECT  MEMBER=(MA,MJ)
                INDD=INOUTC
       EXCLUDE  MEMBER=(MM,MN)
          COPY  OUTDD=INOUTB,INDD=INOUTD
                INDD=((INOUTC,R),INOUTB)
          COPY  OUTDD=INOUTD,INDD=((INOUTB,R))
        SELECT  MEMBER=MM
/*
```

The control statements are as follows:

- OUTA DD defines a partitioned data set, DATASETA, which contains three members (MA, MB and MD).
- INOUTB DD defines a partitioned data set, DATASETB, which contains two members (MA and MJ).
- INOUTC DD defines a partitioned data set, DATASETC, which contains four members (MF, ML, MM and MN).
- INOUTD DD defines a partitioned data set, DATASETD, which contains two members (MM and MP).
- INE DD defines a partitioned data set, DATASETE, which contains three members (MA, MJ and MK).
- SYSUT3 and SYSUT4 DD define temporary spill data sets. One track is allocated for each on a disk volume.
- SYSIN DD defines the control data set, which follows in the input stream. The data set contains three COPY statements, two SELECT statements, one EXCLUDE statement, and several INDD statements.
- The first COPY statement indicates the start of a copy operation. The OUTDD operand specifies DATASETA as the output data set.

  The first INDD statement specifies DATASETE as the first input data set to be processed. The SELECT statement specifies that members MA and MJ are to be copied from DATASETE to DATASETA. Processing occurs as follows:

  1. Member MA is searched for and found, but is not copied because the replace option is not specified.
  2. Member MJ is searched for, found, and copied to DATASETA.

  The second INDD statement marks the end of the first copy step and the beginning of the second copy step within the first copy operation. It specifies DATASETC as the second input data set to be processed. Members MF and ML, which are not named on the EXCLUDE statement, are copied because DATASETA contains neither one of them. EXCLUDE specifies that members MM and MN are not to be copied from DATASETC to DATASETA.
- The second COPY statement indicates the start of another copy operation. The OUTDD parameter specifies DATASETB as the output data set. The INDD parameter specifies DATASETD as the first input data set to be processed. Members MP and MM are copied to DATASETB.

  The next INDD statement specifies DATASETC as the second and DATASETB as the third input data set to be processed. Members MF, ML, MM and MN are copied from DATASETC. Member MM is copied, although DATASETB already contains a member MM, because the replace option is specified.

  The pointer in DATASETB's directory is changed to point to the new (copied) member MM. Thus, the space occupied by the replaced member MM is embedded, unused space. DATASETB is then

compressed in place to remove embedded, unused space. (DATASETB is specified as both the input and output data sets.)

- The third COPY statement indicates the start of the last copy operation. The OUTDD parameter specifies DATASETD as the output data set. The INDD parameter specifies DATASETB as the input data set.

  SELECT specifies that member MM is to be copied from DATASETB to DATASETD. Since the replace option is specified on the data set level, member MM is copied and replaces DATASETDs member MM.

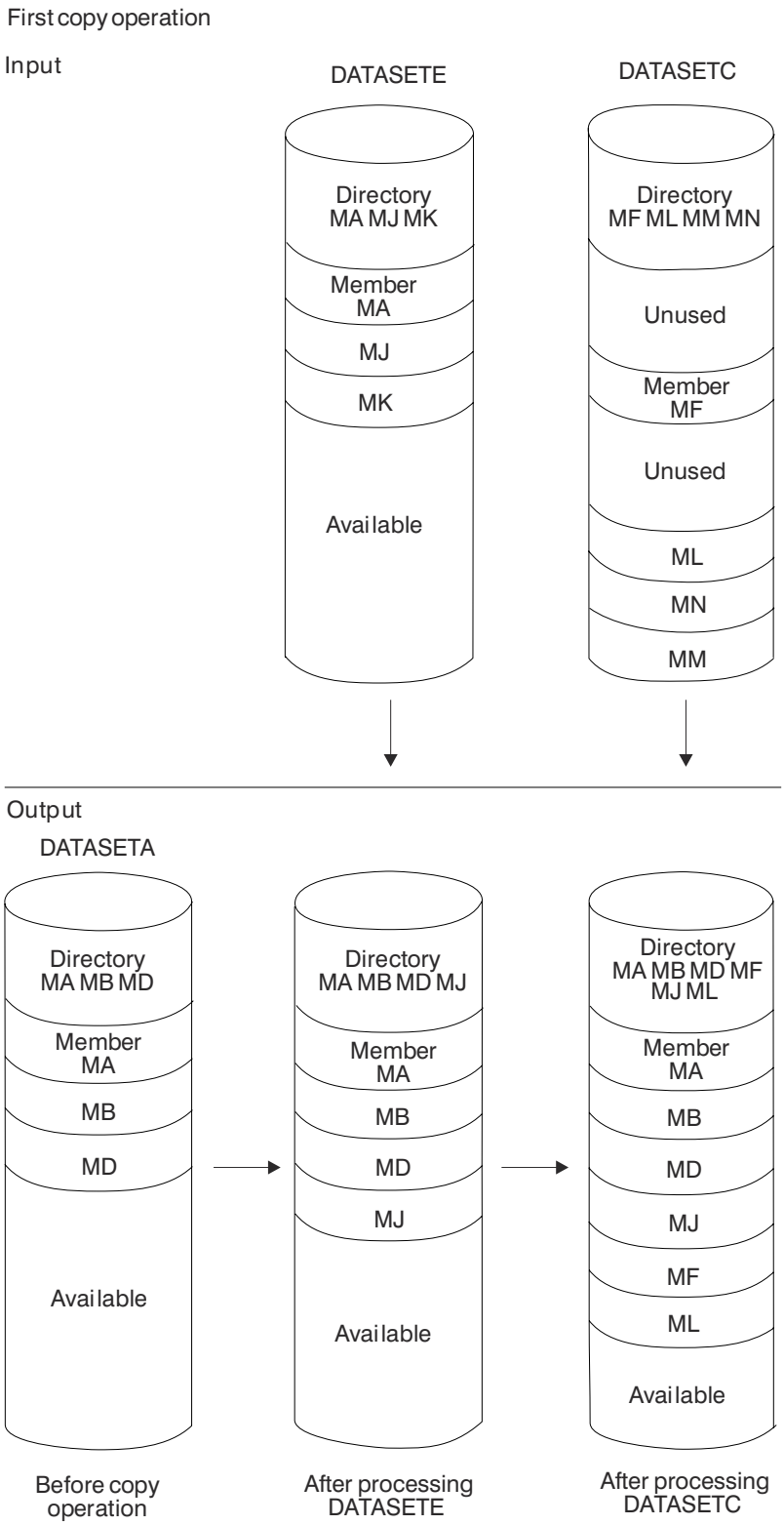


Figure 10. Multiple copy operations/copy steps within a job step (Part 1 of 3)

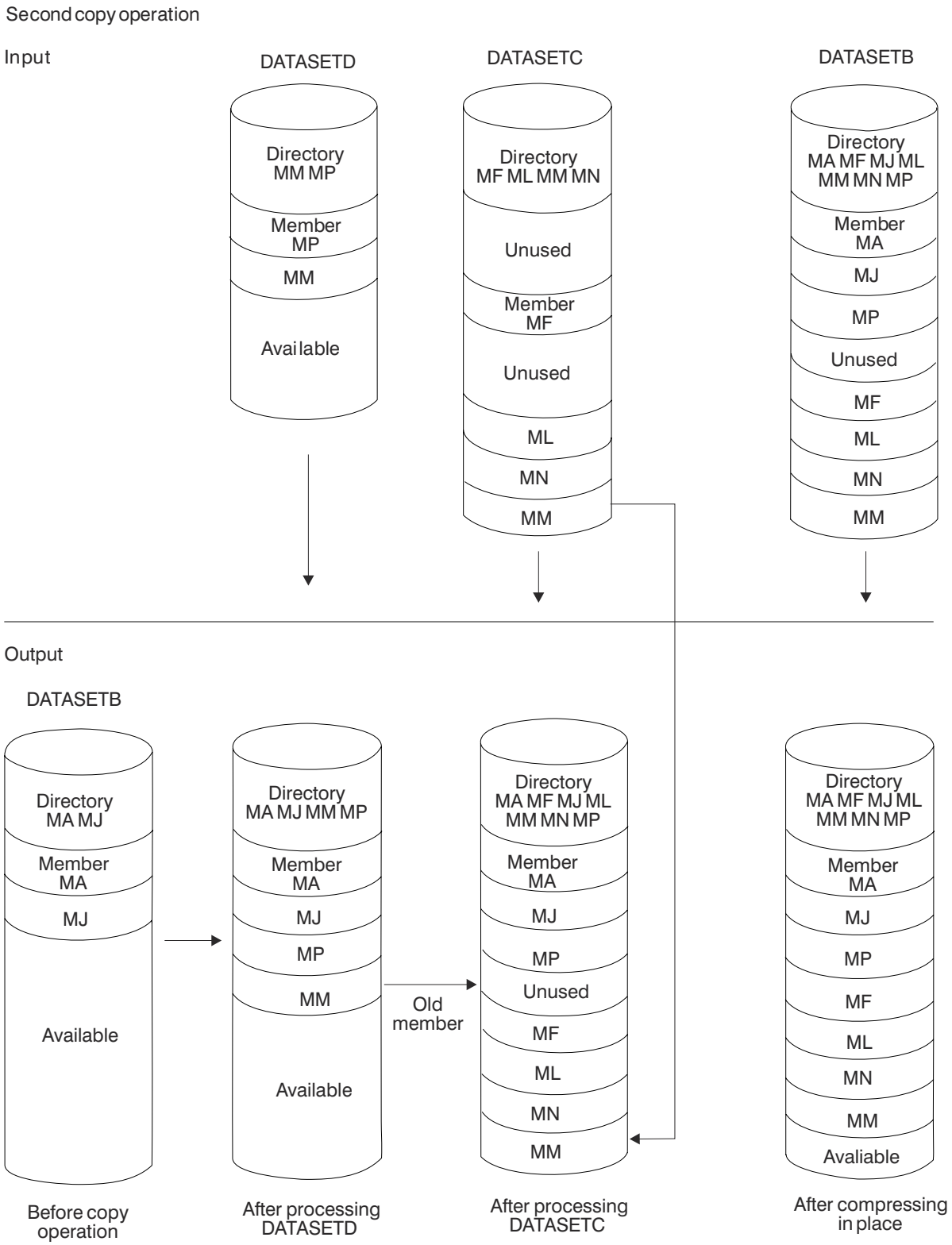


Figure 11. Multiple copy operations/copy steps within a job step (Part 2 of 3)

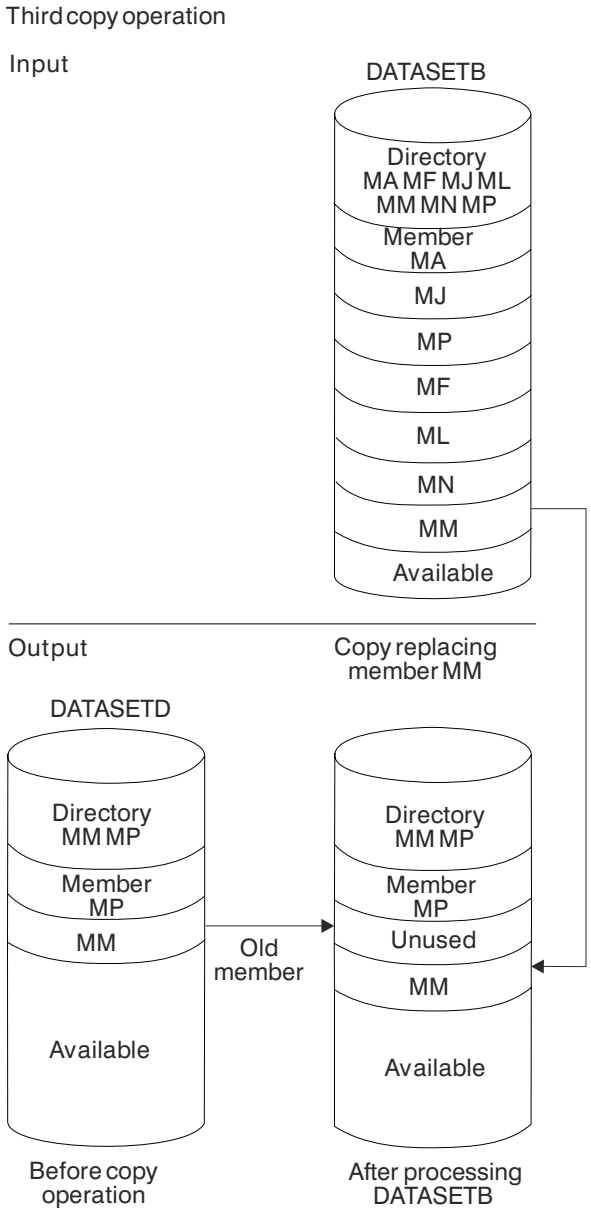


Figure 12. Multiple copy operations/copy steps within a job step (Part 3 of 3)

Example 8: Loading a Data Set

In this example, a sequential data set that was created by an IEBCOPY unload operation is loaded.

```
//LOAD     JOB  ...
//STEPA    EXEC PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//SYSUT1   DD  DSNAME=UNLOADSET,UNIT=tape,LABEL=(,SL),
//             VOL=SER=TAPE01,DISP=OLD
//SYSUT2   DD  DSNAME=DATASET4,UNIT=disk,VOL=SER=2222222,
//             DISP=(NEW,KEEP),SPACE=(CYL,(10,5,10))
//SYSUT3   DD  DSN=TEMP1,UNIT=disk,VOL=SER=111111,
//             DISP=(NEW,DELETE),SPACE=(80,(15,1))
//SYSIN    DD  DUMMY
/*
```

The control statements are as follows:

- SYSUT1 DD defines a sequential data set that was previously created by an IEBCOPY unload operation. The data set contains 28 members in sequential organization.
- SYSUT2 DD defines a new partitioned data set on a disk volume. This data set is to be kept after the load operation. Ten cylinders are allocated for the data set; ten blocks are allocated for directory entries.
- SYSUT3 DD defines a temporary spill data set on a disk volume.
- SYSIN DD defines the control data set. Because SYSIN is dummied, SYSUT1 defines a sequential data set, and SYSUT2 defines a partitioned data set, the entire SYSUT1 data set will be loaded into the SYSUT2 data set.

Example 9: Unload Selected Members, Load, Copy and Merge

In this example, members are selected, excluded, unloaded, loaded, and copied. Processing will occur as follows:

1. unload, excluding members
2. unload, selecting members
3. load and copy to merge members

```
//COPY     JOB  ...
//STEP     EXEC PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//PDS1     DD  DSNAME=ACCOUNTA,UNIT=disk,VOL=SER=333333,
//             DISP=OLD
//PDS2     DD  DSNAME=ACCOUNTB,UNIT=disk,VOL=SER=333333,
//             DISP=OLD
//SEQ1     DD  DSNAME=SAVAC,UNIT=disk,VOL=SER=333333,
//             DISP=(NEW,KEEP),SPACE=(CYL,(5,2))
//SEQ2     DD  DSNAME=SAVACB,UNIT=tape,VOL=SER=T01911,
//             DISP=(NEW,KEEP),LABEL=(,SL)
//NEWUP    DD  DSNAME=NEWACC,UNIT=tape,VOL=SER=T01219,
//             DISP=OLD,LABEL=(,SL)
//MERGE    DD  DSNAME=ACCUPDAT,UNIT=disk,VOL=SER=222222,
//             DISP=OLD
//SYSUT3   DD  DSNAME=TEMP1,VOL=SER=666666,UNIT=disk,
//             DISP=(NEW,DELETE),SPACE=(80,(1,1))
//SYSUT4   DD  DSNAME=TEMP2,VOL=SER=666666,UNIT=disk,
//             DISP=(NEW,DELETE),SPACE=(256,(1,1)),DCB=(KEYLEN=8)
//SYSIN    DD  *
     COPY      OUTDD=SEQ1,INDD=PDS1
     EXCLUDE   MEMBER=(D,C)
     COPY      OUTDD=SEQ2,INDD=PDS2
     SELECT    MEMBER=(A,K)
     COPY      OUTDD=MERGE,INDD=((NEWUP,R),PDS1,PDS2)
     EXCLUDE   MEMBER=A
/*
```

The control statements are as follows:

- PDS1 DD defines a partitioned data set called ACCOUNTA that contains six members (A, B, C, D, E, and F).
- PDS2 DD defines a partitioned data set called ACCOUNTB that contains three members (A, K, and L).
- SEQ1 DD defines a new sequential data set called SAVAC.
- SEQ2 DD defines a new sequential data set called SAVACB on a tape volume. The tape has IBM standard labels.
- NEWUP DD defines an old sequential data set called NEWACC that is the unloaded form of a partitioned data set that contains eight members (A, B, C, D, M, N, O, and P). It resides on a tape volume.
- MERGE DD defines a partitioned data set called ACCUPDAT that contains six members (A, B, C, D, Q, and R).
- SYSUT3 and SYSUT4 DD define temporary spill data sets.
- SYSIN DD defines the control data set, which follows in the input stream.

- The first COPY statement indicates the start of the first copy operation. The OUTDD parameter specifies that SAVAC is the output data set, and the INNDD parameter specifies that ACCOUNTA is the input data set. Because SAVAC is a sequential data set, ACCOUNTA will be unloaded in this copy operation.

  The EXCLUDE statement specifies that members D and C are not to be unloaded to SAVAC with the rest of ACCOUNTA.
- The second COPY statement indicates the start of the second copy operation. The OUTDD parameter specifies that SAVACB is the output data set, and the INDD parameter specifies that ACCOUNTB is the input data set. Because SAVACB is a sequential data set, ACCOUNTB will be unloaded in this copy operation.

  The SELECT statement specifies that members A and K are the only members of ACCOUNTB that are to be unloaded to SAVACB.
- The third COPY statement indicates the start of the last copy operation. The OUTDD parameter specifies that ACCUPDAT is the output data set. The EXCLUDE statement specifies that member A is excluded from this copy operation. The three data sets specified in the INDD parameter will be processed as follows:

  1. The data set NEWACC is a sequential data set, so it is loaded into ACCUPDAT. Because the replace option is specified, members B, C, and D in NEWACC replace identically named members in ACCUPDAT. The remaining members of NEWACC are also copied to ACCUPDAT, except for A, which is excluded from the copy operation.
  2. The data set ACCOUNTA is a partitioned data set, so its members are copied to ACCUPDAT. Because replacement is not specified, only members E and F are copied.
  3. The data set ACCOUNTB is a partitioned data set, so its members are copied to ACCUPDAT. Only members K and L are copied.

Example 10: Alter Load Modules in Place

In this example, all members of data set MODLIBJ, members MODX, MODY, and MODZ of data set MODLIBK, and all members of data set MODLIBL, except MYMACRO and MYJCL, are altered in place.

```
//ALTERONE JOB  ...
//STEPA    EXEC PGM=IEBCOPY
//SYSPRINT DD  SYSOUT=A
//SYSUT3   DD  UNIT=SYSDA,SPACE=(TRK,(5,1))
//LIBJ     DD  DSNAME=MODLIBJ,DISP=(OLD,KEEP)
//LIBK     DD  DSNAME=MODLIBK,DISP=(OLD,KEEP)
//LIBL     DD  DSNAME=MODLIBL,DISP=(OLD,KEEP)
//SYSIN    DD  *
     ALTERMOD  OUTDD=LIBJ
     ALTERMOD  OUTDD=LIBK,LIST=NO
     SELECT    MEMBER=(MODX,MODY,MODZ)
     ALTERMOD  OUTDD=LIBL
     EXCLUDE   MEMBER=(MYMACRO,MYJCL)
/*
```

The control statements are as follows:

- LIBJ DD defines the partitioned data set MODLIBJ, which has been previously created and cataloged.
- LIBK DD defines the partitioned data set MODLIBK, which has been previously created and cataloged.
- LIBL DD defines the partitioned data set MODLIBL, which has been previously created and cataloged.
- SYSIN DD defines the control data set, which follows in the input stream.
- The first ALTERMOD statement specifies that the entire data set defined in LIBJ is to be altered in place.
- The second ALTERMOD statement plus the following SELECT statement indicates that members MODX, MODY, and MODZ are to be altered in place. The remainder of MODLIBK is unchanged.
- The third ALTERMOD statement plus the following EXCLUDE statement indicates that all of MODLIBL is to be altered in place except the members called MYMACRO and MYJCL. These members remain unchanged.

Example 11: Replace a Load Module Using COPYMOD

In this example, a load module in an existing load library is replaced by another module. The new module originally resides on a 3390 DASD device, whereas the load library to which it is copied resides on a 3380. Because the module has a block size larger than the block size assigned to the output data set, the module must be reblocked before it is added to the load library.

This example illustrates how you can transfer load modules between devices of different sizes. In this case, updated modules are created on a 3390 and tested before being added to the load library for general use.

```
//STEP1    EXEC PGM=IEBCOPY
//REPLACE  JOB  ...
//SYSPRINT DD  SYSOUT=A
//TESTLIB  DD  DSN=JOHNDOE.COBOL.TESTLOAD,DISP=SHR,UNIT=3390,
//             VOL=SER=TEST01,DCB=(BLKSIZE=23470)
//PRODLIB  DD  DSN=PAYROLL.MASTER.LOADLIB,DISP=(OLD,KEEP)
//             UNIT=3380,VOL=SER=PROD01,DCB=(BLKSIZE=19069)
//SYSIN    DD  *
             COPYMOD OUTDD=PRODLIB,INDD=TESTLIB
             SELECT MEMBER=((WAGETAX,,R))
/*
```

The control statements are as follows:

- TESTLIB DD defines a load library on a 3390 direct access device. It has a block size of 23470.
- PRODLIB DD defines a load library on a 3380 direct access device. It has a block size of 19069.
- SYSIN DD defines the control data set, which follows in the input stream.
- The COPYMOD statement identifies PAYROLL.MASTER.LOADLIB as the output data set and JOHNDOE.COBOL.TESTLOAD as the input data set. The SELECT statement indicates that the load module WAGETAX is to be copied from the input data set and is to replace any member with that name that is in the output data set. The member is also reblocked to 19069.

Note that, in this case, COPYMOD has to be used in order to copy the member WAGETAX into the PAYROLL.MASTER.LOADLIB. Because the original block size of WAGETAX is larger than the largest block size that can reside in the output data set, attempting this operation with the COPY statement would be unsuccessful. The problem would be attributed to a DCB validation error because of incorrect block size.

Example 12: Reblock Load Library and Distribute It to Different Device Types

In this example, a load library is distributed (by copying it) to devices whose maximum block size differs from that on which the original load library resides. The library is first reblocked to a maximum block size that is compatible with each of the devices to which the library will be copied. Then, the library is copied to those devices.

This example illustrates how load libraries can be developed on one type of direct access device and then distributed to other types of direct access devices.

```
//RBLKCOPY JOB  ...
//REBLOCK  EXEC PGM=IEBCOPY
//SYSPRINT DD   SYSOUT=A
//TESTED   DD   DSN=TESTED.MASTER.LOADLIB,DISP=SHR
//STDSIZE  DD   DSN=PROGRAM.MASTER.LOADLIB,DISP=(OLD,KEEP),
//              UNIT=3390,VOL=SER=PROG01,DCB=(BLKSIZE=23470)
//SYSIN    DD   *
             COPYMOD OUTDD=STDSIZE,INDD=TESTED,MAXBLK=13030
/*
//DISTRIB  EXEC PGM=IEBCOPY
//SYSPRINT DD   SYSOUT=A
//STDSIZE  DD   DSN=PROGRAM.MASTER.LOADLIB,DISP=SHR
//LIB3350  DD   DSN=PROGRAM.LIB3380.LOADLIB,DISP=(OLD,KEEP),
//              UNIT=3380,VOL=SER=PACK01,DCB=(BLKSIZE=19069)
//LIB3330  DD   DSN=PROGRAM.LIB3380.LOADLIB,DISP=(OLD,KEEP),
//              UNIT=3380,VOL=SER=PACK02,DCB=(BLKSIZE=13030)
//SYSIN    DD   *
             COPY OUTDD=LIB3380,INDD=STDSIZE
```

```
            COPY OUTDD=LIB3380,INDD=STDSIZE
/*
```

The control statements are as follows:

- The REBLOCK EXEC statement begins the reblocking step.
- TESTED DD defines the cataloged load library TESTED.MASTER.LOADLIB.
- STDSIZE DD defines an existing data set, PROGRAM.MASTER.LOADLIB, which resides on a 3390 direct access device and has a block size of 23470.
- The COPYMOD statement in the SYSIN data set specifies that TESTED.MASTER.LOADLIB is to be merged into PROGRAM.MASTER.LOADLIB. It also specifies that PROGRAM.MASTER.LOADLIB is to be reblocked with a maximum block size of 13030. This block size is chosen because it is small enough to fit on both 3380 and 3390 direct access devices.
- The DISTRIB EXEC statement begins the distribution step, where the reblocked data set is copied to devices with different maximum block sizes.
- STDSIZE DD defines the same data set that was reblocked in the previous step.
- LIB3380 DD defines the data set PROGRAM.LIB3380.LOADLIB, which resides on a 3380 direct access device.
- The COPY statements in the SYSIN data set specify that the data set PROGRAM.MASTER.LOADLIB is to be copied to the output data sets without being reblocked. If PROGRAM.MASTER.LOADLIB had not been reblocked to the smaller block size, this step would end unsuccessfully.

Example 13: Convert a Partitioned Data Set to a PDSE

In this example, a partitioned data set is converted to a PDSE.

```
//CONVERT   JOB  ...
//STEP1     EXEC PGM=IEBCOPY
//SYSPRINT  DD  SYSOUT=A
//SYSUT1    DD  DSNAME=PDSSET,DISP=SHR,DSNTYPE=PDS
//SYSUT2    DD  DSNAME=PDSESET,LIKE=PDSSET,DSNTYPE=LIBRARY,
//          DISP=(NEW,CATLG),STORCLAS=SCLASX,DATACLAS=DCLASY
```

The control statements are as follows:

- SYSUT1 DD defines the input data set, PDS, which is a partitioned data set.The DSNTYPE keyword has no effect because it is an existing data set.
- SYSUT2 DD defines the output data set, PDSE, which is a partitioned data set extended. This new data set will be SMS-managed because it has a storage class.

  The LIKE parameter indicates that the DCB and SPACE attributes for PDSESET are to be copied from PDSSET. The DSNTYPE parameter defines the new data set as a PDSE rather than as a partitioned data set. DATACLAS=DCLASY identifies the PPDSE as a program object PDSE with undefined logical record length.

  The Storage Management Subsystem chooses an appropriate volume for the allocation, based on how SCLASX was defined.
- Since the ddnames "SYSUT1" and "SYSUT2" are used to define the input and output data sets, no SYSIN data set is required.

Example 14: Copy Groups from a PDSE to a PDSE

In this example, members and their aliases (groups) are copied from a PDSE to a PDSE (full data set copy). See "Copying Program Objects (COPYGRP and COPYGROUP Statements)" on page 69 for information about copying groups.

```
//CPYGRP    JOB  ...
//STEP1     EXEC PGM=IEBCOPY
//SYSPRINT  DD  SYSOUT=A
//DDIN      DD  DSNAME=PDSESETA,DISP=SHR
//DDOUT     DD  DSNAME=PDSESETB,LIKE=PDSESETA,DSNTYPE=LIBRARY,
//          DISP=(NEW,CATLG)
//SYSUT3    DD  UNIT=SYSDA,SPACE=(TRK,(1,1))
//SYSIN     DD  *
GROUPCPY   COPYGRP  INDD=DDIN,OUTDD=DDOUT
/*
```

The control statements are as follows:

- DDIN DD defines the input data set, PDSESETA, which is a partitioned data set extended.

  DDOUT DD defines the output data set, PDSESETA, which is a partitioned data set extended.

  The LIKE subparameter indicates that the DCB and SPACE attributes for PDSESETB are to be copied from PDSESETA. The DSNTYPE subparameter defines the new data set as a PDSE.

  The Storage Management Subsystem chooses an appropriate volume for the allocation.
- SYSUT3 DD defines a temporary spill data set.
- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPYGRP statement, an INDD statement, and an OUTDD statement.
- COPYGRP indicates the start of the copy operation.

  The INDD parameter shows PDSESETA as the input data set.

  The OUTDD parameter shows PDSESETB as the output data set.

Example 15: Copy Groups from a PDSE to a PDSE with Replace

In this example, members and their aliases are copied in groups from a PDSE to a PDSE with the replace (R) option. See "Replacing Program Objects" on page 70 for information about replacing groups with COPYGRP.

```
//CPYGRP    JOB  ...
//STEP1     EXEC PGM=IEBCOPY
//SYSPRINT  DD  SYSOUT=A
//DDIN      DD  DSNAME=PDSESETA,DISP=SHR
//DDOUT     DD  DSNAME=PDSESETB,LIKE=PDSESETA,DSNTYPE=LIBRARY,
//          DISP=(NEW,CATLG)
//SYSUT3    DD  UNIT=SYSDA,SPACE=(TRK,(1,1))
//SYSIN     DD  *
GROUPCPY   COPYGRP  INDD=((DDIN,R)),OUTDD=DDOUT
/*
```

The control statements are as follows:

- DDIN DD defines the input data set, PDSE, which is a partitioned data set extended.

  DDOUT DD defines the output data set, PDSE, which is a partitioned data set extended.

  The LIKE parameter indicates that the DCB and SPACE attributes for PDSESETB are to be copied from PDSESETA.

  The DSNTYPE parameter defines the new data set as a PDSE.

  The Storage Management Subsystem chooses an appropriate volume for the allocation.
- SYSUT3 DD defines a temporary spill data set.

- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPYGRP statement, an INDD statement, and an OUTDD statement.
- COPYGRP indicates the start of the copy operation.

  The ((INDD,R)) parameter shows PDSESETA as the input data set containing members to replace members with the same name in PDSESETB.

  The OUTDD parameter shows PDSESETB as the output data set.

Example 16: Copy a Selected Group from a PDSE to a PDSE

In this example, a selected member and its aliases are copied from a PDSE to a PDSE. Either the member's name or a maximum of eight characters can be given on the SELECT statement. See “Copying Program Objects (COPYGRP and COPYGROUP Statements)” on page 69 for information about selecting groups on COPYGRP.

```
//CPYGRP    JOB  ...
//STEP1     EXEC PGM=IEBCOPY
//SYSPRINT  DD  SYSOUT=A
//DDIN      DD  DSNAME=PDSESETA,DISP=SHR
//DDOUT     DD  DSNAME=PDSESETB,LIKE=PDSESETA,DSNTYPE=LIBRARY,
//          DISP=(NEW,CATLG)
//SYSUT3    DD  UNIT=SYSDA,SPACE=(TRK,(1,1))
//SYSIN     DD  *
GROUPCPY   COPYGRP  INDD=DDIN,OUTDD=DDOUT
           SELECT   MEMBER=(ALIAS001)
/*
```

The control statements are as follows:

- DDIN DD defines the input data set, PDSE, which is a partitioned data set extended.

  DDOUT DD defines the output data set, PDSE, which is a partitioned data set extended.

  All PDSEs must be managed by the Storage Management Subsystem.

  The LIKE parameter indicates that the DCB and SPACE attributes for PDSESETB are to be copied from PDSESETA.

  The DSNTYPE parameter defines the new data set as a PDSE.

  The Storage Management Subsystem chooses an appropriate volume for the allocation.
- SYSUT3 DD defines a temporary spill data set.
- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPYGRP statement, an INDD statement, and an OUTDD statement.
- COPYGRP indicates the start of the copy operation.

  The INDD parameter shows PDSESETA as the input data set.

  The OUTDD parameter shows PDSESETB as the output data set.
- The SELECT statement indicates that a group that has the alias ALIAS001 is to be selected from the input data set (PDSESETA) and copied to the output data set (PDSESETB).

Example 17: Copy Selected Members and their Aliases from a PDS to a PDS

In this example, selected members and their aliases are copied from a PDS to a PDS, while an EXCLUDE statement is used to prevent other members and their aliases from being copied. The SELECT and EXCLUDE statements can be used only if the operation is a COPYGROUP and member name filter pattern masking is used for both. Refer to the descriptions of the SELECT and EXCLUDE statements for details in using member name filter pattern masking.

```
    //CPYGROUP JOB ...
  //STEP1    EXEC PGM=IEBCOPY
  //SYSPRINT DD SYSOUT=*
  //DDIN     DD DISP=SHR,DSN=PDSA
  //DDOUT    DD DISP=(NEW,CATLG),DSN=PDSB,LIKE=PDSA,DSNTYPE=PDS
  //SYSIN    DD *
  CPYLBL1   COPYGROUP INDD=DDIN,OUTDD=DDOUT
            SELECT MEMBER=(ABCDEFGH,BCD*,%D*,GHIJKLMN)
            EXCLUDE MEMBER=(DEF*,EFGHIJKL,%GHI*)
/*
```

The control statements are as follows:

- DDIN DD defines the input data set, PDSA, a partitioned data set with the following members:
  1. ABCDEFGH
  2. BCDEFGHI
  3. CDEFGHIJ
  4. DEFGHIJK
  5. EFGHIJKL
  6. FGHIJKLM
  7. GHIJKLMN
- DDOUT DD defines the output data set, PDSB, a partitioned data set.
- The LIKE parameter indicates that the DCB and SPACE attributes for PDSB are to be copied from PDSA.
- The DSNTYPE parameter defines the data set as a PDS.
- The Storage Management Subsystem chooses an appropriate volume for the newly allocated data set.
- SYSIN DD defines the control data set, which follows in the input stream. The data set contains a COPYGROUP statement, a SELECT statement and an EXCLUDE statement.
- COPYGROUP indicates the start of a group copy operation.
- The INDD parameter shows PDSA as the input data set.
- The OUTDD parameter shows PDSB as the output data set.
- The SELECT statement is using filter pattern masking and indicates that any member matching the masking pattern and all of its aliases will be selected from the input dataset and copied to the output dataset. In this example the members ABCDEFGH, BCDEFGHI, CDEFGHIJ, and GHIJKLMN and their aliases will be copied.
- The EXCLUDE statement can be used for filter pattern masking for a COPYGROUP operation only when a filter masking SELECT statement is present. This will exclude the matched members and all of their aliases from being copied to the output dataset. In this example the members DEFGHIJK, EFGHIJKL, and FGHIJKLM and their aliases will not be copied.

Example 18: Copy Selected Members from a PDS to a PDS

In this example, selected members are copied from a PDS to a PDS. This example differs from Example 16 in that COPYGROUP allows members to be copied along with their aliases from a PDS to a PDS, whereas COPYGRP only copies aliases when one or both of the input and output data sets is a PDSE. If both are PDSs it will result in a COPY operation. See “Copying Program Objects (COPYGRP and COPYGROUP Statements)” on page 69 for information about selecting groups with COPYGROUP and the differences between the COPYGROUP and COPYGRP statements.

```
   //CPYGROUP JOB ...
  //STEP1    EXEC PGM=IEBCOPY
  //SYSPRINT DD SYSOUT=*
  //DDIN     DD DISP=SHR,DSN=PDSA,DSNTYPE=PDS
  //DDOUT    DD DSN=PDSB,LIKE=PDSA,DSNTYPE=PDS,DISP=(NEW,CATLG)
  //SYSIN    DD *
  CPYLBL2   COPYGROUP INDD=DDIN,OUTDD=DDOUT
            SELECT MEMBER=(MEMBER02)
```

Chapter 7. IEBGENER (Sequential Copy/Generate Data Set) Program

You can use IEBGENER to perform these tasks:

- Create a backup copy of a sequential data set, a member of a partitioned data set or PDSE or a z/OS UNIX System Services (z/OS UNIX) file such as a HFS file.
- Produce a partitioned data set or PDSE, or a member of a partitioned data set or PDSE, from a sequential data set or a z/OS UNIX file.
- Expand an existing partitioned data set or PDSE by creating partitioned members and merging them into the existing data set.
- Produce an edited sequential or partitioned data set or PDSE.
- Manipulate data sets containing double-byte character set data.
- Print sequential data sets, members of partitioned data sets or PDSEs or z/OS UNIX files.
- Reblock or change the logical record length of a data set.
- Copy user labels on sequential output data sets.
- Supply editing facilities and exits for your routines that process labels, manipulate input data, create keys, and handle permanent input/output errors.

Recommendation: If you have the DFSORT product installed, you should be using ICEGENER as an alternative to IEBGENER when making an unedited copy of a data set or member. It may already be installed in your system under the name IEBGENER. It generally gives better performance.

Related reading :

- For information about the linkage conventions for user exit routines, see Appendix C, “Specifying User Exits with Utility Programs,” on page 365.
- For information about user labels on sequential output data sets, see “Processing User Labels” on page 369.

Creating a Backup Copy

You can produce a backup copy of a sequential data set or member of a partitioned data set or PDSE by copying the data set or member to any IBM-supported output device. For example, a copy can be made from tape to tape, from DASD to tape, and so forth.

A data set that resides on a direct access volume can be copied to its own volume, provided that you change the name of the data set.

Restriction:

When you use IEBGENER to process partitioned data sets as sequential data sets, IEBGENER will not perform any directory entry processing (for instance, copying attributes of members of load module libraries). IEBCOPY does perform directory entry processing. IEBGENER does not support copying data sets that have been PACKED using the ISPF (PACK ON) command.

Producing a Partitioned Data Set or PDSE from Sequential Input

Through the use of utility control statements, you can logically divide a sequential data set into *record groups* and assign member names to the record groups. IEBGENER places the newly created members in an output partitioned data set or PDSE.

© Copyright IBM Corp. 1979, 2026

You cannot produce a partitioned data set or PDSE if an input or output data set contains spanned records.

Figure 23 on page 150 shows how a partitioned data set or PDSE is produced from a sequential data set used as input. One side of the figure shows the sequential data set. Utility control statements are used to divide the sequential data set into record groups and to provide a member name for each record group. The other side of the figure shows the partitioned data set or PDSE produced from the sequential input.

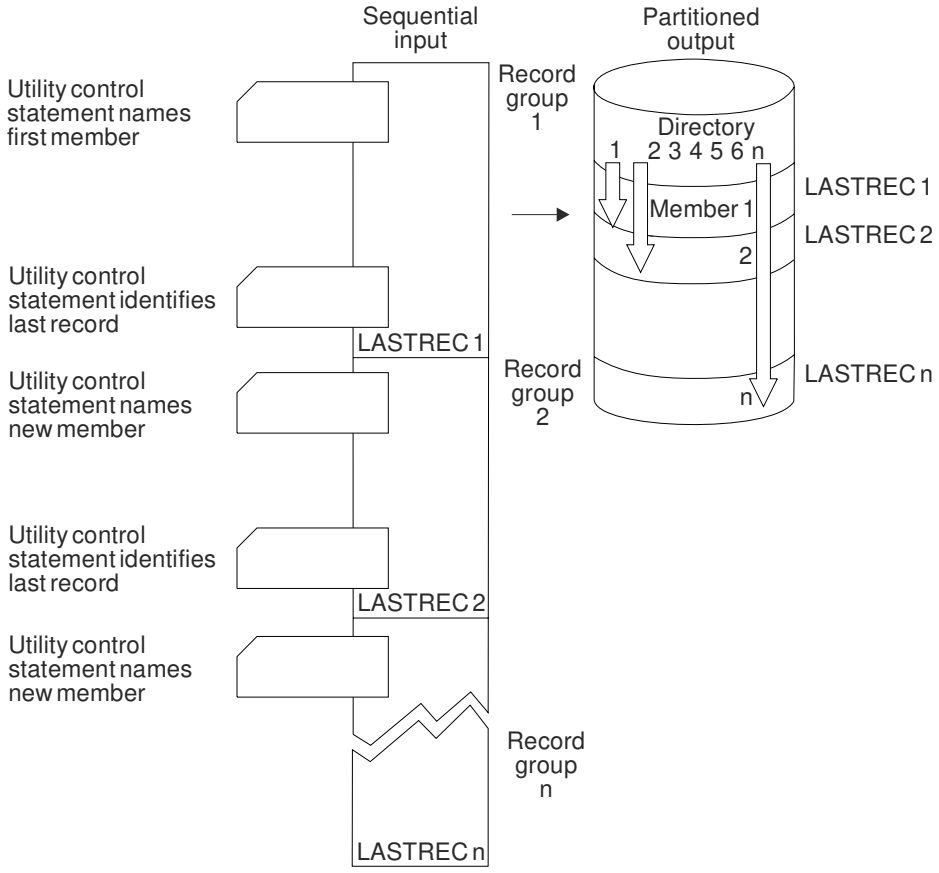


Figure 23. Using IEBGENER to create a partitioned data set or PDSE from sequential input

Adding Members to a Partitioned Data Set or PDSE

You can use IEBGENER to add members to a partitioned data set or PDSE. IEBGENER creates the members from sequential input and adds them to the data set. The merge operation—the ordering of the partitioned directory—is automatically performed by the program.

Figure 24 on page 151 shows how sequential input is converted into members that are merged into an existing partitioned data set or PDSE. The figure shows the sequential input that is to be merged with the existing partitioned data set or PDSE. Utility control statements are used to divide the sequential data set into record groups and to provide a member name for each record group. The figure also shows the expanded partitioned data set or PDSE. Members B, D, and F from the sequential data set were placed in *available space* and are sequentially ordered in the partitioned directory.

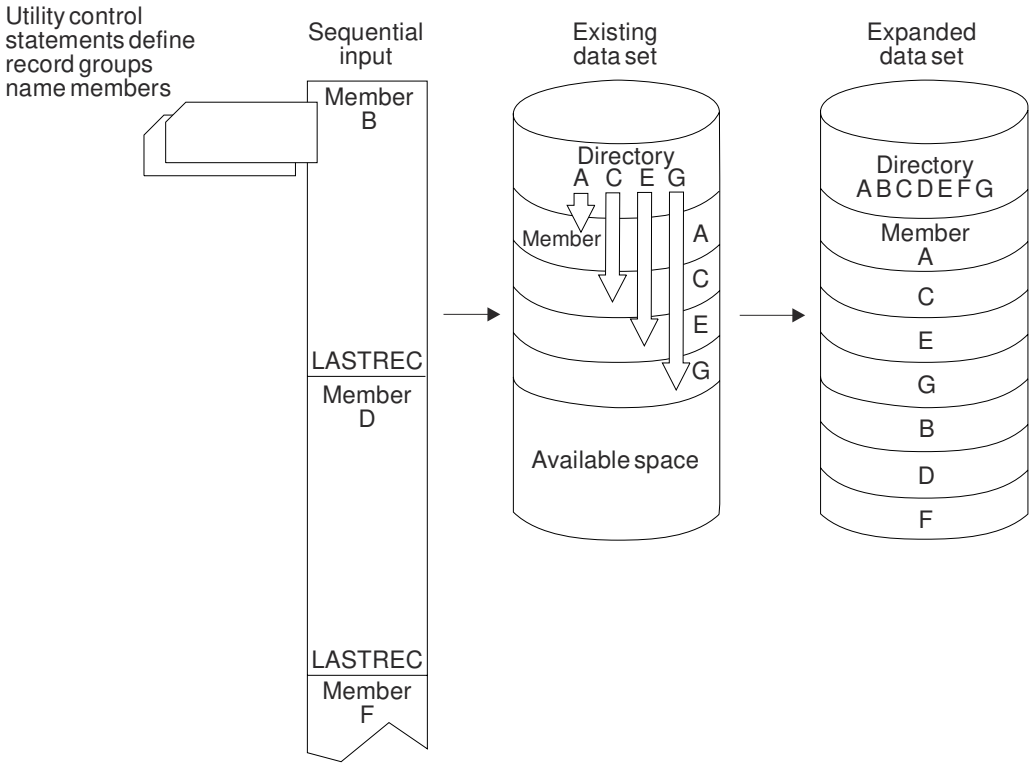


Figure 24. Using IEBGENER to add members to a partitioned data set or PDSE

Producing an Edited Data Set

You can use IEBGENER to produce an edited sequential or partitioned data set, or PDSE. Through the use of utility control statements, you can specify editing information that applies to a record, a group of records, selected groups of records, or an entire data set.

An edited data set can be produced by:

- Rearranging or omitting defined data fields within a record.
- Supplying literal information as replacement data.
- Converting data from packed decimal to unpacked decimal mode, unpacked decimal to packed decimal mode, or BCDIC (used here to mean the standard H character set of binary coded decimal interchange code) to EBCDIC mode.
- Adding or deleting shift-out/shift-in characters X'0E' and X'0F' when double-byte character set data is contained in the data set.

Figure 25 on page 152 shows part of an edited sequential data set. The figure first shows the data set before editing is performed. Utility control statements are used to identify the record groups to be edited and to supply editing information. In this figure, literal replacement information is supplied for information within a defined field. (Data is rearranged, omitted, or converted in the same manner.) The BBBB field in each record in the record group is to be replaced by CCCC. The figure then shows the data set after editing.

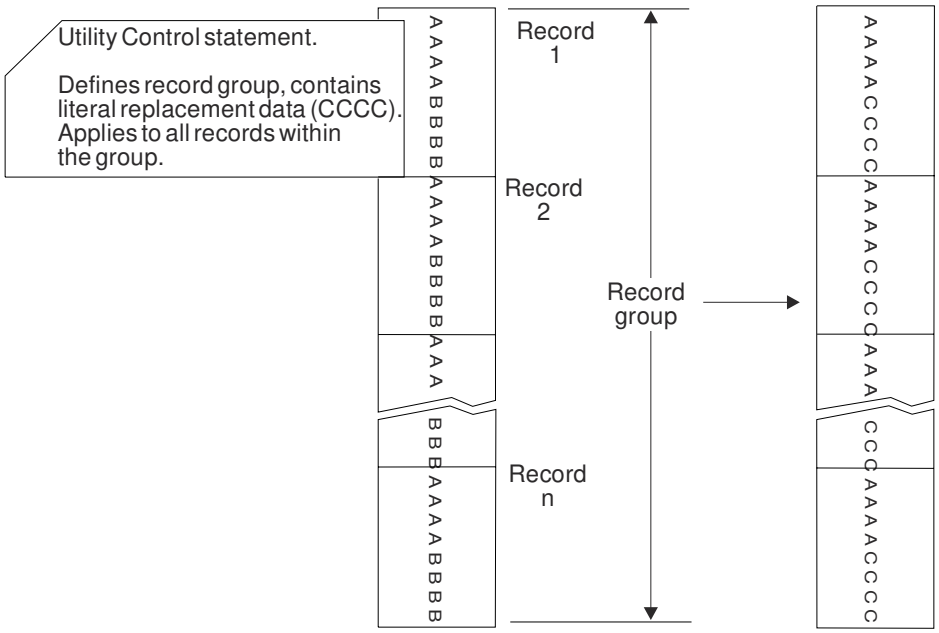


Figure 25. Editing a sequential data set using IEBGENER

IEBGENER cannot be used to edit a data set if the input and output data sets consist of variable spanned (VS) or variable blocked spanned (VBS) records and have equal block sizes and logical record lengths. In these cases, any utility control statements that specify editing are ignored. That is, for each physical record read from the input data set, the utility writes an unedited physical record on the output data set.

Related reading: For more information on converting from BCDIC to EBCDIC, see *z/OS DFSMS Using Data Sets*.

Changing Logical Record Length

You can use IEBGENER to produce a reblocked output data set containing either fixed-length or variable-length records having a logical record length that differs from the input logical record length.

When you create a data set with a logical record length that differs from that of the input data set, you must specify where each byte in the output records is to come (either from literals or from input records). Any unspecified fields will contain unpredictable data. You cannot alter the logical record length if both the input and output data sets have variable or variable blocked record formats and the output logical record length is smaller than the input logical record length. In this case, IEBGENER fails with return code 12 and issues message IEB311I.

Using IEBGENER with Double-Byte Character Set Data

You can use IEBGENER to copy, edit, reblock, or print data sets that contain double-byte character set (DBCS) data. You can also convert sequential data sets containing DBCS data to partitioned data sets. Double-byte character sets are used to represent languages too complex to be represented with a single-byte character set. Japanese, for example, requires a double-byte character set.

DBCS data is typically enclosed in shift-out/shift-in X'0E' and X'0F' single-byte characters to indicate that the data must be handled as pairs of bytes and not single bytes. *Shift-out* indicates that the data is now shifting out of a single-byte character set string and *into* a double-byte character set string, and *shift-in* indicates that a double-byte character set string is shifting into a single-byte character set string. You can add or delete the shift-out/shift-in characters using IEBGENER. If you add them to a data set, however, you must account for the additional bytes they will require when you determine the logical record length of the output data set. You can also validate DBCS data using IEBGENER. DBCS data is not considered "valid" unless each byte of every character has a value between X'41' and X'FE', inclusive, or unless

the DBCS character is a DBCS space (X'4040'). This checking will ensure that each DBCS character is printable.

In order to print a data set containing DBCS data, the DBCS strings must be enclosed in shift-out/shift-in characters.

Input and Output

IEBGENER uses the following input:

- An input data set, which contains the data that is to be copied, edited, converted into a partitioned data set or PDSE, or converted into members to be merged into an existing data set. The input is either a sequential data set or a member of a partitioned data set or PDSE.
- A control data set, which contains utility control statements. The control data set is required if editing is to be performed or if the output data set is to be a partitioned data set or PDSE.
- The PARM parameter on the JCL EXEC statement or the dynamic invocation equivalent.

IEBGENER produces the following output:

- An output data set, which can be either sequential, including large format sequential, or partitioned. The output data set can be either a new data set (created during the current job step) or an existing data set. It can be a sequential data set or a PDS or PDSE that will be expanded. If a partitioned data set or PDSE is created, it is a new member with a new directory entry. None of the information is copied from the previous directory entry. The same sequential data set cannot be used as both the input and output data set. The output sequential data set cannot be any of the input data sets in a sequential concatenation.
- A message data set, which contains informational messages (for example, the contents of utility control statements) and any error messages.

Message IEC507D will be issued twice when adding data or members to an existing data set which has an unexpired expiration date. This occurs because the input and output data sets are opened twice.

If IEBGENER is invoked from an application program, you can dynamically allocate the data sets by issuing SVC 99 before calling IEBGENER.

Related reading :

- For information about the messages issued by the IEBGENER utility, see *z/OS MVS System Messages, Vol 7 (IEB-IEE)*.
- For information about IEBGENER return codes, see Appendix A, “Invoking Utility Programs from an Application Program,” on page 337.
- For information about the JCL EXEC statement, see “EXEC Statement” on page 154.

Control

You control IEBGENER with job and utility control statements. The job control statements run IEBGENER and define the data sets that are used and produced by the program. The utility control statements control the functions of IEBGENER.

Job Control Statements

Table 29 on page 153 shows the job control statements for IEBGENER.

Table 29. Job control statements for IEBGENER

Statement	Use
JOB	Starts the job.
EXEC	Specifies the program name (PGM=IEBGENER) or, if the job control statements reside in a procedure library, the procedure name.

Table 29. Job control statements for IEBGENER (continued)

Statement	Use
SYSPRINT DD	Defines a sequential data set for messages. The data set can be written to a system output device, a tape volume, or a DASD volume.
SYSUT1 DD	Defines the input data set. It can define a sequential data set, a member of a partitioned data set or PDSE or a z/OS UNIX file. A sequential data set can be basic format, large format, extended format, compressed format, spooled input, tape, card reader, TSO terminal or dummy.
SYSUT2 DD	Defines the output data set. It can define a sequential data set, a member of a partitioned data set or PDSE, a partitioned data set or PDSE or a z/OS UNIX file. A sequential data set can be basic format, large format, extended format, compressed format, spooled output, tape, card punch, printer, TSO terminal or dummy.
SYSIN DD	Defines the control data set, or specifies DUMMY when the output is sequential and no editing is specified. The control data set normally resides in the input stream; however, it can be defined as a member in a partitioned data set or PDSE.

EXEC Statement

The EXEC statement is required for each use of IEBGENER.

You can code a value like: PARM='SDB=*xxxxx*' on the EXEC statement. This value is effective only if the output block size is not supplied by any other source. The system calculates an optimal value through a function called “system-determined block size”. This PARM value controls whether the block size can exceed 32 760 bytes. SDB is the only valid parameter for the EXEC statement. You can code one of these values for *xxxxx*.

INPUT or YES
If the input block size is greater than 32 760 bytes, the default output block size greater than 32 760 bytes is permitted. Otherwise, it is not permitted.

SMALL
The default output block size is 32 760 bytes or less.

LARGE
The default output block size can be greater than 32 760 bytes. Older programs cannot read data sets that have such large blocks.

NO
The default output block size is 32 760 bytes or less, but the block size might be less efficient than if you coded SMALL.

Your system programmer sets the default value for SDB by setting the COPYSDB in the DEVSUP*xx* member in SYS1.PARMLIB. The IBM-supplied default for COPYSDB is INPUT. Prior to OS/390 Version 2 Release 10, IEBGENER ignored a PARM value and operated as if SDB=NO had been coded. Currently, only magnetic tape devices and dummy data sets allow a block size that exceeds 32 760 bytes. Do not depend on that restriction. Future levels of operating systems might allow larger blocks in more kinds of data sets.

The following conditions can cause your output data set to have a block size that exceeds 32 760 bytes:

- You coded the BLKSIZE keyword on the DD statement and the device will support the value.
- You coded DISP=MOD on the SYSUT2 DD statement if the output data set exists and has a large BLKSIZE.
- The output device type supports a large block size, but you did not code one, and SDB=LARGE is active.
- The output device type supports a large block size, but you did not code one. The input data set has a large block size and SDB=INPUT is active.

Note: In the last two cases, the system limits the block size to a block size limit. IEBGENER will never calculate a block size value that is too large for the device. With fixed-blocked records, the block size limit does not have to be a multiple of LRECL. The block size limit is the first one of these that is available:

- BLKSZLIM keyword on the output DD statement.
- Block size limit in the data class, even if the data set is not SMS-managed. You can code the DATACLAS keyword on the DD statement, or the system's ACS routines can assign a data class.
- TAPEBLKSZLM keyword in the DEVSUP*xx* member of SYS1.PARMLIB. The system programmer sets this as a system default.
- $32^{5}\!/_{6}$760 bytes.

Using multiple buffers for IEBGENER increases the amount of virtual storage that is needed to run the program. You may need to change, or add, the REGION parameter for the additional storage to avoid 80A abends.

The default for the number of buffers is five. You can override this by specifying BUFNO or NCP on the SYSUT1 or SYSUT2 DD statement.

Before you run IEBGENER, you may need to calculate the region size (in virtual storage) that is needed to run the program. Specify this value in the REGION parameter.

You can calculate the region size by using the formula, as follows:

```
region size = 50K + (2 + SYSUT1 BUFNO)*(SYSUT1 BLKSIZE) +
                    (2 + SYSUT2 BUFNO)*(SYSUT2 BLKSIZE)
```

If you do not use BUFNO in your JCL, use the default value of 5.

Related reading: For information on how to calculate the region size, see "Example 5: Produce Blocked Copy on Tape from Unblocked Disk File" on page 166.

SYSPRINT DD Statement

The SYSPRINT DD statement is required for each use of IEBGENER. The block size for the SYSPRINT data set must be a multiple of 121. Any blocking factor can be specified for this record size, but the maximum value for the block size is 32670 bytes.

SYSUT1 DD Statement

The input data set for IEBGENER, as specified in SYSUT1, can contain fixed, variable, undefined, or variable spanned records. You cannot use concatenated data sets with *unlike* attributes except for block size or device type as input to IEBGENER. The rules for concatenating data sets on SYSUT1 are the same as those for BSAM unless you are concatenating data sets on DASD and tapes. Then the concatenating rules are the same as for QSAM.

If the SYSUT1 data set is in the system input stream and contains JCL statements, code SYSUT1 DD DATA and not SYSUT1 DD *.

Block size for the input data set must be available in the data set label (DSCB), or tape label, or the DD statement. For a tape data set the block size can exceed $32^{5}\!/_{6}$760 bytes.

The default record format is undefined (U) for the input data set. Record format must be specified if the data set is new, undefined, or a dummy data set.

The input logical record length must be specified when the record format is fixed blocked, variable spanned, or variable blocked spanned, or when the data set is new, or a dummy data set. If the input and output logical record length are not available and the record format is V or VB, IEBGENER sets the maximum record length to the block size -4, with a maximum of $32^{5}\!/_{6}$760. In all other cases, a default logical record length of 80 is used.

A partitioned data set or PDSE cannot be produced if an input data set contains spanned records.

If both the SYSUT1 and the SYSUT2 DD statements specify standard user labels (SUL), IEBGENER copies user labels from SYSUT1 to SYSUT2.

Related reading:

- For information on concatenated data sets for BSAM and QSAM, see *z/OS DFSMS Using Data Sets*.
- For information about available options for user label processing, see "Processing User Labels" on page 369.

SYSUT2 DD Statement

The output data set for IEBGENER, as specified in SYSUT2, can contain fixed, variable, undefined, or variable spanned records (except partitioned data sets or PDSEs, which cannot contain variable spanned records). IEBGENER can reblock records if you specify a new maximum block length on the SYSUT2 DD statement. If you are reblocking fixed-length or variable-length records, keys can be retained only if you supply an exit routine to retain them. You cannot retain keys when you reblock variable spanned records.

If the output data set is on a card punch or a printer (not a spooled data set), you must specify DCB information on the SYSUT2 DD statement.

When the data set is on DASD or tape and record format and logical record length are not specified in the JCL for the output data set, and the data class does not supply them, values for each are copied from the input data set. Logical record length is not copied if the RECFM is undefined.

Note: If you are using IEBGENER with an SMF dump data set, IEBGENER can truncate your data. The SMF dump program, IFASMFDP, creates a data set with a logical record length larger than IEBGENER allows. Thus, IEBGENER will truncate the data set to a logical record length of 32 760, resulting in a loss of 7 bytes. If you do not have actual data in those 7 bytes, you will lose nothing in the truncation. However, care should be taken in using IEBGENER with the SMF dump program.

The output block size need not be specified, if logical record length and record format are specified or available for the input data set. The EXEC statement input data set and output device type affect the output block size (when not specified). If the output device type is tape or dummy, the block size can exceed 32 760 bytes. If you specify BLKSIZE, then SDB in the PARM value on the EXEC statement will have no effect.

The output logical record length must be specified when editing is to be performed and the record format is fixed blocked, variable spanned or variable blocked spanned.

A partitioned data set or PDSE cannot be produced if an input or output data set contains spanned records.

Table 30 on page 156 shows the effect of the availability of RECFM, LRECL, and BLKSIZE in the SYSUT2 DD statement.

Table 30. Effect of output DD statements. "Available" means that you coded it or that the value is in the data set label or data class.

RECFM	LRECL	BLKSIZE	Result
Available	Available	Available	All are used.
Available	Available	Omitted	Block size is determined by the system.
Available	Omitted	Available	LRECL is copied from the input unless RECFM is undefined. The specified RECFM and BLKSIZE are used.
Available	Omitted	Omitted	LRECL is copied from the input unless RECFM is undefined. The BLKSIZE is copied from the input unless it is too large for the device; in that case, the system determines an optimal BLKSIZE.
Omitted	Available	Available	RECFM is copied from the input. The specified LRECL and BLKSIZE are used.

Table 30. Effect of output DD statements. "Available" means that you coded it or that the value is in the data set label or data class. *(continued)*			
RECFM	**LRECL**	**BLKSIZE**	**Result**
Omitted	Available	Omitted	RECFM is copied from the input. The specified LRECL is used. BLKSIZE is determined by system.
Omitted	Omitted	Available	RECFM and LRECL are copied from the input, but LRECL is not copied if RECFM is undefined.
Omitted	Omitted	Omitted	All three are copied unless BLKSIZE is too large for the device. In that case, the system determines an optimal BLKSIZE.

Related reading: For a description of SDB, see “EXEC Statement” on page 154.

When using IEBGENER, it is possible to override the existing RECFM, LRECL, and BLKSIZE attributes. This can occur when you are adding data to a sequential data set, or when adding or replacing a member to a PDS or PDSE. If the data set is a PDS or PDSE, any changes to these fields will affect all members in the data set. The new values will permanently override the existing values that stored in the DSCB until another override occurs.

If you override the attributes when adding to a non-empty data set, you probably have damaged the data set. Here are examples where you have not damaged the data set:

- The record format is variable and you set a larger record length that is not too large for the block size.
- The record format is undefined and you set a larger block size.
- The record format is fixed unblocked and you changed it to fixed blocked (FB) with a BLKSIZE value that is a multiple of LRECL.

In all of these cases, you have set attributes that are consistent with the previous records.

SYSIN DD Statement

The SYSIN DD statement is required for each use of IEBGENER. The block size for the SYSIN data set must be a multiple of 80. Any blocking factor can be specified for this block size. If SYSIN is dummy, IEBGENER copies the input data set sequentially.

Utility Control Statements

IEBGENER is controlled by utility control statements. The statements and the order in which they must appear are listed in Table 31 on page 158.

The control statements are included in the control data set as required. If no utility control statements are included in the control data set, the entire input data set is copied sequentially.

When the output is to be sequential and editing is to be performed, one GENERATE statement and as many RECORD statements as required are used. If you are providing exit routines, an EXITS statement is required.

When the output is to be partitioned, one GENERATE statement, one MEMBER statement per output member, and RECORD statements, as required, are used. If you are providing exit routines, an EXITS statement is required.

A continuation line must start in columns 4 to 16, and a nonblank continuation line in column 72 is optional.

Table 31. IEBGENER utility control statements

Statement	Use
GENERATE	Indicates the number of member names and alias names, record identifiers, literals, and editing information contained in the control data set.
EXITS	Indicates that user routines are provided.
LABELS	Specifies user-label processing.
MEMBER	Specifies the member name and alias of a member of a partitioned data set or PDSE to be created.
RECORD	Defines a record group to be processed and supplies editing information.

GENERATE Statement

The GENERATE statement is required when: output is to be partitioned; editing is to be performed; or user routines are provided or label processing is specified. The GENERATE statement must appear before any other IEBGENER utility statements. If it contains errors or is inconsistent with other statements, IEBGENER is ended.

The syntax of the GENERATE statement is:

Label	Statement	Parameters
[*label*]	GENERATE	[,MAXNAME=*n*] [,MAXFLDS=*n*] [,MAXGPS=*n*] [,MAXLITS=*n*] [,DBCS={YES|NO}]

where:

MAXNAME=*n*
specifies a number, from 1 to 3276, that is greater than or equal to the total number of member names and aliases appearing in subsequent MEMBER statements. MAXNAME is required if there are one or more MEMBER statements.

MAXFLDS=*n*
specifies a number, from 1 to 4095, that is greater than or equal to the total number of FIELD parameters appearing in subsequent RECORD statements. MAXFLDS is required if there are any FIELD parameters in subsequent RECORD statements.

MAXGPS=*n*
specifies a number, from 1 to 2520, that is greater than or equal to the total number of IDENT or IDENTG parameters appearing in subsequent RECORD statements. MAXGPS is required if there are any IDENT or IDENTG parameters in subsequent RECORD statements.

MAXLITS=*n*
specifies a number, from 1 to 2730, that is greater than or equal to the total number of characters contained in the FIELD literals of subsequent RECORD statements. Any DBCS characters used as literals on FIELD parameters count as two characters each.

MAXLITS is required if the FIELD parameters of subsequent RECORD statements contain literals. MAXLITS does not apply to literals used in IDENT or IDENTG parameters.

DBCS={YES|NO}
specifies if the input data set contains double-byte character set data.

EXITS Statement

The EXITS statement is used to identify exit routines you want IEBGENER to use. The syntax of the EXITS statement is:

Label	Statement	Parameters
[*label*]	EXITS	[INHDR=*routinename*] [,OUTHDR=*routinename*] [,INTLR=*routinename*] [,OUTTLR=*routinename*] [,KEY=*routinename*] [,DATA=*routinename*] [,IOERROR=*routinename*] [,TOTAL=(*routinename*,*size*)]

where:

INHDR=*routinename*
specifies the name of the routine that processes user input header labels.

OUTHDR=*routinename*
specifies the name of the routine that creates user output header labels. OUTHDR is ignored if the output data set is partitioned.

INTLR=*routinename*
specifies the name of the routine that processes user input trailer labels.

OUTTLR=*routinename*
specifies the name of the routine that processes user output trailer labels. OUTTLR is ignored if the output data set is partitioned.

KEY=*routinename*
specifies the name of the routine that creates the output record key. This routine does not receive control when a data set consisting of variable spanned (VS) or variable blocked spanned (VBS) type records is processed because no processing of keys is permitted for this type of data.

DATA=*routinename*
specifies the name of the routine that modifies the physical record (logical record for variable blocked type records) before it is processed by IEBGENER.

IOERROR=*routinename*
specifies the name of the routine that handles permanent input/output error conditions.

TOTAL=(*routinename*,*size*)
specifies that a user exit routine is to be provided before writing each record. The keyword OPTCD=T must be specified for the SYSUT2 DD statement. TOTAL is valid only when IEBGENER is used to process sequential data sets. These values must be coded:

routinename
specifies the name of your totaling routine.

size
specifies the number of bytes needed to contain totals, counters, pointers, and so forth. *Size* should be coded as a whole decimal number.

Related reading :

- For information about exit routines, see Appendix C, “Specifying User Exits with Utility Programs,” on page 365.
- For information about the processing of user labels as data set descriptors and user label totaling, see “Processing User Labels” on page 369.

LABELS Statement

The LABELS statement specifies if user labels are to be treated as data by IEBGENER. For a detailed discussion of this option, refer to “Processing User Labels” on page 369.

The LABELS statement is used when you want to specify that: no user labels are to be copied to the output data set; user labels are to be copied to the output data set from records in the data portion of the SYSIN data set; or user labels are to be copied to the output data set after they are modified by the user's label processing routines. If more than one valid LABELS statement is included, all but the last LABELS statement are ignored.

The syntax of the LABELS statement is:

Label	Statement	Parameters
[*label*]	LABELS	[DATA={YES|NO|ALL| ONLY|INPUT}]

where:

DATA={YES|NO|ALL|ONLY|INPUT}
specifies if user labels are to be treated as data by IEBGENER. These values can be coded:

YES
specifies that any user labels that are not rejected by a label processing routine you have specified on the EXITS statement are to be treated as data. Processing of labels as data ends in compliance with standard return codes. YES is the default.

NO
specifies that user labels are not to be treated as data. In order to make standard user label (SUL) exits inactive, NO must be specified when processing input/output data sets with nonstandard labels (NSL).

ALL
specifies that all user labels in the group currently being processed are to be treated as data. A return code of 16 causes IEBGENER to complete processing the remainder of the group of user labels and to stop the job step.

ONLY
specifies that only user header labels are to be treated as data. User header labels are processed as data regardless of any return code. The job ends upon return from the OPEN routine.

INPUT
specifies that user labels for the output data set are supplied as 80-byte input records in the data portion of SYSIN. The number of input records that should be treated as user labels must be identified by a RECORD statement.

LABELS DATA=NO must be specified to make standard user labels (SUL) exits inactive when input/output data sets with nonstandard labels (NSL) are to be processed.

MEMBER Statement

The MEMBER statement is used when the output data set is to be partitioned. One MEMBER statement must be included for each member to be created by IEBGENER. The MEMBER statement provides the name and alias names of a new member.

All RECORD statements following a MEMBER statement refer to the one named in that MEMBER statement. If no MEMBER statements are included, the output data set is organized sequentially.

The syntax of the MEMBER statement is:

Label	Statement	Parameters
[*label*]	MEMBER	NAME=(*name*[,*alias 1*][,*alias 2*] [,...])

where:

NAME=(*name*[,*alias*][,...])
specifies a member name followed by a list of its aliases. If no aliases are specified in the statement, the member name need not be enclosed in parentheses.

RECORD Statement

The RECORD statement is used to define a record group and to supply editing information. A record group consists of records that are to be processed identically.

The RECORD statement is used when: the output is to be partitioned; editing is to be performed; or user labels for the output data set are to be created from records in the data portion of the SYSIN data set. The RECORD statement defines a record group by identifying the last record of the group with a literal name.

If no RECORD statement is used, the entire input data set or member is processed without editing. More than one RECORD statement might appear in the control statement stream for IEBGENER.

Within a RECORD statement, one IDENT or IDENTG parameter can be used to define the record group; one or more FIELD parameters can be used to supply the editing information applicable to the record group; and one LABELS parameter can be used to indicate that this statement is followed immediately by output label records.

If both output header labels and output trailer labels are to be contained in the SYSIN data set, you must include one RECORD statement (including the LABELS parameter), indicating the number of input records to be treated as user header labels and another RECORD statement (also including the LABELS parameter) for user trailer labels. The first such RECORD statement indicates the number of user header labels; the second indicates the number of user trailer labels. If only output trailer labels are included in the SYSIN data set, a RECORD statement must be included to indicate that there are no output header labels in the SYSIN data set (LABELS=0). This statement must precede the RECORD LABELS=n statement which signals the start of trailer label input records.

For a further discussion of the LABELS option, refer to "Processing User Labels" on page 369.

The syntax of the RECORD statement is:

Label	Statement	Parameters
[*label*]	RECORD	[{IDENT|IDENTG}=(*length*, '*name*',*input-location*)] [,FIELD=([*length*], [{*input-location*| '*literal*'}], [*conversion*], [*output-location*])] [,FIELD=...] [,LABELS=*n*]

where:

{IDENT|IDENTG}= (*length*,'*name*',*input-location*)
identifies the last record of a collection of records in the input data set. You use this parameter to identify the last record to be edited according to the FIELD parameters on the same RECORD statement. If you are creating a partitioned data set or PDSE, this parameter identifies the last record to be included in the partitioned data set or PDSE member named in the previous MEMBER statement. If the RECORD statement is not followed by additional RECORD or MEMBER statements, IDENT or IDENTG also defines the last record to be processed.

IDENT is used to identify a standard, single-byte character string. IDENTG is used to identify a double-byte character string.

The values for IDENT or IDENTG can be coded:

length
specifies the length (in bytes) of the identifying name. The length of your identifier cannot be greater than eight.

For IDENTG, the length must be an even number.

'*name*'
specifies the literal that identifies the last input record of a group of records. '*Name*' must be coded within single apostrophes.

If you are using IDENTG, '*name*' must be a double-byte character string. The DBCS string must be enclosed in shift-out/shift-in (SO/SI) characters. The SO/SI characters are not considered part of the literal specified by '*name*', and they should not be included in the count for *length*. IEBGENER disregards the SO/SI characters when it looks for a match for '*name*'.

'*Name*' can be specified in hexadecimal. To do so, code '*name*' as *name* . Thus, if you do not have a keyboard that can produce certain characters, you can specify them in their hexadecimal versions. The values of the SO/SI characters are X'0E' and X'0F', respectively.

If no match for '*name*' is found, the remainder of the input data is considered to be in one record group; subsequent RECORD and MEMBER statements are ignored.

input-location
specifies the starting position of the field that contains the identifying name in the input records. *Input-location* should be coded as a whole decimal number.

If you do not specify IDENT or IDENTG, all of the input data is considered to be in one record group. Only the first RECORD and MEMBER statements are used by IEBGENER.

FIELD=([*length*], [{*input-location*| '*literal*'}], [*conversion*],[*output-location*])
specifies field-processing and editing information. Only the contents of specified fields in the input record are copied to the output record; that is, any field in the output record that is not specified contains meaningless data.

Note that the variables on the FIELD parameter are positional; if any of the options are not coded, the associated comma preceding that variable must be coded.

The values that can be coded are:

length
specifies the length (in bytes) of the input field or literal to be processed. If *length* is not specified, a length of 80 is assumed. If a literal is to be processed, a length of 40 or less must be specified. If the data set is variable blocked, the length must be 4 bytes less than the LRECL of the output data set.

input-location
specifies the starting position of the field to be processed. *Input-location* should be coded as a whole decimal number. If *input-location* is not specified, it defaults to 1.

'*literal*'
specifies a literal (maximum length of 40 bytes) to be placed in the specified output location. If a literal contains apostrophes, each apostrophe must be written as two consecutive apostrophes.

You can specify a literal in hexadecimal by coding X'literal' You can also specify a double-byte character set string as the literal.

conversion
specifies a code that indicates the type of conversion to be performed on this field. If no conversion is specified, the field is moved to the output area without change. The values that can be coded are:

CG
specifies that shift-out/shift-in characters are to be removed, but that DBCS data is not to be validated. DBCS=YES must be specified on the GENERATE statement.

CV
specifies that DBCS data is to be validated, and that the input records contain single-byte character set data and double-byte. DBCS=YES must be specified on the GENERATE statement.

GC
specifies that shift-out/shift-in characters are to be inserted to enclose the DBCS data. DBCS=YES must be specified on the GENERATE statement.

GV
specifies that DBCS data is to be validated, and that the DBCS data is not enclosed by shift-out/shift-in characters. DBCS=YES must be specified on the GENERATE statement.

HE
specifies that H-set BCDIC data is to be converted to EBCDIC.

PZ
specifies that packed decimal data is to be converted to unpacked decimal data. Unpacking of the low-order digit and sign might result in an alphabetic character. This maximum length of an input packed decimal field is 16380 bytes.

VC
specifies that DBCS data is to be validated, and that shift-out/shift-in characters are to be inserted to enclose the DBCS data. DBCS=YES must be specified on the GENERATE statement.

VG
specifies that DBCS data is to be validated, and that shift-out/shift-in characters are to be eliminated from the records. DBCS=YES must be specified on the GENERATE statement.

ZP
specifies that unpacked decimal data is to be converted to packed decimal data.

When the ZP parameter is specified, the conversion is performed in place. The original unpacked field is replaced by the new packed field; therefore, the ZP parameter must be omitted from subsequent references to that field. If the field is needed in its original unpacked form, it must be referenced before the use of the ZP parameter.

If *conversion* is specified in the FIELD parameter, the length of the output record can be calculated for each conversion specification. When L is equal to the length of the input record, the calculation is made as follows:

- For a PZ (packed-to-unpacked) specification, 2L-1.
- For a ZP (unpacked-to-packed) specification, (L/2) + C. If L is an odd number, C is 1/2; if L is an even number, C is 1.
- For an HE (H-set BCDIC to EBCDIC) specification, L.
- For the DBCS conversion codes, the shift-out/shift-in characters account for one byte each. If you add or delete them, account for the additional bytes.

output-location
specifies the starting location of this field in the output records. *Output-location* should be coded as a whole decimal number.

If *output-location* is not specified, the location defaults to 1.

LABELS=*n*
is an optional parameter that indicates the number of records in the SYSIN data set to be treated as user labels. The number *n*, which is a number from 0 to 8, must specify the exact number of label records that follow the RECORD statement. If this parameter is included, DATA=INPUT must be coded on a LABELS statement before it in the input stream.

IEBGENER Examples

The examples that follow illustrate some of the uses of IEBGENER. You can use Table 32 on page 164 as a quick-reference guide to IEBGENER examples. The numbers in the "Example" column refer to the examples that follow.

Table 32. IEBGENER example directory

Operation	Data Set Organization	Device	Comments	Example
PRINT	Sequential	Disk and Printer	Data set is listed on a printer.	"Example 1: Print a Sequential Data Set" on page 164
CONVERT	Sequential to Partitioned	Tape and Disk	Blocked output. Three members are to be created.	"Example 2: Create a Partitioned Data Set from Sequential Input" on page 164
MERGE	Sequential into Partitioned	Disk	Blocked output. Two members are to be merged into existing data set.	"Example 3: Convert Sequential Input into Partitioned Members" on page 165
COPY	Sequential	In-stream and Tape	Blocked output.	"Example 4: In-stream Input, Sequential Data Set to Tape Volume" on page 166
Copy and reblock	Sequential	Disk and Tape	Makes blocked tape copy from disk; explicit buffer request.	"Example 5: Produce Blocked Copy on Tape from Unblocked Disk File" on page 166
COPY–with editing	Sequential	Tape	Blocked output. Data set edited as one record group.	"Example 6: Edit and Copy a Sequential Input Data Set with Labels" on page 167
COPY–with editing	Sequential	z/OS UNIX file to Disk	Blocked output. New record length specified for output data set. Two record groups specified.	"Example 7: Edit and Copy a Sequential z/OS UNIX File to a Sequential Data Set" on page 168
COPY–with DBCS validation	Sequential	Disk	DBCS data is validated and edited before copying.	"Example 8: Edit Double-Byte Character Set Data" on page 169

Examples that use **disk** or **tape** in place of actual device numbers must be changed before use. The actual device numbers depend on how your installation has defined the devices to your system.

Example 1: Print a Sequential Data Set

In this example, a sequential data set is printed. The printed output is left-aligned, with one 80-byte record appearing on each line of printed output.

```
//PRINT    JOB  ...
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=A
//SYSIN    DD  DUMMY
//SYSUT1   DD  DSNAME=D80.DATA,DISP=SHR
//SYSUT2   DD  SYSOUT=A
```

The job control statements are as follows:

- SYSIN DD defines a dummy data set. Since no editing is performed, no utility control statements are required.
- SYSUT1 DD defines the input sequential data set.
- SYSUT2 DD indicates that the output is to be written on the system output device (printer). IEBGENER copies LRECL and RECFM from the SYSUT1 data set and the system determines a BLKSIZE.

Example 2: Create a Partitioned Data Set from Sequential Input

In this example, a partitioned data set (consisting of three members) is created from sequential input.

```
//TAPEDISK JOB  ...
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=A
//SYSUT1   DD  DSNAME=INSET,UNIT=tape,LABEL=(,SL),
//             DISP=(OLD,KEEP),VOLUME=SER=001234
```

```
//SYSUT2   DD  DSNAME=NEWSET,UNIT=disk,DISP=(,KEEP),
//             VOLUME=SER=111112,SPACE=(TRK,(10,5,5)),
//             DCB=(RECFM=FB,LRECL=80,BLKSIZE=2000)
//SYSIN    DD  *
     GENERATE  MAXNAME=3,MAXGPS=2
       MEMBER  NAME=MEMBER1
GROUP1 RECORD  IDENT=(8,'FIRSTMEM',1)
       MEMBER  NAME=MEMBER2
GROUP2 RECORD  IDENT=(8,'SECNDMEM',1)
       MEMBER  NAME=MEMBER3
/*
```

The control statements are as follows:

- SYSUT1 DD defines the input data set (INSET). The data set is the first data set on a tape volume.
- SYSUT2 DD defines the output partitioned data set (NEWSET). The data set is to be placed on a disk volume. Ten tracks of primary space, five tracks of secondary space, and five blocks (256 bytes each) of directory space are allocated to allow for future expansion of the data set. The output records are blocked to reduce the space required by the data set.
- SYSIN DD defines the control data set, which follows in the input stream. The utility control statements are used to create members from sequential input data; the statements do not specify any editing.
- GENERATE indicates that three member names are included in subsequent MEMBER statements and that the IDENT parameter appears twice in subsequent RECORD statements.
- The first MEMBER statement assigns a member name (MEMBER1) to the first member.
- The first RECORD statement (GROUP1) identifies the last record to be placed in the first member. The name of this record (FIRSTMEM) appears in the first eight positions of the input record.
- The remaining MEMBER and RECORD statements define the second and third members. Note that, as there is no RECORD statement associated with the third MEMBER statement, the remainder of the input file will be loaded as the third member.

Example 3: Convert Sequential Input into Partitioned Members

In this example, sequential input is converted into two partitioned members. The newly created members are merged into an existing partitioned data set. User labels on the input data set are passed to the user exit routine.

```
//DISKTODK JOB  ...
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=A
//SYSUT1   DD  DSNAME=INSET,UNIT=disk,DISP=(OLD,KEEP),
//             VOLUME=SER=111112,LABEL=(,SUL)
//SYSUT2   DD  DSNAME=EXISTSET,UNIT=disk,DISP=(MOD,KEEP),
//             VOLUME=SER=111113
     GENERATE  MAXNAME=3,MAXGPS=1
        EXITS  INHDR=ROUT1,INTLR=ROUT2
       MEMBER  NAME=(MEMX,ALIASX)
GROUP1 RECORD  IDENT=(8,'FIRSTMEM',1)
       MEMBER  NAME=MEMY
```

The control statements are as follows:

- SYSUT1 DD defines the input data set (INSET). The input data set, which resides on a disk volume, has standard user labels.
- SYSUT2 DD defines the output partitioned data set (EXISTSET). The members created during this job step are merged into the partitioned data set.
- The SYSIN DD statement is omitted. Because the GENERATE line does not begin with //, the system assumes it is preceded by a //SYSIN DD * line. SYSIN DD defines the control data set, which follows in the input stream. The utility control statements are used to create members from sequential input data; the statements do not specify any editing.A /* at the end of any DD * data set is unnecessary because a JCL statement or end of the job stream marks the end of the input stream data set.
- GENERATE indicates that a maximum of three names and aliases are included in subsequent MEMBER statements and that one IDENT parameter appears in a subsequent RECORD statement.

- EXITS defines the user routines that are to process user labels.
- The first MEMBER statement assigns a member name (MEMX) and an alias (ALIASX) to the first member.
- The RECORD statement identifies the last record to be placed in the first member. The name of this record (FIRSTMEM) appears in the first eight positions of the input record.
- The second MEMBER statement assigns a member name (MEMY) to the second member. The remainder of the input data set is included in this member.

Example 4: In-stream Input, Sequential Data Set to Tape Volume

In this example, an in-stream input, sequential data set is copied to a tape volume.

```
//CDTOTAPE JOB  ...
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=A
//SYSIN    DD  DUMMY
//SYSUT2   DD  DSNAME=OUTSET,UNIT=tape,LABEL=(,SL),
//             DISP=(,KEEP),VOLUME=SER=001234,
//             DCB=(RECFM=FB,LRECL=80,BLKSIZE=2000)
//SYSUT1   DD  *
(in-stream data)
/*
```

The job control statements are as follows:

- SYSIN DD defines a dummy data set. No editing is performed; therefore, no utility control statements are needed.
- SYSUT2 DD defines the output data set, OUTSET. The data set is written to a tape volume with IBM standard labels. The data set is to reside as the first (or only) data set on the volume.
- SYSUT1 DD defines the in-stream data which is actually a JES SYSIN data set. The data set contains no statements.

Example 5: Produce Blocked Copy on Tape from Unblocked Disk File

In this example, a blocked copy on tape is made from an unblocked sequential disk file. Because the disk data set has a relatively small block size, the number of buffers explicitly requested is larger than the default of five. This improves performance by permitting more overlap of reading the SYSUT1 data set with writing the SYSUT2 data set.

```
//COPYJOB  JOB
//STEP1    EXEC PGM=IEBGENER,REGION=318K
//SYSPRINT DD  SYSOUT=A
//SYSIN    DD  DUMMY
//SYSUT1   DD  DSNAME=INPUT,UNIT=disk,
//             DISP=OLD,VOL=SER=X13380,
//             DCB=(BUFNO=20,RECFM=F,LRECL=2000,BLKSIZE=2000)
//SYSUT2   DD  DSNAME=OUTPUT,UNIT=tape,DISP=(NEW,KEEP),
//             DCB=(RECFM=FB,LRECL=2000,BLKSIZE=32000)
```

The job control statements are as follows:

- The EXEC statement names the IEBGENER program and specifies the virtual storage region size required. (Calculation of region size is described in Table 23 on page 121.)
- The SYSIN DD statement is a dummy, since no editing is to be performed.
- The SYSUT1 DD statement identifies an input disk file. Normally, the DCB RECFM, LRECL, and BLKSIZE information should not be specified in the DD statement for an existing disk file because the information exists in the data set label in the VTOC; it is specified in this example to illustrate the contrast with the output data set. The unit and volume serial information could be omitted if the data set were cataloged. The DCB information specifies BUFNO=20 to allow up to twenty blocks to be read with each rotation of the disk, assuming the disk track will hold that many blocks.

- The SYSUT2 DD statement identifies the output tape data set and specifies a block size of 32,000 bytes. The default of five buffers should be enough to keep pace with the input.

Example 6: Edit and Copy a Sequential Input Data Set with Labels

In this example, a sequential input data set is edited and copied.

```
//TAPETAPE JOB  ...
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD  SYSOUT=A
//SYSUT1   DD  DSNAME=OLDSET,UNIT=tape,DISP=(OLD,KEEP),
//             VOLUME=SER=001234,LABEL=(3,SUL)
//SYSUT2   DD  DSNAME=NEWSET,UNIT=tape,DISP=(NEW,PASS),
//             DCB=(RECFM=FB,LRECL=80),
//             VOLUME=SER=001235,LABEL=(,SUL)
//SYSIN    DD  *
     GENERATE  MAXFLDS=3,MAXLITS=11
       RECORD  FIELD=(10,'**********',,1),
               FIELD=(5,1,HE,11),FIELD=(1,'=',,16)
        EXITS  INHDR=ROUT1,OUTTLR=ROUT2
       LABELS  DATA=INPUT
       RECORD  LABELS=2

(first header label record)
(second header label record)

       RECORD   LABELS=2

(first trailer label record)
(second trailer label record)

/*
```

The control statements are as follows:

- SYSUT1 DD defines the sequential input data set (OLDSET). The data set was originally written as the third data set on a tape volume.
- SYSUT2 DD defines the sequential output data set (NEWSET). The data set is written as the first data set on a tape volume. The output records are blocked to reduce the space required by the data set and to reduce the access time required when the data set is subsequently referred to. The BLKSIZE parameter is omitted so that the system will calculate an optimal value that is less than or equal to 32,760 bytes unless the system programmer sets the default differently. The data set is passed to a subsequent job step. The LABEL=(,SUL) is required because of the user labels created.
- SYSIN DD defines the control data set, which follows in the input stream.
- GENERATE indicates that a maximum of three FIELD parameters is included in subsequent RECORD statements and that a maximum of 11 literal characters are included in subsequent FIELD parameters.
- The first RECORD statement controls the editing, as follows: asterisks are placed in positions 1 through 10; positions 1 through 5 of the input record are converted from H-set BCDIC to EBCDIC mode and moved to positions 11 through 15; and an equal sign is placed in position 16.
- EXITS indicates that the specified user routines require control when SYSUT1 is opened and when SYSUT2 is closed.
- LABELS indicates that labels are included in the input stream.
- The second RECORD statement indicates that the next two records from SYSIN should be written out as user header labels on SYSUT2.
- The third RECORD statement indicates that the next two records from SYSIN should be written as user trailer labels on SYSUT2.

This example shows the relationship between the RECORD LABELS statement, the LABELS statement, and the EXITS statement. IEBGENER tries to write a first and second label trailer as user labels at close time of SYSUT2 before returning control to the system; the user routine, ROUT2, can review these records and change them, if necessary.

Related reading: For more information, see “EXEC Statement” on page 154.

Example 7: Edit and Copy a Sequential z/OS UNIX File to a Sequential Data Set

In this example,a z/OS UNIX System Services (z/OS UNIX) file is edited and copied. The logical record length of the output data set is less than that of the input data set.

```
//DISKDISK JOB  ...
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=A
//SYSUT1 DD PATH='/dist3/stor44/sales.mon',FILEDATA=TEXT,PATHOPTS=ORDONLY,
//          LRECL=100,BLKSIZE=1000,RECFM=FB
//SYSUT2   DD DSNAME=NEWSET,UNIT=disk,DISP=(NEW,KEEP),
//            VOLUME=SER=111113,DCB=(RECFM=FB,LRECL=80,
//            BLKSIZE=640),SPACE=(TRK,(20,10))
//SYSIN    DD *
     GENERATE MAXFLDS=4,MAXGPS=1
        EXITS IOERROR=ERRORRT
GRP1 RECORD IDENT=(8,'FIRSTGRP',1),FIELD=(21,80,,60),FIELD=(59,1,,1)
GRP2 RECORD FIELD=(11,90,,70),FIELD=(69,1,,1)
/*
```

The control statements are as follows:

- SYSUT1 DD defines the input file. Its name is `/dist3/stor44/sales.mon`. It contains text in 100–byte records. The record delimiter is not stated here. The file might be on a non-System/390 system that is available via Network File System (NFS).
- SYSUT2 DD defines the output data set (NEWSET). Twenty tracks of primary storage space and ten tracks of secondary storage space are allocated for the data set on a disk volume. The logical record length of the output records is 80 bytes, and the output is blocked.
- SYSIN DD defines the control data set, which follows in the input stream.
- GENERATE indicates that a maximum of four FIELD parameters are included in subsequent RECORD statements and that one IDENT parameter appears in a subsequent RECORD statement.
- EXITS identifies the user routine that handles input/output errors.

Figure 26 on page 168 shows how a sequential input data set is edited and copied.

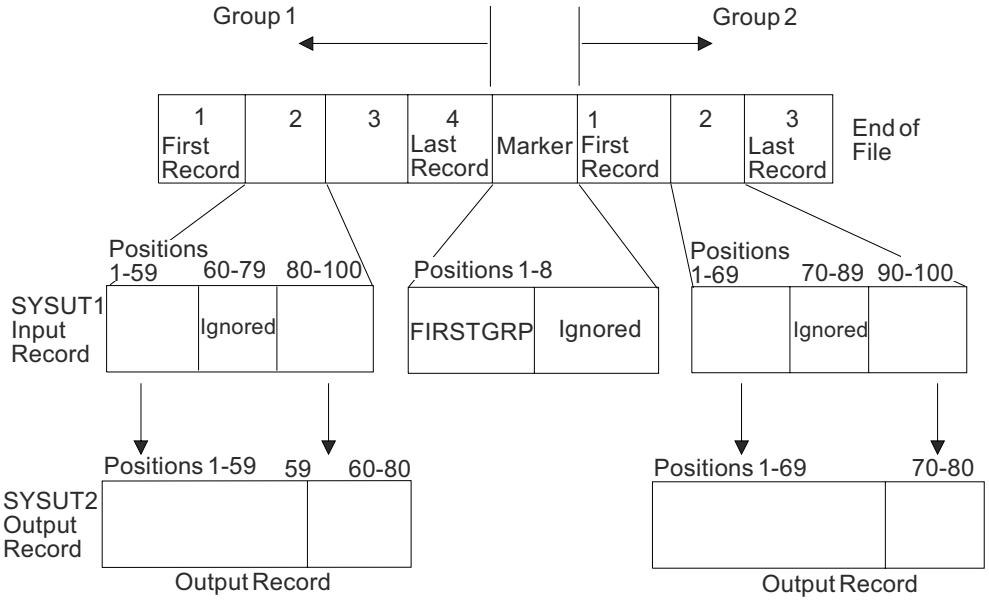


Figure 26. How a sequential data set is edited and copied

- The first RECORD statement (GRP1) controls the editing of the first record group. FIRSTGRP, which appears in the first eight positions of an input record, is defined as being the last record in the first group of records. The data in positions 80 through 100 of each input record are moved into positions 60 through 80 of each corresponding output record. (This example implies that the data in positions

60 through 79 of the input records in the first record group are no longer required; thus, the logical record length is shortened by 20 bytes.) The data in the remaining positions within each input record are transferred directly to the output records, as specified in the second FIELD parameter.

- The second RECORD statement (GRP2) indicates that the remainder of the input records are to be processed as the second record group. The data in positions 90 through 100 of each input record are moved into positions 70 through 80 of the output records. (This example implies that the data in positions 70 through 89 of the input records from group 2 are no longer required; thus, the logical record length is shortened by 20 bytes.) The data in the remaining positions within each input record are transferred directly to the output records, as specified in the second FIELD parameter.

Example 8: Edit Double-Byte Character Set Data

In this example, an edited data set containing double-byte character set data is created. Shift-out/shift-in characters (< and >) are inserted to enclose the DBCS strings.

```
//DBLBYTE  JOB  ...
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=A
//SYSUT1   DD DSN=INPUT,DISP=(OLD,KEEP),UNIT=disk
//SYSUT2   DD DSN=OUTPUT,UNIT=disk,DISP=(NEW,CATLG),
//            DCB=(LRECL=80,BLKSIZE=3200,RECFM=FB),SPACE=(TRK,(1,1))
//SYSIN    DD *
     GENERATE MAXFLDS=4,MAXLITS=9,DBCS=YES
     RECORD FIELD=(20,1,,1),FIELD=(16,33,VC,21),
            FIELD=(30,50,VC,39),FIELD=(9,'*********',,72)
/*
```

The control statements are as follows.

- SYSUT1 DD defines the input data set, INPUT, which resides on a disk volume.
- SYSUT2 DD defines the output data set, OUTPUT, which will reside on a disk volume.
- SYSIN DD defines the control data set, which follows in the input stream.
- GENERATE indicates that a maximum of four FIELD parameters and nine literal characters will appear on subsequent RECORD statements, and that the input data set contains DBCS data.
- RECORD specifies how input records will be edited before being placed in the output data set. The first FIELD parameter indicates that the first 20 positions (bytes) of the input records are to be placed in the first 20 positions of the output records.
- The second FIELD parameter indicates that data in positions 33 through 48 are to be checked to ensure that they are valid DBCS data, and that shift-out/shift-in characters are to be inserted around this field. For DBCS data to be valid, each byte of the 2-byte characters must have a hexadecimal value between X'41' and X'FE', or the 2-byte character must be a DBCS space (X'4040'). Once the checking and inserting are completed, this field is to be copied to the output records beginning at position 21.
- The third FIELD parameter operates on the 30-byte field beginning at position 50 in the input records. This field is checked for valid DBCS data, and shift-out/shift-in characters are inserted around the field. The resulting field is copied to the output records beginning at position 39.

Notice that in specifying the output locations in the FIELD parameter, you have to account for the additional positions that the SO/SI characters will use. For instance, the eight-character (16-byte) DBCS string beginning at position 21 does not end at position 36, but at 38. The SO/SI characters are single-byte characters, so the pair will take up two positions.

The final FIELD parameter clears out the final positions of the output records with asterisks.